



DEFINITIVE REFERENCE

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF 9 Master Bible

The definitive 294-page reference covering every aspect of VCF 9 architecture, deployment, operations, and advanced configurations.

294 PAGES

ARCHITECTURE

DEPLOYMENT

ADVANCED CONFIG

VCF 9.0

VMware Cloud Foundation

© 2026 Virtual Control LLC — All Rights Reserved

PROPRIETARY & CONFIDENTIAL

VCF 9.0

MASTER BIBLE

Complete Reference for VMware Cloud Foundation 9.0.1

Architecture | Deployment | Operations | NSX | vSAN | Security | Troubleshooting | Commands | Disaster
Recovery

Nested Lab Environment — VMware Workstation

lab.local | 192.168.1.0/24 | 4 ESXi Hosts | vCenter | NSX 9.0 | vSAN ESA

Generated February 2026

Document Color Guide

Each Part of this bible uses a distinct color theme for quick visual identification:

■■■ Part I	Purple	Architecture & Fundamentals
■■■ Part II	Blue	Deployment Guide
■■■ Part III	Green	Day 2 Operations
■■■ Part IV	Teal	NSX Networking & Security
■■■ Part V	Orange	vSAN Storage
■■■ Part VI	Gold	Security, Certificates & Compliance
■■■ Part VII	Red	Troubleshooting & Recovery
■■■ Part VIII	Slate	Complete Command Reference
■■■ Part IX	Crimson	Disaster Recovery & Health Checks
■■■ Appendices	Indigo	Quick Reference, Ports, Logs, Glossary

How to Use This Bible

- **Building from scratch?** Start with Part I (Architecture) then follow Part II (Deployment) step by step
- **Running Day 2 operations?** Jump to Part III (Operations) for VCF Operations configuration
- **Troubleshooting an issue?** Go to Part VII (Troubleshooting) — use the flowcharts in Section 7.8
- **Recovering from a crash?** Part IX (Disaster Recovery) has the complete crash recovery runbook and environment health checks
- **Need a command?** Part VIII has every CLI command organized by component
- **Quick lookups?** Appendices have IP tables, port numbers, log paths, and glossary

Table of Contents

PART I: Architecture & Fundamentals

1.1 VCF 9.0 Platform Overview

- What is VMware Cloud Foundation?
- Key Value Propositions
- VCF 9.0 vs Previous Versions (5.x)
- Management Domain vs Workload Domain
- Architecture Types

1.2 Component Architecture

- Component Stack Diagram
- SDDC Manager — The Orchestrator
- vCenter Server — Compute Management
- NSX Manager — Networking & Security
- vSAN — Software-Defined Storage
- VCF Operations — Monitoring & Fleet Management
- VCF Installer / Cloud Builder
- Component Interaction Diagram

1.3 Network Architecture

- Network Segments & VLANs
- VMkernel Adapter Layout
- Host IP Assignments
- NSX TEP Configuration
- DNS & NTP Requirements

1.4 Storage Architecture

- vSAN ESA vs OSA Comparison
- Storage Policies
- Nested Lab SSD Configuration

1.5 Licensing Model

- Per-Core Licensing
- Air-Gapped / Offline Activation

1.6 Hardware Requirements

- Production Requirements

- Nested Lab Requirements
- VMware Workstation VMX Settings
- Per-Component VM Resource Allocation

PART II: Deployment Guide

2.1 Prerequisites & Planning

- Complete IP Address Plan
- DNS Records Required
- Pre-Deployment Checklist

2.2 Nested Lab Setup (VMware Workstation)

- VMX Configuration
- Network & Disk Configuration
- SSD Marking and Verification

2.3 Offline Depot Server Setup

- 2.3.1 Required Files
- 2.3.2 Certificate Generation
- 2.3.3 Python HTTPS Server Script
- 2.3.4 Directory Structure
- 2.3.5 Firewall Rules
- 2.3.6 Start Server and Test
- 2.3.7 SDDC Manager Depot Configuration
- 2.3.8 Offline Depot Troubleshooting

2.4 VCF Installer / Cloud Builder

- OVA Deployment
- vSAN ESA HCL Bypass
- Wizard Configuration
- JSON Configuration File

2.5 Manual Component Deployment via ovftool

- 2.5.1 VCF Operations Deployment
- 2.5.2 NSX Manager Deployment
- 2.5.3 Aria Suite Lifecycle Deployment
- 2.5.4 Probing OVA Property Names

2.6 SDDC Manager Bringup

- Bringup Process

Validation Fixes

Management Domain Creation

2.7 Post-Deployment Verification (VDT)

VDT Download & Upload

Running VDT

Common VDT Failures and Fixes

PART III: Day 2 Operations

3.1 VCF Operations First Login & Setup

3.1.1 Environment Reference

3.1.2 Initial Access

3.1.3 Navigation Overview

3.1.4 Initial Setup Wizard

3.1.5 CEIP Opt-Out

3.2 License Registration (Air-Gapped)

3.2.1 Download Registration File

3.2.2 Upload to Business Services Console

3.2.3 Import License File

3.2.4 Verification

3.2.5 Ongoing License Reporting (180 Days)

3.3 Fleet Management & Depot Configuration

3.3.1 Fleet Management Appliance Registration

3.3.2 Configure Offline Depot for Management

3.3.3 Verify Depot Connection

3.3.4 Configure Offline Depot for SDDC Manager

3.3.5 Bundle Management & Update Scheduling

3.4 Data Source Connections (VCF Cloud Account)

3.4.1 Add VMware Cloud Foundation Account

3.4.2 Start Data Collection

3.4.3 What Happens Automatically

3.4.4 Add Individual vCenter Account

3.4.5 Verify Data Collection

3.5 SSO / Identity & Access Management

3.5.1 Configure VCF Single Sign-On

- 3.5.2 Verify Authentication Source
- 3.5.3 Import Directory Users and Groups
- 3.5.4 Assign Permissions
- 3.5.5 Add Active Directory Identity Source

3.6 Certificate Management

- 3.6.1 View All Certificates
- 3.6.2 Configure Microsoft CA
- 3.6.3 Configure OpenSSL CA
- 3.6.4 Replace Default Certificates
- 3.6.5 Enable Automatic Renewal

3.7 Password Management & Rotation

- 3.7.1 View Password Status
- 3.7.2 Managed Components and Accounts
- 3.7.3 Password Functions Reference
- 3.7.4 Update a Password
- 3.7.5 Rotate Passwords
- 3.7.6 Remediate Passwords

3.8 Compliance Monitoring

- 3.8.1 Access Compliance
- 3.8.2 Activate SDDC Benchmarks
- 3.8.3 Activate Regulatory Benchmarks
- 3.8.4 Configure Drift Detection

3.9 Alerts, Notifications & Dashboards

- 3.9.1 Outbound Notification Plug-Ins
- 3.9.2 Standard Email Plug-In
- 3.9.3 Other Available Plug-Ins
- 3.9.4 Create Notification Rules
- 3.9.5 Key Predefined Dashboards
- 3.9.6 Create a Custom Dashboard

3.10 Backup Configuration

- 3.10.1 Fleet-Level Backups
- 3.10.2 Instance-Level Backups

3.11 VCF Operations for Logs

- 3.11.1 Deploy via Fleet Management

- 3.11.2 Integrate with VCF Operations
- 3.11.3 Enable Log Collection
- 3.11.4 Configure SDDC Manager Log Forwarding
- 3.12 SDDC-to-VCF-Ops Task Migration Reference
 - 3.12.1 Known Issues (VCF Operations 9.0.1)
 - 3.12.2 Post-Configuration Verification Checklist

PART IV: NSX Networking & Security

4.1 NSX Manager Setup

- 4.1.1 NSX Architecture Overview
- 4.1.2 Deployment Sizing for Nested Environments
- 4.1.3 Manual Deployment via ovftool
- 4.1.4 VIP Configuration
- 4.1.5 DNS/NTP via Admin CLI
- 4.1.6 Register Compute Manager
- 4.1.7 Joining with SDDC Manager
- 4.1.8 Initial Verification

4.2 Transport Node Configuration

- 4.2.1 Transport Zone Architecture
- 4.2.2 Transport Node Profile Creation
- 4.2.3 vmk0 TEP Configuration (NSX 9.0)
- 4.2.4 Apply Transport Node Profile to Cluster
- 4.2.5 Troubleshooting Transport Node Failures
- 4.2.6 Verification Commands

4.3 Segments, Routing & Security Policies

- 4.3.1 Segment Types
- 4.3.2 Creating an Overlay Segment
- 4.3.3 Creating a VLAN-Backed Segment
- 4.3.4 Tier-0 and Tier-1 Gateway Concepts
- 4.3.5 Distributed Firewall (DFW)
- 4.3.6 Creating DFW Rules
- 4.3.7 Tag-Based Security Groups

4.4 NSX Monitoring Integration

- 4.4.1 Automatic Discovery via VCF Account
- 4.4.2 Verify NSX Adapter Status

- 4.4.3 NSX Monitoring Features
- 4.4.4 VCF Operations for Networks (Advanced)
- 4.4.5 Key NSX Metrics to Monitor

4.5 NSX Troubleshooting Quick Reference

- 4.5.1 OOM Issues in Nested Environments
- 4.5.2 Transport Node Connectivity Issues
- 4.5.3 Service Status Checks
- 4.5.4 NSX Certificate Replacement
- 4.5.5 Common NSX Shell Commands
- 4.5.6 ESXi Host NSX Service Commands
- 4.5.7 NSX Port Requirements
- 4.5.8 Traceflow for Network Diagnostics

PART V: vSAN Storage

5.1 vSAN ESA Configuration

- 5.1.1 ESA vs OSA Comparison
- 5.1.2 vSAN ESA Bypass for Nested Environments
- 5.1.3 SSD Detection and SATP Claim Rules
- 5.1.4 Storage Policy Creation
- 5.1.5 vSAN Datastore Verification

5.2 Disk Management & Cleanup

- 5.2.1 Disk Identification Commands
- 5.2.2 Adding and Removing Disks
- 5.2.3 Disk Group Management (OSA)
- 5.2.4 Orphaned Object Cleanup

5.3 Storage Migration (Thick to Thin)

- 5.3.0 Why SDDC Manager Starts on Local Storage (Bootstrap Constraint)
- 5.3.1 The Problem
- 5.3.2 Solution: vmkfstools Per-Disk Migration
- 5.3.3 Step-by-Step Procedure
- 5.3.4 Post-Migration Verification
- 5.3.5 VM Reconfiguration After Migration

5.4 vSAN Monitoring & Health

- 5.4.1 VCF Operations Integration
- 5.4.2 vSAN Health Check Commands

5.4.3 Key Metrics to Monitor

5.4.4 esxtop for Storage Performance

5.4.5 vSAN Observer

5.5 vSAN Troubleshooting

5.5.1 Common Issues in Nested Environments

5.5.2 Disk Not Detected

5.5.3 vSAN Network Partition

5.5.4 Object Health Issues

5.5.5 Resync Monitoring

5.5.6 Trace Files and Logging

PART VI: Security, Certificates & Compliance

6.1 Certificate Architecture in VCF

6.1.1 How Certificates Flow in VCF

6.1.2 Self-Signed vs CA-Signed Certs

6.1.3 Certificate Lifecycle

6.1.4 Which Components Use Which Certs

6.2 NSX Certificate Replacement (Full Procedure)

6.2.1 OpenSSL Configuration File

6.2.2 Certificate Generation Commands

6.2.3 Import via NSX API

6.2.4 Apply Certificate

6.2.5 Trust Store Updates

6.3 ESXi Certificate Regeneration

6.3.1 When Regeneration Is Needed

6.3.2 Diagnosis

6.3.3 Regeneration Procedure

6.3.4 Update Thumbprints

6.4 Certificate Authority Configuration

6.4.1 Microsoft CA Setup

6.4.2 OpenSSL CA Configuration

6.4.3 Certificate Templates

6.5 Password Management & Rotation

6.5.1 SDDC Manager Password Management

- 6.5.2 Password Rotation Procedures
- 6.5.3 Default Passwords Reference
- 6.5.4 Password Policy Configuration

6.6 Compliance Monitoring

- 6.6.1 Available Frameworks
- 6.6.2 Enabling Compliance in VCF Operations
- 6.6.3 Compliance Dashboard
- 6.6.4 Remediation Workflows

6.7 Java Keystore Reference

- 6.7.1 Trust Store Paths and Passwords
- 6.7.2 Common keytool Operations

PART VII: Troubleshooting & Recovery

7.1 VDT (VMware Deployment Toolkit)

- 7.1.1 Download from Broadcom
- 7.1.2 Upload to SDDC Manager
- 7.1.3 Installation
- 7.1.4 Running VDT
- 7.1.5 Interpreting Results
- 7.1.6 Common VDT Failures and Fixes

7.2 SDDC Manager Troubleshooting

- 7.2.1 Service Management
- 7.2.2 Log Locations
- 7.2.3 Timeout Loop Issues
- 7.2.4 NFS Mount Issues
- 7.2.5 SSH Quirks
- 7.2.6 Database Issues
- 7.2.7 API Troubleshooting
- 7.2.8 SoS Diagnostic Bundle

7.3 vCenter Troubleshooting

- 7.3.1 Stuck Deployments
- 7.3.2 PostgreSQL Issues
- 7.3.3 Service Management
- 7.3.4 VPXD Issues
- 7.3.5 Deployment Failure Reference Tokens

7.3.6 Cleanup After Failed Deployment

7.4 vMotion Troubleshooting

7.4.1 vhv.enable Ghost Setting

7.4.2 Memory Convergence Failures

7.4.3 EVC Compatibility Issues

7.4.4 Network Troubleshooting for vMotion

7.5 NSX Troubleshooting

7.5.1 OOM in Nested Environments

7.5.2 Transport Node Issues

7.5.3 Certificate Problems

7.5.4 Service Status Checks

7.5.5 Manager Cluster Issues

7.5.6 Traceflow for Network Debugging

7.6 Offline Depot Troubleshooting

7.6.1 TLS/FIPS Compatibility

7.6.2 404 Errors

7.6.3 Missing Catalog Entries

7.6.4 Certificate Mismatch

7.6.5 Python HTTPS Server Issues

7.6.6 PVC Missing

7.7 Recovery Procedures

7.7.1 SDDC Manager Recovery

7.7.2 vCenter Recovery

7.7.3 NSX Manager Recovery

7.7.4 Full Environment Cleanup and Redeployment

7.7.5 ESXi Host Recovery

7.7.6 Backup Recommendations

7.8 Troubleshooting Flowcharts

7.8.1 Deployment Failure Flowchart

7.8.2 Certificate Issue Flowchart

7.8.3 Offline Depot Connection Failure

7.8.4 Service Failure Flowchart

7.8.5 vSAN Issue Flowchart

7.8.6 ESXi Certificate Mismatch

7.8.7 vCenter Deployment Stuck

- 7.8.8 vLCM Host Seeding Failure
- 7.8.9 General Problem Identification Tree
- 7.8.10 Common Error Messages Quick Reference
- 7.8.11 Log Locations Quick Reference
- 7.8.12 Critical Port Numbers

PART VIII: Complete Command Reference

8.1 ESXi Commands

- 8.1.1 esxcli Commands
- 8.1.2 vmkfstools Commands
- 8.1.3 Other ESXi Commands (vdq, esxtop, vim-cmd)

8.2 vCenter Commands

8.3 SDDC Manager Commands

8.4 NSX Manager Commands

- 8.4.1 NSX CLI Commands
- 8.4.2 NSX API Commands (curl)

8.5 Certificate Commands

- 8.5.1 OpenSSL Commands
- 8.5.2 Keytool Commands

8.6 Windows / Depot Commands

8.7 API Quick Reference

PART IX: Disaster Recovery & Health Checks

9.1 Windows Update Crash — Incident Summary

- Impact Assessment
- Environment Reference

9.2 Phase 1: vSAN Cluster Recovery

- 9.2.1 Symptoms & Diagnosis
- 9.2.2 vSwitch Failover Policy Fix
- 9.2.3 vSAN Object Resync Verification

9.3 Phase 2: NSX Manager Recovery

- 9.3.1 Service Restart Procedure
- 9.3.2 NSX Verification

9.4 Phase 3: SDDC Manager Recovery

9.4.1 CPU Soft Lockup Diagnosis

9.4.2 Hard Reset via vCenter API

9.4.3 Spring Boot Service Startup

9.4.4 Service Health Verification

9.5 Phase 4: VCF Management Components Deployment Recovery

9.5.1 Deployment Task Status & Fleet CPU Soft Lockup

9.5.2 Resource Contention Mitigation

9.5.3 Fleet Database Investigation

9.5.4 SDDC Manager Task Retry

9.6 Phase 5: VCF Operations Cluster Initialization

9.6.1 HSQldb Reset Procedure

9.6.2 Admin Password Reset

9.6.3 Cluster Initialization & Verification

9.7 Phase 6: VCF Operations Admin Roles & Adapter Configuration

9.7.1 Admin Role Fix via Suite-API

9.7.2 Collector Recovery & Adapter Restart

9.7.3 NSX Adapter Manual Creation

9.7.4 Final Adapter Status

9.8 VCF Environment Health Check Procedure

9.8.1 Pre-Check: Physical/Virtual Host

9.8.2 ESXi Host Health

9.8.3 vSAN Health

9.8.4 vCenter Health

9.8.5 NSX Manager Health

9.8.6 SDDC Manager Health

9.8.7 VCF Operations Health

9.8.8 Fleet (vRSLCM) Health

9.8.9 Complete Health Check Script

9.9 Key Learnings & Common Pitfalls

9.9.1 Service Startup Times Under Load

9.9.2 API Gotchas Reference

APPENDICES

Appendix A: Environment Quick Reference

- A.1.1 Complete IP Address Table
- A.1.2 DNS Records (Forward and Reverse)
- A.1.3 Default Credentials Reference
- A.1.4 VM Resource Specifications
- A.1.5 VMkernel Layout

Appendix B: Port Reference

- B.1 Management Ports
- B.2 vSAN Ports
- B.3 NSX Ports
- B.4 VCF Operations Ports
- B.5 SDDC Manager Ports
- B.6 vMotion and Other Ports

Appendix C: Log File Matrix

- C.1 Log Files by Component
- C.2 Log Files by Issue Type

Appendix D: Glossary and Acronyms

Appendix E: Key Lessons Learned

- E.1 Nested Environment Gotchas
- E.2 Component-Specific Lessons
- E.3 Deployment Best Practices
- E.4 VCF 9.0.1 Component Versions (BOM)
- E.5 Certificate File Locations

Appendix F: Diagnostic Scripts Quick Reference

- F.1 Quick Reference Card
- F.2 Scripts by Category
- F.3 Diagnostic Escalation Path
- F.4 Service Recovery Sequence
- F.5 Troubleshooting Decision Tree
- F.6 Database Repair Quick Reference

Appendix G: Technical Accomplishments & Highlights

- G.1 Infrastructure Built from Scratch
- G.2 Major Problems Diagnosed and Solved (10)
- G.3 Automation & Tooling Built (20 Scripts)
- G.4 Documentation Produced (13 Documents, ~430 Pages)
- G.5 Database-Level Expertise

G.6 Undocumented by Broadcom — 35 Discoveries

G.7 Technical Skills Demonstrated

G.8 Interview Verbal Summary (60–90 Seconds)

Appendix H: Interview Cheat Sheet

H.1 VCF Experience Narrative

H.2 Problem Solving Stories

H.3 Troubleshooting Methodology

H.4 Key Technical Details

H.5 Quick Answer Reference

H.6 Undocumented by Broadcom — 35 Discoveries

H.7 Closing Questions to Ask

Appendix I: SDDC Manager REST API Reference

I.1 API Architecture & Authentication

I.2 Core Endpoints Reference

I.3 Task Lifecycle & Async Operations

I.4 Credentials & Resource Locks

I.5 Python Automation Examples

I.6 Troubleshooting API Issues

Alphabetical Index

Active Directory Identity Source	3.5.5
Air-Gapped License Activation	1.5 , 3.2
Alerts & Notifications	3.9
API Authentication (Bearer Token)	Appendix I
API Endpoint Reference (SDDC Manager)	Appendix I
API Quick Reference	8.7
API Task Lifecycle	Appendix I
Aria Suite Lifecycle Deployment	2.5.3
Backup Configuration	3.10 , 7.7.6
Bringup Process	2.6
Certificate Architecture	6.1
Certificate Authority (Microsoft CA)	3.6.2 , 6.4.1
Certificate Authority (OpenSSL)	3.6.3 , 6.4.2
Certificate Commands (keytool)	6.7 , 8.5.2
Certificate Commands (openssl)	6.2.2 , 8.5.1
Certificate Mismatch	7.6.4
Certificate Replacement (NSX)	6.2 , 4.5.4
Certificate Troubleshooting Flowchart	7.8.2
Cloud Builder / VCF Installer	1.2 , 2.4
Compliance Monitoring	3.8 , 6.6
Component Architecture	1.2
Credential Cascade Failure	7.2.6 , Appendix G
Credentials Reference	A.1.3

Custom Dashboards	3.9.6
Data Source Connections	3.4
Deployment Failure Flowchart	7.8.1
Diagnostic Scripts	Appendix F
Disk Management (vSAN)	5.2
Distributed Firewall (DFW)	4.3.5 , 4.3.6
DNS Records	1.3 , 2.1 , A.1.2
Drift Detection	3.8.4
ESXi Certificate Regeneration	6.3
ESXi Commands	8.1
ESXi Host Recovery	7.7.5
esxcli Commands	8.1.1
esxtop	5.4.4 , 8.1.3
EVC Compatibility	7.4.3
Fleet Management	3.3
Flowcharts (All)	7.8
Full Cleanup & Redeployment	7.7.4
Glossary	Appendix D
Hardware Requirements	1.6
Interview Cheat Sheet	Appendix H
IP Address Plan	2.1 , A.1.1
Java Keystore	6.7
JSON Configuration File	2.4
keytool Commands	6.7.2 , 8.5.2
License Registration	3.2
Licensing Model	1.5

Log File Matrix	Appendix C
Log Forwarding (SDDC Manager)	3.11.4
Management Domain	1.1 , 2.6
Memory Convergence (vMotion)	7.4.2
Network Architecture	1.3
NFS Mount Issues	7.2.4
NSX API Commands	8.4.2
NSX CLI Commands	8.4.1
NSX Manager Recovery	7.7.3
NSX Manager Setup	4.1
NSX Monitoring	4.4
NSX OOM Issues	4.5.1 , 7.5.1
NSX Port Requirements	4.5.7 , A.2.3
NSX Troubleshooting	4.5 , 7.5
Offline Depot Setup	2.3
Offline Depot Troubleshooting	7.6
OpenSSL Configuration	6.2.1 , 6.4.2
Orphaned Object Cleanup	5.2.4
ovftool Deployment	2.5
OVA Property Names	2.5.4
Password Management	3.7 , 6.5
Password Rotation	3.7.5 , 6.5.2
Port Reference	Appendix B
PAGER=cat (psql)	7.2.6 , 8.3
PostgreSQL (SDDC Manager)	7.2.6 , 8.3
PostgreSQL Issues (vCenter)	7.3.2

Python HTTPS Server	2.3.3 , 7.6.5
Recovery Procedures	7.7
SATP Claim Rules	5.1.3
SDDC Manager API Handbook	Appendix I
SDDC Manager Bootstrap (Local Storage)	5.3.0
SDDC Manager Commands	8.3
SDDC Manager Recovery	7.7.1
SDDC Manager SSH	7.2.5
SDDC Manager Troubleshooting	7.2
task_metadata (platform DB)	7.2.6 , Appendix F
Technical Accomplishments	Appendix G
Undocumented by Broadcom (35 Discoveries)	G.6
Segments (NSX)	4.3
Service Failure Flowchart	7.8.4
SoS Diagnostic Bundle	7.2.8
SSO Configuration	3.5
Storage Architecture	1.4
Storage Migration (Thick→Thin)	5.3
TEP Configuration (vmk0)	4.2.3
Tier-0/Tier-1 Gateways	4.3.4
Timeout Loop Issues	7.2.3
TLS/FIPS Compatibility	7.6.1
Traceflow	4.5.8 , 7.5.6
Transport Node Configuration	4.2
Transport Node Troubleshooting	4.2.5 , 7.5.2
Trust Store Updates	6.2.5

vCenter Commands	8.2
vCenter Deployment Stuck	7.3.1 , 7.8.7
vCenter Recovery	7.7.2
vCenter Troubleshooting	7.3
VCF Cloud Account	3.4.1
VCF Installer	2.4
VCF Operations First Login	3.1
VCF Operations for Logs	3.11
VDT (Deployment Toolkit)	2.7 , 7.1
vhv.enable Ghost Setting	7.4.1
vLCM Host Seeding Failure	7.8.8
vMotion IP Assignments	2.1
vMotion Troubleshooting	7.4
vmkfstools Commands	5.3 , 8.1.2
VMkernel Layout	1.3 , A.1.5
VMware Workstation VMX Settings	1.6 , 2.2
VMX Configuration	2.2
VPXD Issues	7.3.4
vSAN ESA Configuration	5.1
vSAN ESA vs OSA	1.4 , 5.1.1
vSAN Health Check	5.4.2
vSAN Issue Flowchart	7.8.5
vSAN Monitoring	5.4
vSAN Observer	5.4.5
vSAN Troubleshooting	5.5
Windows / Depot Commands	8.6

Workload Domains	1.1
------------------	---------------------

PART I: Architecture & Fundamentals

1.1 VCF 9.0 Platform Overview

What is VMware Cloud Foundation?

VMware Cloud Foundation (VCF) is a unified software-defined data center (SDDC) platform that integrates compute virtualization (vSphere/ESXi), software-defined networking (NSX), software-defined storage (vSAN), and centralized lifecycle management (SDDC Manager) into a single, validated, and automated stack. VCF delivers a turnkey private cloud that can be deployed, operated, and upgraded as a cohesive unit rather than managing individual VMware products separately.

Key Value Propositions

- **Unified Operations Interface** -- Single pane of glass for building, operating, and securing private cloud infrastructure through VCF Operations
- **Automated Lifecycle Management** -- Coordinated patching and upgrades across the entire stack via SDDC Manager
- **Self-Service Cloud Consumption** -- VCF Automation enables self-service provisioning of VMs, Kubernetes clusters, networking (VPC), databases, container registries, and AI workloads
- **Integrated Container & VM Platform** -- Kubernetes and virtualization are integrated out of the box with no separate stack assembly
- **Security & Compliance** -- FIPS 140-2 and 140-3 compliant by default (cannot be disabled), critical for government and regulated industries
- **Fleet Management** -- Manage multiple VCF instances from a single VCF Operations deployment

VCF 9.0 vs Previous Versions (5.x)

Change	VCF 5.x	VCF 9.0
Deployment tool	Cloud Builder	VCF Installer (same OVA as SDDC Manager)
Management UI	SDDC Manager UI (primary)	VCF Operations (SDDC Manager UI deprecated)
Operations suite	Aria Suite (optional)	VCF Operations (mandatory)

Licensing	11 license keys, per-socket	2 keys (per-core + per-TiB), 16-core minimum per CPU
FIPS mode	Optional	Enabled by default, cannot be disabled
NSX availability	Standalone or VCF	VCF only (no standalone NSX)
vSAN default	OSA or ESA	ESA recommended for new deployments
vLCM baselines	Supported	Removed -- must use vLCM Images (desired state)
IWA authentication	Supported	Removed -- use AD over LDAPS or Identity Federation
Host Profiles	Supported	Deprecated -- use vSphere Configuration Profiles
Post-deployment installer	Power off Cloud Builder	VCF Installer transforms into SDDC Manager

Management Domain vs Workload Domain

Management Domain (Required)

- Created during initial VCF deployment (bringup)
- Hosts ALL infrastructure components: SDDC Manager, vCenter, NSX Manager, VCF Operations
- Minimum 4 ESXi hosts
- Should NOT run production workloads
- One management domain per VCF instance

VI Workload Domains (Optional)

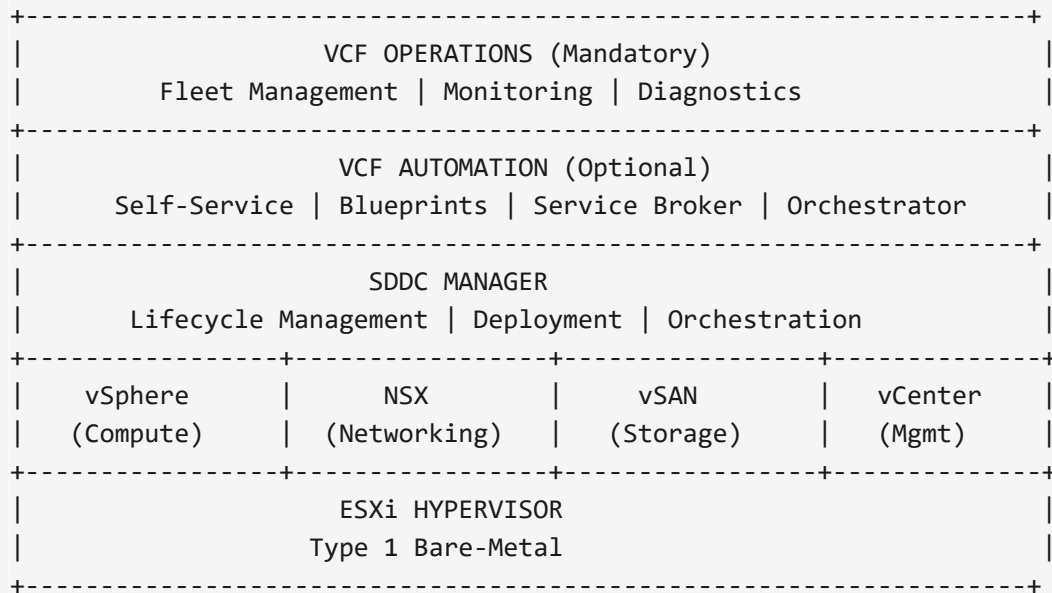
- Created after initial deployment for customer/application workloads
- Has dedicated or shared vCenter
- Shares NSX infrastructure with the management domain (or can have dedicated NSX)
- Minimum 3 ESXi hosts
- Can be added and removed as needed
- Isolates workloads from management infrastructure

Architecture Types

Architecture	Description	Minimum Hosts
Consolidated	Management + Edge services on same hosts	4
Standard	Separate management and edge clusters	3 management + edge hosts

1.2 Component Architecture

Component Stack Diagram



SDDC Manager -- The Orchestrator

SDDC Manager is the central lifecycle management and orchestration platform for VCF. In the lab, it runs at `192.168.1.241` (`sddc-manager.lab.local`).

Attribute	Details
Purpose	Central lifecycle management, deployment, orchestration
Version	9.0.1.0 build 24962180
Key Services	domainmanager, lcm, operationsmanager, commonsvcs, nginx, postgresql
Log Location	<code>/var/log/vmware/vcf/</code>
UI Port	443 (HTTPS)
SSH Access	Only <code>vcf</code> user can SSH in; root access via <code>su -</code> from <code>vcf</code> session
REST API	<code>https://sddc-manager.lab.local/v1/</code>

Key Functions:

- Automated deployment of workload domains
- Coordinated patching and upgrades across the stack
- Certificate management and password rotation
- Health monitoring integration with VCF Operations
- REST API for programmatic automation
- Bundle management (online or offline depot)

Lab lesson: SCP does not work to SDDC Manager due to its restricted shell. Use `ssh vcf@host "cat > file" < localfile` for file transfer instead.

vCenter Server -- Compute Management

vCenter manages all ESXi hosts, VMs, clusters, DRS, HA, and vMotion. In the lab, it runs at `192.168.1.69` (`vcenter.lab.local`).

Attribute	Details
Purpose	Compute virtualization management
Version	9.0.1.0 build 24957454
Key Services	vpxd, vsphere-ui, vmware-postgres, sso (sts), vlcm, eam
Log Location	<code>/var/log/vmware/</code>
UI Port	443 (vSphere Client), 5480 (VAMI)
Resources	4 vCPU, 19GB RAM

Key Functions:

- ESXi host management and cluster configuration
- VM lifecycle operations (create, clone, migrate, delete)
- DRS (Distributed Resource Scheduler) for automatic load balancing
- HA (High Availability) for VM restart on host failure
- vMotion orchestration for live VM migration
- vSphere Lifecycle Manager (vLCM) for ESXi image management

NSX Manager -- Networking & Security

NSX provides software-defined networking, overlay networks, micro-segmentation, and gateway firewalls. In the lab, a single-node NSX Manager runs at `192.168.1.71` (`nsx-node1.lab.local`) with VIP at `192.168.1.70` (`nsx-vip.lab.local`).

Attribute	Details
Purpose	Software-defined networking and security
Version	9.0.1.0 build 24952114
Key Services	proton, corfu, nsx-proxy (on hosts)
Log Location	<code>/var/log/proton/</code>
Cluster Ports	1234 (agent), 1235 (cluster)
Resources	6 vCPU, 32GB RAM (minimum for nested)

Key Concepts:

- **VIP (Virtual IP):** A floating IP (`192.168.1.70`) that provides the management endpoint for the NSX cluster. In HA mode (3 nodes), the VIP moves to the active manager.
- **Transport Zones:** Define the scope of overlay or VLAN segments across hosts.
- **TEP (Tunnel Endpoint):** Where overlay encapsulation (GENEVE) happens on each host. In the lab, vmk0 serves as the TEP using NSX 9.0's "Use VMkernel Adapter" feature.
- **DFW (Distributed Firewall):** Micro-segmentation at the VM vNIC level for east-west traffic.

Lab lesson: NSX Manager `small` deployment needs **32GB RAM and 6 vCPU minimum** in nested environments. 16GB causes kernel OOM, 24GB runs but crashes under load (e.g., transport node deployment).

vSAN -- Software-Defined Storage

vSAN aggregates local disks across ESXi hosts into a shared datastore. In the lab, vSAN ESA runs across all 4 hosts as datastore `vcenter-cl01-ds-vsan01` .

Attribute	Details
Purpose	Software-defined storage
Architecture	ESA (Express Storage Architecture)
Key Services	vsanmgmtd, clomd, vsan-health
Minimum Hosts	3 for cluster, 4 for VCF management domain

Default Policy	RAID-1 (FTT=1)
----------------	----------------

VCF Operations -- Monitoring & Fleet Management

VCF Operations (formerly Aria Operations) provides monitoring, diagnostics, fleet management, and the primary management UI for VCF 9.0. In the lab, it runs at `192.168.1.77` (`vcf-ops.lab.local`).

Attribute	Details
Purpose	Monitoring, diagnostics, fleet management, primary VCF UI
Version	9.0.2.0 build 25137838
Deployment Model	xsmall (Simple -- single node)
Resources	2 vCPU, 8GB RAM

Key Functions:

- Fleet management across multiple VCF instances
- Proactive health monitoring and recommendations
- Governance and compliance dashboards
- Lifecycle import and Cloud Proxy deployment
- Identity broker for SSO (AD/LDAP integration)

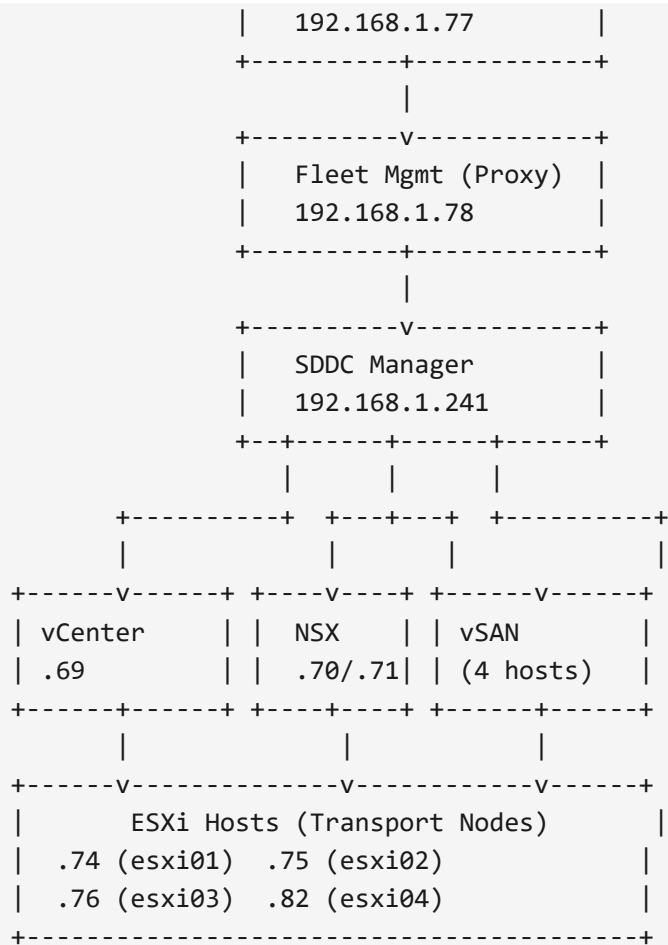
VCF Installer / Cloud Builder

The VCF Installer is new in VCF 9.0 and replaces Cloud Builder from VCF 5.x. The VCF Installer OVA is the **same OVA as SDDC Manager** -- it serves dual purpose. When deployed on the management domain ESXi host, it runs as the installer; after bringup completes, it **transforms into SDDC Manager**.

Aspect	Cloud Builder (5.x)	VCF Installer (9.0)
Purpose	Initial deployment only	Deployment + fleet management
Post-deployment	Power off and archive	Transforms into SDDC Manager
Integration	Standalone	Integrated with VCF Operations

Component Interaction Diagram





1.3 Network Architecture

Network Segments

Network	Purpose	Subnet	MTU	VMkernel
Management	ESXi mgmt, vCenter, SDDC Manager, NSX TEP (overlay)	192.168.1.0/24	1500	vmk0
vMotion	Live VM migration	192.168.11.0/24	9000 (recommended)	vmk1
vSAN	Storage traffic	192.168.12.0/24	9000 (recommended)	vmk2

NSX Hyperbus	NSX internal	169.254.0.0/16	--	vmk50
-----------------	--------------	----------------	----	-------

VMkernel Adapter Layout

VMkernel	TCP/IP Stack	Purpose
vmk0	defaultTcpipStack	Management + NSX TEP (overlay)
vmk1	vmotion	vMotion
vmk2	defaultTcpipStack	vSAN
vmk50	hyperbus	NSX Hyperbus (internal, auto-created)

Host VMkernel IP Assignments

Host	vmk0 (Mgmt/TEP)	vmk1 (vMotion)	vmk2 (vSAN)
esxi01.lab.local	192.168.1.74	192.168.11.121	192.168.12.121
esxi02.lab.local	192.168.1.75	192.168.11.120	192.168.12.120
esxi03.lab.local	192.168.1.76	192.168.11.122	192.168.12.122
esxi04.lab.local	192.168.1.82	192.168.11.123	192.168.12.123

Virtual Switch Topology

In the lab, all networking runs through a single VDS (vSphere Distributed Switch):

```
VDS: vcenter-cl01-vds01
├─ Port Group: vcenter-cl01-vds01-pg-vm-mgmt      (Management)
├─ Port Group: vcenter-cl01-vds01-pg-vmotion      (vMotion)
└─ Port Group: vcenter-cl01-vds01-pg-vsan         (vSAN)
```

Each ESXi VM in VMware Workstation has 4x vmxnet3 adapters in bridged mode. Promiscuous mode is enabled in the VMX file for all NICs (`ethernet*.noPromisc = "FALSE"`) to allow nested VM traffic to flow.

NSX TEP Configuration

NSX 9.0 introduces the **"Use VMkernel Adapter"** option for TEP assignment, which reuses vmk0 (the management VMkernel) as the tunnel endpoint. This eliminates the need for a dedicated TEP VLAN and IP pool -- ideal for nested lab environments.

- **Transport Node Profile:** tn-profile-mgmt
- **VDS:** vcenter-cl01-vds01
- **Transport Zone:** nsx-overlay-transportzone
- **Uplink Profile:** nsx-default-uplink-hostswitch-profile
- **IPv4 Assignment:** "Use VMkernel Adapter" (vmk0)

DNS Requirements

Both forward (A) and reverse (PTR) records are required for ALL VCF components. The DNS server in the lab is a Windows VM at 192.168.1.230 which also serves as the Active Directory domain controller for lab.local .

```
# Forward Records (A)
192.168.1.69      vcenter.lab.local
192.168.1.70      nsx-vip.lab.local
192.168.1.71      nsx-node1.lab.local
192.168.1.74      esxi01.lab.local
192.168.1.75      esxi02.lab.local
192.168.1.76      esxi03.lab.local
192.168.1.82      esxi04.lab.local
192.168.1.77      vcf-ops.lab.local
192.168.1.78      fleet.lab.local
192.168.1.79      collector.lab.local
192.168.1.90      automation.lab.local
192.168.1.94      aria-lifecycle.lab.local
192.168.1.241     sddc-manager.lab.local
```

Important: PTR records (reverse DNS) must also be created for every entry. VCF Installer validation and NSX both require working reverse DNS.

DNS entries NOT needed for Simple Mode deployment:

- nsx-node2, nsx-node3 (only needed for 3-node NSX cluster in HA mode)
- vcf-ops-rep, vcf-ops-data, vcf-ops-lb (only needed for HA VCF Operations)
- automation-node1/2/3/4, automation-upgrade (only needed for HA Automation)

NTP Requirements

All VCF components must synchronize time from the same NTP source. In the lab, **192.168.1.230** serves as both DNS and NTP. NTP configuration on NSX Manager is done via the admin CLI, not the UI:

```
# SSH to NSX Manager as admin
set name-servers 192.168.1.230
set ntp-servers 192.168.1.230
```

1.4 Storage Architecture

vSAN ESA vs OSA Comparison

Feature	vSAN ESA	vSAN OSA
Architecture	Single storage tier (flat pool)	Disk groups (cache + capacity tiers)
Disk Type	NVMe SSDs only	SAS/SATA/NVMe (mixed)
Disk Groups	None	Up to 5 per host, 1 cache + 7 capacity each
Performance	Higher (optimized for flash)	Standard
Compression/Dedup	Higher efficiency	Standard
Minimum Devices	4 NVMe per host	1 cache SSD + 1 capacity per group
Nested Lab Support	Yes (with HCL bypass)	Yes
VCF 9.0 Default	Recommended for new deployments	Supported for existing infrastructure

vSAN ESA in the Nested Lab

The lab uses vSAN ESA across 4 hosts. Because nested virtual disks are not on the VMware HCL, a bypass is required before running the VCF Installer:

```
# SSH to VCF Installer (192.168.1.240) as root
echo "vsan.esa.sddc.managed.disk.claim=true" >>
```

```
/etc/vmware/vcf/domainmanager/application-prod.properties
systemctl restart domainmanager
```

Virtual SATA disks must be marked as SSD in the VMX file:

```
sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
```

Storage Policies

vSAN storage policies define data protection levels using FTT (Failures to Tolerate):

FTT	Can Survive	RAID-1 Min Hosts	RAID-5/6 Min Hosts
1	1 failure	3	4
2	2 failures	5	6
3	3 failures	7	N/A

- **RAID-1 (Mirroring):** Default policy, FTT=1 with 3+ hosts. Full copy of data on separate hosts.
- **RAID-5 (Erasure Coding):** Requires 4+ hosts for FTT=1. More space-efficient than mirroring.
- **RAID-6:** Requires 6+ hosts for FTT=2. vSAN ESA auto-policy with RAID-6 requires minimum 8 hosts.

vSAN Datastore in the Lab

- **Datastore name:** vcenter-cl01-ds-vsan01
- **Hosts contributing:** esxi01, esxi02, esxi03, esxi04
- **Health status:** Green (yellow on network latency, expected for nested)
- **Network latency:** 4-7ms between hosts (normal for nested; physical threshold is 5ms)

1.5 Licensing Model

Simplified Licensing Structure

VCF 9.0 reduces licensing complexity from 11 license keys to just 2:

License Key	Purpose	Model
VMware Cloud Foundation	Compute licensing	Per-core (16-core minimum per CPU)

VMware vSAN	Storage licensing	Per-terabyte (TiB)
-------------	-------------------	--------------------

Per-Core Licensing Details

- **16-core minimum per CPU** -- even if the CPU has fewer physical cores
- Must license ALL physical cores on each host, including BIOS-disabled cores
- Significant change from the previous per-socket model
- Multiple subscriptions pool into a single license; single license file imports all entitlements

VCF Licensing Tiers

Tier	Included Features
VCF Starter	Basic SDDC: vSphere, vSAN, NSX networking
VCF Standard	+ NSX Advanced security (DFW, IDS/IPS), vSAN Enterprise, VCF Operations
VCF Enterprise	+ VCF Automation, Kubernetes support, multi-cloud capabilities

Note: VCF Operations is mandatory across all tiers in VCF 9.0.

Air-Gapped / Offline License Activation

- 90-day evaluation mode is available for initial deployment
- License usage submissions are required every 90 days in connected mode
- For air-gapped environments, license management is handled through VCF Operations connected to the Broadcom Support Portal (or manually via offline activation)
- License keys are registered and applied through VCF Operations Fleet Management

1.6 Hardware Requirements

Production Hardware Requirements

Requirement	Specification
Minimum hosts (mgmt domain)	4 ESXi hosts
Minimum hosts (workload domain)	3 ESXi hosts

CPU	Intel VT-x or AMD-V capable, on VMware HCL
RAM per host	Minimum 256GB (512GB+ recommended)
Storage (vSAN ESA)	4+ NVMe SSDs per host
Storage (vSAN OSA)	1 cache SSD + capacity disks per disk group
Network	2x 25GbE minimum (10GbE supported, 100GbE recommended)
MTU	1600+ for NSX TEP, 9000 for vSAN/vMotion
NIC	On VMware HCL

Nested Lab Requirements (This Lab)

Component	Specification
Physical Host	Dell Precision 7920, 35-core CPU, 192GB RAM
Storage	D: 2TB SSD, E: 2TB SSD, 2x 4TB HDD
Hypervisor	VMware Workstation (latest)
Network Mode	Bridged (all ESXi VMs on same physical network)
Nested ESXi Hosts	4 VMs
DNS/AD Server	Windows VM at 192.168.1.230
Total RAM consumed	~192GB (4x48GB ESXi + management VMs)

ESXi VM Specifications (per host)

Setting	Value
vCPUs	32
Cores per Socket	4
RAM	48GB (49,152 MB)
Network Adapters	4x vmxnet3 (bridged)
Boot Disk	SCSI (pvscsi)
vSAN Disk 1	SATA (sata0:0) -- marked as SSD

vSAN Disk 2	SATA (sata0:2) -- marked as SSD
Guest OS	vmkernel9
Firmware	EFI
Hardware Version	21 (virtualHW.version = "21")

VMware Workstation VMX Settings

The following settings must be added to each ESXi VM's `.vmx` file for nested virtualization to work:

```
# =====
# NESTED VIRTUALIZATION SETTINGS
# =====
vhv.enable = "TRUE"
vpmc.enable = "TRUE"
vvtd.enable = "TRUE"

# =====
# PROMISCUOUS MODE FOR NESTED VM TRAFFIC
# =====
ethernet0.noPromisc = "FALSE"
ethernet0.allowGuestConnectionControl = "TRUE"
ethernet1.noPromisc = "FALSE"
ethernet1.allowGuestConnectionControl = "TRUE"
ethernet2.noPromisc = "FALSE"
ethernet2.allowGuestConnectionControl = "TRUE"
ethernet3.noPromisc = "FALSE"
ethernet3.allowGuestConnectionControl = "TRUE"

# =====
# MARK DISKS AS SSD FOR VSAN
# =====
sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
```

For esxi01 only (has extra disk for local storage): add `sata0:3.virtualSSD = "1"`

VMX file locations:

```
D:\VMs\esxi01.lab.local\esxi01.lab.local.vmx    (D: 2TB SSD)
E:\VMs\esxi02.lab.local\esxi02.lab.local.vmx    (E: 2TB SSD)
```

```
E:\VMs\esxi03.lab.local\esxi03.lab.local.vmx (4TB HDD)
F:\VMs\esxi04.lab.local\esxi04.lab.local.vmx (F: 4TB HDD)
```

VMX Setting Reference

Setting	Purpose
<code>vhv.enable = "TRUE"</code>	Passes VT-x/AMD-V to nested ESXi (required for nested VMs)
<code>vpmc.enable = "TRUE"</code>	Virtual Performance Counters for CPU monitoring
<code>vvtd.enable = "TRUE"</code>	Virtual Intel VT-d (IOMMU) for nested passthrough
<code>ethernet*.noPromisc = "FALSE"</code>	Allows nested VM traffic to flow through VMware Workstation vSwitch
<code>ethernet*.allowGuestConnectionControl</code>	Allows ESXi to control network connections
<code>sata:*.virtualSSD = "1"</code>	Marks virtual SATA disks as SSD for vSAN detection

Per-Component VM Resource Allocation

VM	vCPU	RAM	Storage	Deployed By
vCenter Server	4	19GB	vSAN	VCF Installer
NSX Manager	6	32GB	vSAN (thin)	Manual (ovftool)
SDDC Manager	4	16GB	vSAN (thin, ~108GB used)	VCF Installer bringup
VCF Operations	2	8GB	vSAN (thin)	Manual (ovftool)
Fleet (Cloud Proxy)	2	4GB	vSAN (thin)	VCF Operations import

Windows Host Prerequisites

Hyper-V, VBS, and related features must be disabled on the Windows host for nested virtualization to work:

```
# Run in PowerShell as Administrator
bcdedit /set hypervisorlaunchtype off
Disable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All -
NoRestart
Disable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -
NoRestart
```

```
Disable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-  
Linux -NoRestart
```

Also disable Memory Integrity: Windows Security > Device Security > Core isolation details > Turn OFF "Memory integrity".

REBOOT REQUIRED after these changes.

Verify after reboot:

```
bcdedit /enum | findstr hypervisor  
# Should return nothing or "hypervisorlaunchtype Off"  
  
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace  
root\Microsoft\Windows\DeviceGuard  
# VirtualizationBasedSecurityStatus should be 0
```

PART II: Deployment Guide

2.1 Prerequisites & Planning

Complete IP Address Plan

Component	IP Address	FQDN	Role
esxi01	192.168.1.74	esxi01.lab.local	ESXi Host 1
esxi02	192.168.1.75	esxi02.lab.local	ESXi Host 2
esxi03	192.168.1.76	esxi03.lab.local	ESXi Host 3
esxi04	192.168.1.82	esxi04.lab.local	ESXi Host 4
vCenter	192.168.1.69	vcenter.lab.local	vCenter Server
NSX VIP	192.168.1.70	nsx-vip.lab.local	NSX Manager Virtual IP
NSX Node 1	192.168.1.71	nsx-node1.lab.local	NSX Manager Node
VCF Operations	192.168.1.77	vcf-ops.lab.local	VCF Operations
Fleet (Cloud Proxy)	192.168.1.78	fleet.lab.local	Fleet Management
Collector	192.168.1.79	collector.lab.local	Operations Collector
Automation	192.168.1.90	automation.lab.local	VCF Automation
Aria Lifecycle	192.168.1.94	aria-lifecycle.lab.local	Lifecycle Manager
SDDC Manager	192.168.1.241	sddc-manager.lab.local	SDDC Manager
NSX Manager (SDDC reg)	192.168.1.70	nsx-manager.lab.local	SDDC Manager's registered NSX FQDN
DNS / NTP / AD	192.168.1.230	dc.lab.local	DNS, NTP, Active Directory
Gateway	192.168.1.1	--	Default gateway

Critical: SDDC Manager registers NSX using the FQDN `nsx-manager.lab.local` (mapped to VIP .70). NSX

certificates must include this name in the SAN field, not just `nsx-node1.lab.local`.

vMotion IP Assignments

Host	vMotion IP (vmk1)
esxi01	192.168.11.121
esxi02	192.168.11.120
esxi03	192.168.11.122
esxi04	192.168.11.123

vSAN IP Assignments

Host	vSAN IP (vmk2)
esxi01	192.168.12.121
esxi02	192.168.12.120
esxi03	192.168.12.122
esxi04	192.168.12.123

DNS Records Required

All of the following must have both forward (A) and reverse (PTR) records:

```
# ESXi hosts
192.168.1.74    esxi01.lab.local
192.168.1.75    esxi02.lab.local
192.168.1.76    esxi03.lab.local
192.168.1.82    esxi04.lab.local

# Core infrastructure
192.168.1.69    vcenter.lab.local
192.168.1.70    nsx-vip.lab.local
192.168.1.70    nsx-manager.lab.local
192.168.1.71    nsx-node1.lab.local
192.168.1.241   sddc-manager.lab.local

# VCF Operations ecosystem
```

```
192.168.1.77    vcf-ops.lab.local
192.168.1.78    fleet.lab.local
192.168.1.79    collector.lab.local
192.168.1.90    automation.lab.local
192.168.1.94    aria-lifecycle.lab.local
```

Pre-Deployment Checklist

```
[ ] Physical host: Hyper-V disabled, Memory Integrity off, rebooted
[ ] VMware Workstation installed
[ ] 4 ESXi VMs created with correct specs (32 vCPU, 48GB RAM, 4x vmxnet3)
[ ] VMX files edited with nested virtualization + promiscuous mode + SSD marking
[ ] ESXi 9.0.1 installed on all 4 VMs from VMware ISO
[ ] DNS server running with all A and PTR records
[ ] NTP server accessible from all hosts
[ ] ESXi hosts have only vSwitch0 with vmk0 (clean state)
[ ] ESXi hosts not connected to any vCenter
[ ] SSH enabled on all ESXi hosts
[ ] Nested virtualization verified: cat /proc/cpuinfo | grep -E "vmx|svm"
[ ] SSD status verified: esxcli storage core device list | grep "Is SSD"
[ ] VCF Installer OVA downloaded from Broadcom Support Portal
[ ] Offline depot prepared (if not using online Broadcom depot)
[ ] Common password set on all ESXi hosts (used during VCF Installer wizard)
```

2.2 Nested Lab Setup (VMware Workstation)

VMX Configuration for Each ESXi VM

Each ESXi VM must have the following settings. These go at the END of the `.vmx` file (the VM must be powered off when editing):

```
# =====
# NESTED VIRTUALIZATION SETTINGS
# =====

# Hardware virtualization passthrough
vhv.enable = "TRUE"

# Virtual Performance Counters
vpmc.enable = "TRUE"
```

```
# Virtual VT-d / IOMMU
vvtd.enable = "TRUE"

# =====
# PROMISCUOUS MODE FOR NESTED VM TRAFFIC
# =====

ethernet0.noPromisc = "FALSE"
ethernet0.allowGuestConnectionControl = "TRUE"
ethernet1.noPromisc = "FALSE"
ethernet1.allowGuestConnectionControl = "TRUE"
ethernet2.noPromisc = "FALSE"
ethernet2.allowGuestConnectionControl = "TRUE"
ethernet3.noPromisc = "FALSE"
ethernet3.allowGuestConnectionControl = "TRUE"

# =====
# MARK DISKS AS SSD FOR VSAN
# =====

sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
```

Network Adapter Configuration

All 4 network adapters should be configured as **vmxnet3** in **Bridged** mode, connected to the same physical NIC as the host's management network. This allows all nested VMs to communicate on the 192.168.1.0/24 subnet.

Disk Configuration for vSAN

Each ESXi VM should have at minimum:

- **Boot disk:** SCSI (pvscsi controller), ~100GB
- **vSAN Disk 1:** SATA (sata0:0), capacity as needed
- **vSAN Disk 2:** SATA (sata0:2), capacity as needed

SSD Marking and Verification

After powering on each ESXi VM, verify SSD detection:

```
# SSH to ESXi host
ssh root@192.168.1.74

# Verify nested virtualization is working
cat /proc/cpuinfo | grep -E "vmx|svm"
# Should output lines containing "vmx" or "svm"

# Verify disks detected as SSD
esxcli storage core device list | grep -E "Display Name|Is SSD"
# Each vSAN disk should show "Is SSD: true"
```

If disks show as HDD, verify the VMX file has `sata0:0.virtualSSD = "1"` entries and perform a full power cycle (shutdown + power on, not just reboot).

2.3 Offline Depot Server Setup

For air-gapped or lab environments without direct internet access, an offline depot server provides VCF binaries to the SDDC Manager / VCF Installer over HTTPS.

2.3.1 Required Files

Download the following from the Broadcom Support Portal:

Metadata (required):

- `vcf-9.0.1.0-offline-depot-metadata.zip` -- Contains the PROD directory structure with manifest, product version catalog, vSAN HCL data, and compatibility data

Appliances and Binaries:

File	Component
<code>VCF-SDDC-Manager-Appliance-9.0.1.0.24962180.ova</code>	SDDC Manager
<code>VMware-VCSA-all-9.0.1.0.24957454.iso</code>	vCenter Server
<code>nsx-unified-appliance-9.0.1.0.24952114.ova</code>	NSX Manager
<code>VCF-OPS-Lifecycle-Manager-Appliance-9.0.1.0.24960371.ova</code>	Aria Lifecycle
<code>Operations-Appliance-9.0.1.0.24960351.ova</code>	VCF Operations

Operations-Cloud-Proxy-9.0.1.0.24960349.ova	Operations Cloud Proxy
011N_VA-9.0.1.0.24923009.ova	Orchestrator
vmosp-vcfa-combined-9.0.1.0.24965341.tar	VCF Automation
VmwareCompatibilityData.json	Compatibility data

2.3.2 Certificate Generation

Generate a self-signed TLS certificate for the depot server. Run on the Windows depot server (requires OpenSSL -- included with Git for Windows):

```
openssl req -x509 -newkey rsa:2048 `
  -keyout "C:\VCF-Depot\server.key" `
  -out "C:\VCF-Depot\server.crt" `
  -days 365 -nodes `
  -subj "/CN=192.168.1.52" `
  -addext "subjectAltName=IP:192.168.1.52"
```

Important: The SAN must include the IP address that SDDC Manager will use to connect. If using a hostname, add a DNS entry as well.

2.3.3 Python HTTPS Server Script

Save the following as `C:\VCF-Depot\https_server.py` :

```
#!/usr/bin/env python3
"""
HTTPS server for VCF Offline Depot
Serves files with TLS 1.2+ for SDDC Manager compatibility
"""

import http.server
import ssl
import os
import base64
import socketserver
from functools import partial
```

```

# Configuration
PORT = 8443
CERT_FILE = 'server.crt'
KEY_FILE = 'server.key'
USERNAME = 'admin'
PASSWORD = 'admin'

class AuthHandler(http.server.SimpleHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def __init__(self, *args, directory=None, **kwargs):
        super().__init__(*args, directory=directory, **kwargs)

    def do_HEAD(self):
        if not self.authenticate():
            return
        super().do_HEAD()

    def do_GET(self):
        if not self.authenticate():
            return
        super().do_GET()

    def authenticate(self):
        auth_header = self.headers.get('Authorization')
        if auth_header is None:
            self.send_auth_request()
            return False

        try:
            auth_type, credentials = auth_header.split(' ', 1)
            if auth_type.lower() != 'basic':
                self.send_auth_request()
                return False

            decoded = base64.b64decode(credentials).decode('utf-8')
            username, password = decoded.split(':', 1)

            if username == USERNAME and password == PASSWORD:
                return True
        except Exception:
            pass

        self.send_auth_request()
        return False

```

```

def send_auth_request(self):
    self.send_response(401)
    self.send_header('WWW-Authenticate', 'Basic realm="VCF Depot"')
    self.send_header('Content-type', 'text/html')
    self.send_header('Content-Length', '23')
    self.send_header('Connection', 'close')
    self.end_headers()
    self.wfile.write(b'Authentication required')

def log_message(self, format, *args):
    print(f"{self.client_address[0]} - {format % args}")

class ThreadedHTTPServer(socketserver.ThreadingMixIn, http.server.HTTPServer):
    daemon_threads = True

def run_server():
    os.chdir(os.path.dirname(os.path.abspath(__file__)))

    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    context.maximum_version = ssl.TLSVersion.TLSv1_3

    if hasattr(context, 'post_handshake_auth'):
        context.post_handshake_auth = False

    context.options |= ssl.OP_NO_TICKET
    context.options |= getattr(ssl, 'OP_NO_RENEGOTIATION', 0)
    context.load_cert_chain(CERT_FILE, KEY_FILE)

    try:
        context.set_ciphers('DEFAULT:!aNULL:!MD5:!DSS')
    except ssl.SSLError:
        pass

    handler = partial(AuthHandler, directory=os.getcwd())
    server = ThreadedHTTPServer(('0.0.0.0', PORT), handler)
    server.socket = context.wrap_socket(server.socket, server_side=True)

    print(f"VCF Offline Depot Server")
    print(f"=====")
    print(f"Serving: {os.getcwd()}")
    print(f"URL: https://192.168.1.52:{PORT}/")
    print(f"Credentials: {USERNAME} / {PASSWORD}")

```

```

print(f"TLS: 1.2 - 1.3")
print(f"Press Ctrl+C to stop")

try:
    server.serve_forever()
except KeyboardInterrupt:
    print("\nStopped.")
    server.shutdown()

if __name__ == '__main__':
    run_server()

```

Key server design decisions:

- `OP_NO_RENEGOTIATION` prevents Java TLS renegotiation errors from SDDC Manager
- `HTTP/1.1` protocol version is required for Java clients
- `ThreadingMixIn` handles concurrent requests from SDDC Manager (which makes parallel downloads)
- `DEFAULT:!aNULL:!MD5:!DSS` cipher string provides FIPS-compatible TLS

2.3.4 Directory Structure

Extract the official metadata zip and place binaries in the correct locations:

```

# Extract metadata
Expand-Archive -Path "vcf-9.0.1.0-offline-depot-metadata.zip" -DestinationPath
"C:\VCF-Depot\metadata-extract" -Force
Copy-Item "C:\VCF-Depot\metadata-extract\PROD\*" "C:\VCF-Depot\PROD\" -Recurse -
Force

# Create component directories
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\SDDC_MANAGER_VCF" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VCENTER" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\NSX_T_MANAGER" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VRSLCM" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VROPS" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VCF_OPS_CLOUD_PROXY" -
Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VRA" -Force
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\VRO" -Force

```

```
New-Item -ItemType Directory -Path "C:\VCF-Depot\PROD\COMP\SDDC_MANAGER_VCF\lcm\productVersionCatalog" -Force
```

File placement map:

File	Destination
VCF-SDDC-Manager-Appliance-*.ova	PROD\COMP\SDDC_MANAGER_VCF\
VMware-VCSA-all-*.iso	PROD\COMP\VCENTER\
nsx-unified-appliance-*.ova	PROD\COMP\NSX_T_MANAGER\
VCF-OPS-Lifecycle-Manager-*.ova	PROD\COMP\VRSLCM\
Operations-Appliance-*.ova	PROD\COMP\VROPS\
Operations-Cloud-Proxy-*.ova	PROD\COMP\VCF_OPS_CLOUD_PROXY\
011N_VA-*.ova	PROD\COMP\VRO\
vmosp-vcfa-combined-*.tar	PROD\COMP\VRA\
VmwareCompatibilityData.json	PROD\COMP\SDDC_MANAGER_VCF\Compatibility\
productVersionCatalog.json	PROD\COMP\SDDC_MANAGER_VCF\lcm\productVersionCatalog\

Final directory tree:

```
C:\VCF-Depot\
├─ https_server.py
├─ server.crt
├─ server.key
└─ PROD\
    ├─ metadata\
    │   ├─ manifest\v1\
    │   │   └─ vcfManifest.json
    │   └─ productVersionCatalog\v1\
    │       ├─ productVersionCatalog.json
    │       └─ productVersionCatalog.sig
    ├─ vsan\hcl\
    │   ├─ all.json
    │   └─ lastupdatedtime.json
    └─ COMP\
        ├─ SDDC_MANAGER_VCF\
        │   └─ VCF-SDDC-Manager-Appliance-9.0.1.0.24962180.ova
```

```

|   |   | Compatibility\
|   |   |   | VmwareCompatibilityData.json
|   |   |   | lcm\productVersionCatalog\
|   |   |   |   | productVersionCatalog.json
|   | VCENTER\
|   |   | VMware-VCSA-all-9.0.1.0.24957454.iso
|   | NSX_T_MANAGER\
|   |   | nsx-unified-appliance-9.0.1.0.24952114.ova
|   | VRSLCM\
|   |   | VCF-OPS-Lifecycle-Manager-Appliance-9.0.1.0.24960371.ova
|   | VROPS\
|   |   | Operations-Appliance-9.0.1.0.24960351.ova
|   | VCF_OPS_CLOUD_PROXY\
|   |   | Operations-Cloud-Proxy-9.0.1.0.24960349.ova
|   | VRA\
|   |   | vmosp-vcfa-combined-9.0.1.0.24965341.tar
|   | VRO\
|   |   | 011N_VA-9.0.1.0.24923009.ova

```

2.3.5 Firewall Rules

Allow inbound traffic on port 8443 on the Windows depot server:

```
netsh advfirewall firewall add rule name="Allow 8443 Inbound" dir=in action=allow
protocol=tcp localport=8443
```

Lab lesson: If the Windows network profile is set to "Public", the firewall blocks all inbound connections silently. Change the network profile to "Private" in Windows Settings > Network & Internet > Ethernet > Network profile type.

2.3.6 Start the Server and Test

```
cd C:\VCF-Depot
python https_server.py
```

From SDDC Manager, verify connectivity:

```
curl -k -u admin:admin
https://192.168.1.52:8443/PROD/metadata/productVersionCatalog/v1/productVersionCatalog
```

2.3.7 SDDC Manager Depot Configuration

Import certificate into SDDC Manager trust store:

SSH into SDDC Manager as root:

```
# Pull the depot server certificate
openssl s_client -connect 192.168.1.52:8443 </dev/null 2>/dev/null | openssl x509 >
/tmp/depot.crt

# Find Java cacerts path
CACERTS=$(find /usr -name cacerts 2>/dev/null | head -1)
echo "Truststore: $CACERTS"

# Import certificate
keytool -import -trustcacerts -alias vcf-depot -file /tmp/depot.crt -keystore
$CACERTS -storepass changeit -noprompt

# Restart services to pick up new certificate
systemctl restart commonsvc domainmanager lcm operationsmanager
```

Configure depot in VCF Installer UI:

Field	Value
FQDN or IP Address	192.168.1.52
Port	8443
Username	admin
Password	admin

Click **Configure**. On success, available VCF versions appear in the UI.

2.3.8 Offline Depot Troubleshooting

"Secure protocol communication error"

- Cause: Self-signed certificate not in SDDC Manager Java trust store
- Fix: Import depot cert into Java cacerts (see above), restart services

"Path not found - 404 File not found"

- Cause: Binary not at the expected path
- Fix: Watch HTTPS server console for the exact requested path, place the file at that location under `C:\VCF-Depot\`

"Product Version Catalog (PVC) does not exist"

- Cause: `productVersionCatalog.json` not extracted from official metadata zip, or LCM copy missing
- Fix: Extract from `vcf-9.0.1.0-offline-depot-metadata.zip` ; copy to `PROD\COMP\SDDC_MANAGER_VCF\lcm\productVersionCatalog\`

TLS/FIPS connection issues

- Cause: Server not using TLS 1.2+ or incompatible ciphers
- Fix: Ensure `context.minimum_version = ssl.TLSVersion.TLSv1_2` and `OP_NO_RENEGOTIATION` are set in the server script

2.4 VCF Installer / Cloud Builder

OVA Deployment

1. Browse to `https://192.168.1.74/ui` (esxi01 Host Client)
2. Virtual Machines > Create/Register VM > Deploy OVF
3. Upload: `VCF-SDDC-Manager-Appliance-9.0.1.0.24962180.ova`
4. Configure:
 - IP: `192.168.1.240`
 - FQDN: `vcf-installer.lab.local`
 - Gateway: `192.168.1.1`
 - DNS: `192.168.1.230`
5. Power on and wait for full boot (~5-10 minutes)

vSAN ESA HCL Bypass for Nested Environments

VCF 9.0.1 has a built-in bypass. After the VCF Installer OVA is running:

```
# SSH to VCF Installer as root
ssh root@192.168.1.240

# Add the vSAN ESA HCL bypass
echo "vsan.esa.sddc.managed.disk.claim=true" >>
/etc/vmware/vcf/domainmanager/application-prod.properties

# Restart the domain manager service
systemctl restart domainmanager

# Verify the property was added
cat /etc/vmware/vcf/domainmanager/application-prod.properties | grep vsan
```

VCF Installer Wizard Configuration

1. Browse to `https://vcf-installer.lab.local`
2. Login: `admin@local`
3. Configure software depot:
 - **Online:** Enter Broadcom Support Portal token
 - **Offline:** Enter depot server URL `https://192.168.1.52:8443` with credentials
4. Select: **New Fleet > Simple Model**
5. Follow wizard pages:
 - **vCenter details:** vcenter.lab.local, 192.168.1.69
 - **NSX Manager details:** nsx-manager.lab.local, 192.168.1.70 (VIP), 192.168.1.71 (node)
 - **SDDC Manager details:** auto-populated from installer VM
 - **ESXi host credentials:** common password for all hosts
 - **Network configuration:** management, vMotion, vSAN subnets/VLANs
 - **Storage:** vSAN ESA
6. Run validations (fix any failures before proceeding)
7. Deploy

Lab note: The VCF Installer in Simple Mode deploys vCenter, configures vSAN ESA across all 4 hosts, and creates the VDS. After deployment, the installer OVA transforms into SDDC Manager.

JSON Configuration File Structure

The VCF Installer wizard generates a JSON configuration internally. The key structure contains:

```

{
  "skipEsxThumbprintValidation": true,
  "managementPoolName": "mgmt-pool",
  "ceipEnabled": false,
  "fipsModeEnabled": true,
  "ntpServers": ["192.168.1.230"],
  "dnsSpec": {
    "nameserver": "192.168.1.230",
    "domain": "lab.local"
  },
  "sddcManagerSpec": {
    "hostname": "sddc-manager",
    "ipAddress": "192.168.1.241"
  },
  "networkSpecs": [
    { "networkType": "MANAGEMENT", "subnet": "192.168.1.0/24", "gateway":
"192.168.1.1" },
    { "networkType": "VMOTION", "subnet": "192.168.11.0/24" },
    { "networkType": "VSAN", "subnet": "192.168.12.0/24" }
  ],
  "nsxtSpec": {
    "nsxtManagerSize": "small",
    "nsxtManagers": [
      { "hostname": "nsx-node1", "ip": "192.168.1.71" }
    ],
    "vip": "192.168.1.70",
    "vipFqdn": "nsx-vip.lab.local"
  },
  "vsanSpec": {
    "vsanName": "vcenter-cl01-ds-vsan01",
    "datastoreName": "vcenter-cl01-ds-vsan01",
    "esaEnabled": true
  },
  "hostSpecs": [
    { "hostname": "esxi01.lab.local", "ipAddress": "192.168.1.74" },
    { "hostname": "esxi02.lab.local", "ipAddress": "192.168.1.75" },
    { "hostname": "esxi03.lab.local", "ipAddress": "192.168.1.76" },
    { "hostname": "esxi04.lab.local", "ipAddress": "192.168.1.82" }
  ]
}

```

2.5 Manual Component Deployment via ovftool

In nested lab environments, SDDC Manager's automated deployment often times out. The workaround is to deploy components manually using ovftool directly on the VCF Installer/SDDC Manager CLI.

Key lesson: Always probe an OVA with `ovftool <ova>` first to discover the correct OVF property names. Property names vary between OVAs and are not always documented.

Key lesson: ovftool on VCF Installer/SDDC Manager requires SINGLE-LINE commands. Backslash line continuation breaks `--noSSLVerify` and other flags.

2.5.1 VCF Operations Deployment

```
/usr/bin/ovftool --skipManifestCheck --powerOn --diskMode=thin --acceptAllEulas --
allowExtraConfig --ipProtocol=IPv4 --ipAllocationPolicy=fixedPolicy --noSSLVerify -
-datastore=vcenter-cl01-ds-vsan01 --network=vcenter-cl01-vds01-pg-vm-mgmt --
deploymentOption=xsmall --name=vcf-ops --prop:root_password='Success01!0909!!' --
prop:ipv4_address.VMware_Aria_Operations=192.168.1.77 --
prop:ipv4_type.VMware_Aria_Operations=Static --
prop:domain.VMware_Aria_Operations=vcf-ops.lab.local --
prop:ipv4_gateway.VMware_Aria_Operations=192.168.1.1 --
prop:DNS.VMware_Aria_Operations=192.168.1.230 --
prop:ipv4_netmask.VMware_Aria_Operations=255.255.255.0 --X:waitForIp --overwrite --
X:logFile=/tmp/vcf-ops-manual.log --X:logLevel=verbose /nfs/vmware/vcf/nfs-
mount/bundle/8a3336da-1b81-5144-b43e-d84eae7a8d8f/8a3336da-1b81-5144-b43e-
d84eae7a8d8f/Operations-Appliance-9.0.2.0.25137838.ova
"vi://administrator%40vsphere.local:Success01%210909%21%21@vcenter.lab.local/vcenter-
dc01/host/vcenter-cl01"
```

Warning: SDDC Manager will delete manually deployed VMs it does not recognize if it is in an active deployment loop. Wait for any SDDC Manager deployment tasks to fail completely before deploying manually.

2.5.2 NSX Manager Deployment

```
/usr/bin/ovftool --skipManifestCheck --powerOn --diskMode=thin --acceptAllEulas --
allowExtraConfig --ipProtocol=IPv4 --noSSLVerify --datastore=vcenter-cl01-ds-vsan01
--network=vcenter-cl01-vds01-pg-vm-mgmt --deploymentOption=small --name=nsx-manager
--prop:nsx_role='NSX Manager' --prop:nsx_passwd_0='Success01!0909!!' --
prop:nsx_cli_passwd_0='Success01!0909!!' --
prop:nsx_cli_audit_passwd_0='Success01!0909!!' --prop:nsx_hostname=nsx-
node1.lab.local --prop:nsx_ip_0=192.168.1.71 --prop:nsx_netmask_0=255.255.255.0 --
prop:nsx_gateway_0=192.168.1.1 --prop:nsx_dns1_0=192.168.1.230 --
prop:nsx_domain_0=lab.local --prop:nsx_ntp_0=192.168.1.230 --
prop:nsx_isSSHEnabled=True --prop:nsx_allowSSHRootLogin=True --X:waitForIp --
X:logFile=/tmp/nsx-manager.log --X:logLevel=verbose /nfs/vmware/vcf/nfs-
mount/bundle/028849ee-d3e7-5748-9b90-47d503c6dd3e/028849ee-d3e7-5748-9b90-
47d503c6dd3e/nsx-unified-appliance-9.0.1.0.24952114.ova
"vi://administrator%40vsphere.local:Success01%210909%21%21@vcenter.lab.local/vcenter-
dc01/host/vcenter-cl01"
```

Post-deployment NSX configuration:

1. Wait 15+ minutes for all NSX services to start (MANAGER, SEARCH, UI, NODE_MGMT)
2. Configure VIP: NSX UI > System > Appliances > Set Virtual IP > 192.168.1.70
3. Configure DNS (via admin CLI SSH): set name-servers 192.168.1.230
4. Configure NTP (via admin CLI SSH): set ntp-servers 192.168.1.230
5. Add compute manager: NSX UI > System > Fabric > Compute Managers > Add
vcenter.lab.local

2.5.3 Aria Suite Lifecycle Deployment

```
/usr/bin/ovftool --skipManifestCheck --powerOn --diskMode=thin --acceptAllEulas --
allowExtraConfig --ipProtocol=IPv4 --noSSLVerify --datastore=vcenter-cl01-ds-vsan01
--network=vcenter-cl01-vds01-pg-vm-mgmt --name=aria-lifecycle --
prop:vami.hostname=automation.lab.local --prop:varoot-password='Success01!0909!!' -
--prop:admin-password='Success01!0909!!' --prop:va-ssh-enabled=True --
prop:vami.ip0.VCF_OPS_Management_Appliance=192.168.1.90 --
prop:vami.netmask0.VCF_OPS_Management_Appliance=255.255.255.0 --
prop:vami.gateway.VCF_OPS_Management_Appliance=192.168.1.1 --
prop:vami.DNS.VCF_OPS_Management_Appliance=192.168.1.230 --
prop:vami.domain.VCF_OPS_Management_Appliance=lab.local --X:waitForIp --
X:logFile=/tmp/aria-lifecycle.log --X:logLevel=verbose /nfs/vmware/vcf/nfs-
mount/bundle/7301e3db-1ea7-5dd8-be67-c778bec936/7301e3db-1ea7-5dd8-be67-
c778bec936/VCF-OPS-Lifecycle-Manager-Appliance-9.0.1.0.24960371.ova
```

```
"vi://administrator%40vsphere.local:Success01%210909%21%21@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01"
```

Important: The OVF property names for this appliance use `VCF_OPS_Management_Appliance` as the VM identifier (e.g., `vami.ip0.VCF_OPS_Management_Appliance`). These were discovered by probing the OVA with `ovftool <ova>`. The format is NOT `vami.ip0.VCF-OPS-Lifecycle-Manager` or any other variant.

2.5.4 Probing OVA Property Names

Before deploying any OVA via `ovftool`, probe it to discover the correct property names:

```
/usr/bin/ovftool /path/to/component.ova
```

This outputs all available OVF properties including their correct keys, types, and default values. Use these exact property names in the `--prop:` arguments.

2.6 SDDC Manager Bringup

Bringup Process

After the VCF Installer deploys vCenter and vSAN (Phase 1), or after manually deploying all components, the bringup process registers everything into a management domain.

1. **Access VCF Installer UI:** `https://vcf-installer.lab.local` (or the SDDC Manager IP)
2. **Login:** `admin@local`
3. **Run Bringup Wizard:** Point to existing vCenter, NSX, and ESXi hosts
4. **Validation:** Installer runs 12+ prechecks
5. **Bringup:** Creates management domain "mgmt", registers all components, creates resource pool
6. **Transformation:** VCF Installer transforms into SDDC Manager

Validation Fixes (Common Errors from Lab)

The following validation errors were encountered and fixed during the lab bringup:

Validation Error	Fix
NSX VIP not configured	NSX UI > System > Appliances > Set Virtual IP > 192.168.1.70
Compute manager not found in NSX	NSX UI > System > Fabric > Compute Managers > Add vcenter.lab.local
DNS not configured in NSX	SSH admin@192.168.1.71 > <code>set name-servers 192.168.1.230</code>
NTP not configured in NSX	SSH admin@192.168.1.71 > <code>set ntp-servers 192.168.1.230</code>
DRS not fully automated	vCenter > vcenter-cl01 > Configure > DRS > Fully Automated
VM evacuation policy mismatch	vCenter > vcenter-cl01 > Configure > vSphere Lifecycle Manager > Enable "Migrate powered off and suspended VMs"
Aria Lifecycle IP in use (.94)	Deleted existing VM at .94, let installer redeploy fresh
NSX certificate (EC vs RSA)	Resolved after NSX health stabilized
NSX cluster not stable	Resolved after RAM increase to 32GB
NSX minimum version check	Resolved after NSX services came fully online (9.0.1 > 4.2.1 minimum)

Key lesson: Many installer validation errors are cascading failures from an unhealthy NSX Manager. Fix NSX health first (ensure adequate RAM, wait for all services to start) and most other errors resolve automatically.

Management Domain Creation

After passing all validations, bringup creates the management domain:

- **Domain name:** mgmt
- **Status:** ACTIVE
- **Components registered:** vCenter (.69), NSX (.70/.71), 4 ESXi hosts
- **Resource pool:** Created in vcenter-cl01, all management VMs moved into it
- **VCF Installer:** Transformed into SDDC Manager at 192.168.1.241

Post-Bringup Verification

```
# SSH to SDDC Manager as vcf, then su - to root
ssh vcf@192.168.1.241
su -
```

```
# Check all SDDC Manager services
systemctl status vcf-services

# Check individual critical services
systemctl status domainmanager
systemctl status lcm
systemctl status operationsmanager
systemctl status nginx
systemctl status postgresql

# Verify management domain via API
curl -k -X POST https://localhost/v1/tokens -H "Content-Type: application/json" -d
'{"username":"admin@local","password":"Success01!0909!!"}'
# Use the returned accessToken for subsequent API calls

curl -k -X GET https://localhost/v1/domains -H "Authorization: Bearer <token>"
# Should show domain "mgmt" with status "ACTIVE"

curl -k -X GET https://localhost/v1/hosts -H "Authorization: Bearer <token>"
# Should show all 4 ESXi hosts with status "ACTIVE"
```

Fleet Management Workaround

In the lab, Fleet Management (Cloud Proxy) deployment failed during bringup with error "Upload binary content Operations-Cloud-Proxy-9.0.1.0.24960349.ova to VCF Operations fleet management failed."

Workaround -- Deploy via VCF Operations import:

1. Open VCF Operations (<https://192.168.1.77>) > Fleet Management > Lifecycle
2. Import VCF Operations into Lifecycle management -- this automatically deploys a Cloud Proxy at 192.168.1.78
3. Configure SSO: Embedded identity broker with AD/LDAP (lab.local domain at 192.168.1.230)
 - Attribute mappings: userName > sAMAccountName, firstName > givenName, lastName > sn, email > mail
 - Group provisioning: Domain Admins synced, nested groups enabled
 - Base DN: DC=lab,DC=local
4. Add VCF Instance "vcf-lab" connecting SDDC Manager at 192.168.1.241

2.7 Post-Deployment Verification (VDT)

VDT Overview

The VCF Diagnostic Tool (VDT) is a read-only Python diagnostic tool that checks VCF environment health including certificates, services, inventory, disk, NFS, locks, credentials, NSX, and LCM configuration.

VDT is NOT pre-installed on SDDC Manager. It must be downloaded separately from Broadcom KB article 344917 and uploaded manually.

VDT Download and Upload

```
# On your workstation, download from Broadcom KB 344917
# File: vdt-2.2.7_02-05-2026.zip
# MD5: cc5780c93984ffff13c91b8756d3b497d
# SHA256: 8801db4dfa3ed0ac19b8d33482d8dbff0634f0ac03f0d36926b438eab7cb43fc

# Upload to SDDC Manager (SCP works from external machine TO SDDC Manager)
scp vdt-2.2.7_02-05-2026.zip vcf@192.168.1.241:/home/vcf/

# SSH to SDDC Manager
ssh vcf@192.168.1.241

# Extract
unzip vdt-2.2.7_02-05-2026.zip
```

Running VDT

```
cd /home/vcf/vdt-2.2.7_02-05-2026
python vdt.py
```

VDT prompts for the SSO administrator password (`administrator@vsphere.local`). It then runs all health checks and produces both text and JSON reports.

Results location:

```
/var/log/vmware/vcf/vdt/vdt-<timestamp>.txt
/var/log/vmware/vcf/vdt/vdt-<timestamp>.json
```

VDT Check Categories

Category	What It Checks
SDDC Manager Info	Version, hostname, build
NTP Service & Server	NTP daemon running, server responding
/etc/hosts	Properly formatted
SDDC Manager Services	COMMON_SERVICES, LCM, DOMAIN_MANAGER, OPERATIONS_MANAGER, SDDC_MANAGER_UI
Disk Utilization	Filesystem space and inodes
Host/Domain/vCenter/NSX Status	All components ACTIVE in inventory
Certificate Trust/Expiry/SAN	Certs in trust stores, not expired, SAN contains hostname+IP
Deployment/Resource/Changelog Locks	No stuck locks
Credential Health	No invalid transactions, no stale credentials
NFS Mount Ownership	Correct owner (root:vcf) on /nfs/vmware/vcf/nfs-mount/
Transport Node FQDNs	FQDN matches display name
LCM Manifest	Manifest file present in DB

Common VDT Failures and Fixes

FAIL: NFS Mount Ownership

Symptom: `/nfs/vmware/vcf/nfs-mount/` owned by `nginx` instead of `root`

```
# Fix
chown root:vcf /nfs/vmware/vcf/nfs-mount/

# Verify
ls -la /nfs/vmware/vcf/
# Should show: drwxrwxr-x root vcf nfs-mount/
```

Reference: <https://knowledge.broadcom.com/external/article/392923>

FAIL: NSX Certificate SAN Missing

Symptom: VDT reports "SAN contains neither hostname nor IP" for NSX VIP and NSX Manager. Default NSX self-signed cert has `SAN=*.lab.local` which VDT does not accept.

Fix: Generate a new self-signed certificate with explicit SAN entries and apply via NSX API. Full procedure:

```
# Step 1: Create OpenSSL config on NSX Manager (SSH as root)
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF

# Step 2: Generate self-signed certificate
openssl req -x509 -nodes -days 825 -newkey rsa:2048 -keyout /tmp/nsx.key -out
/tmp/nsx.crt -config /tmp/nsx-cert.conf -sha256

# Step 3: Create JSON payload (Python avoids PEM escaping issues)
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
```

```
" > /tmp/nsx-import.json

# Step 4: Import certificate (single-line curl -- NSX shell has no backslash
continuation)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates?action=import" -H "Content-Type: application/json" -d
@/tmp/nsx-import.json
# Returns certificate ID, e.g.: 701d1416-5054-4038-8749-4ac495980ebd

# Step 5: Get node UUID
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster
# Returns node UUID, e.g.: 95493642-ef4a-cb8e-ed7c-5bc20033f2c2

# Step 6: Apply to NSX Manager node
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=API&node_id=<node-uuid>"

# Step 7: Apply to cluster VIP
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Important: `DNS.3 = nsx-manager.lab.local` is required in the SAN because SDDC Manager registers NSX using this FQDN. Without it, VDT fails the SAN check.

FAIL: NSX Certificate Trust (after replacing cert)

After replacing the NSX self-signed certificate, import it into SDDC Manager's trust stores:

```
# On SDDC Manager as root:

# Pull the active NSX certificate
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# Import into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt -keystore
/etc/vmware/vcf/commonsvcs/trusted_certificates.store -storepass "$KEY" -noprompt

# Import into Java cacerts
```

```
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt -keystore
/etc/alternatives/jre/lib/security/cacerts -storepass changeit -noprompt

# Restart SDDC Manager services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

Services take approximately 5 minutes to restart. After restart, re-run VDT to confirm all NSX cert trust checks pass.

Reference: <https://knowledge.broadcom.com/external/article/316056>

WARN: vCenter Certificate SAN

Symptom: VDT reports "SAN contains hostname but not IP" for vCenter. This is cosmetic and acceptable for lab environments -- vCenter's default certificate includes the FQDN but not the IP address in the SAN.

Final VDT Results (Lab -- All Remediated)

Check	Result
SDDC Manager Info	PASS -- Version 9.0.1.0.24962180
NTP Service & Server	PASS -- 192.168.1.230 responding
/etc/hosts	PASS
SDDC Manager Services	PASS -- All 5 services ACTIVE
Commonservices API	PASS -- HTTP 200
Disk Utilization (space + inodes)	PASS
Host/Domain/vCenter/PSC/Cluster/NSX Status	PASS -- All ACTIVE
SDDC Cert (Trust/Expiry/SAN)	PASS -- 717 days remaining
vCenter Cert Trust/Expiry	PASS
vCenter Cert SAN	WARN (hostname but not IP -- cosmetic)
NSX VIP Cert (Trust/Expiry/SAN)	PASS -- 825 days remaining
NSX Manager Cert (Trust/Expiry/SAN)	PASS
Deployment/Resource/Changelog Locks	PASS -- No locks
Service Account Auth	PASS

Credential Transactions	PASS
NFS Mount Ownership	PASS (after fix)
NFS Subdirectories	PASS
Transport Node FQDNs	PASS
LCM Manifest	PASS

PART III: Day 2 Operations

3.1 VCF Operations First Login & Setup

VCF Operations (formerly VMware Aria Operations) is the **mandatory central management console** for the entire VCF 9.0 platform. The SDDC Manager UI is deprecated and will be removed in a future release. VCF Operations is now the primary interface for fleet management, lifecycle management, licensing, monitoring, certificate management, password management, and all Day 2 operations.

3.1.1 Environment Reference

Component	Address
VCF Operations	192.168.1.77 (vcf-ops.lab.local)
SDDC Manager	192.168.1.241 (sddc-manager.lab.local)
vCenter Server	192.168.1.69 (vcenter.lab.local)
Offline Depot Server	192.168.1.52:8443
ESXi Hosts	esxi01 (.74), esxi02 (.75), esxi03 (.76), esxi04 (.82)
NSX Manager	192.168.1.71 (nsx-node1.lab.local)
NSX VIP	192.168.1.70 (nsx-vip.lab.local)
Fleet Management (Cloud Proxy)	192.168.1.78 (fleet.lab.local)
DNS Server	192.168.1.230 (Windows AD DC for lab.local)
Mode	Air-gapped / Disconnected

3.1.2 Initial Access

1. Open a browser and navigate to `https://192.168.1.77`
2. Log in with the credentials configured during bringup:
 - **Username:** `admin`
 - **Password:** The password set during VCF Installer deployment

3. Upon first login, you land on the **Fleet Management** dashboard

3.1.3 Navigation Overview

The left navigation pane displays the main sections:

Section	Purpose
Fleet Management	Lifecycle management, depot configuration, component health
Infrastructure Operations	Monitoring, dashboards, alerts, diagnostics
Security & Compliance	Compliance benchmarks, drift detection
License Management	Registration and license file management
Administration	Integrations, accounts, access control, system settings

Note: If licensing has not been completed, some menu items may be grayed out. VCF Operations runs in evaluation mode for up to 90 days after deployment.

3.1.4 Initial Setup Wizard (Manual OVA Deployment Only)

If VCF Operations was deployed manually via OVA rather than through the VCF Installer, the initial setup wizard appears automatically on first access:

1. Click **NEXT** on the welcome page
2. **Set Admin Password:** Enter a new password for the **admin** user (minimum 8 characters, upper, lower, number, special character)
3. Select **EXPRESS INSTALLATION** to deploy a single-node configuration
4. Accept the EULA/license agreement
5. The wizard completes and brings you to the main VCF Operations interface

3.1.5 CEIP Opt-Out

VCF Operations ships with the Customer Experience Improvement Program (CEIP) enabled by default. For air-gapped labs this should be disabled:

1. Navigate to **Administration > Management**
2. Locate the **CEIP** or **Customer Experience** setting
3. Toggle to **Disabled**
4. Click **Save**

Tip: In a disconnected environment, CEIP data cannot be sent anyway, but disabling prevents unnecessary connection attempts that clutter logs.

3.2 License Registration (Air-Gapped)

VCF 9.0 uses a unified subscription-based license file model. The old 25-character license keys are replaced by license files. There are only two license types: **VMware Cloud Foundation (cores)** and **VMware vSAN (TiBs)**. All other components (NSX, vCenter, VCF Automation, etc.) are automatically licensed when a primary license is assigned.

3.2.1 Download the Registration File

Navigation: VCF Operations > **License Management** > **Registration**

1. In the left navigation, click **License Management**
2. Click **Registration**
3. In the **Download Registration File** card, click **Download**
4. Save the `.jws` (JSON Web Signed) file to a local machine or USB drive

3.2.2 Upload Registration to VCF Business Services Console

This step is performed on a machine with internet access:

1. Transfer the `.jws` file to a computer with internet access via USB drive or secure transfer
2. Open a browser and navigate to `https://vcf.broadcom.com`
3. Log in with your **Broadcom Support Portal** credentials
4. Select the **Site ID** you want to register this VCF Operations instance against
5. Upload the registration file when prompted
6. **Add licenses** to your license server -- you must add licenses to each license server to complete registration
7. The Business Services Console generates a **license file** in exchange
8. Click **Download** to save the license file
9. Click **Finish**

3.2.3 Import the License File into VCF Operations

Navigation: VCF Operations > **License Management** > **Registration**

1. Return to VCF Operations at <https://192.168.1.77>
2. Navigate to **License Management > Registration**
3. Click **Import License File**
4. Click **Browse** and select the downloaded license file
5. Click **Import**
6. Upon completion, click **Complete**

3.2.4 Verification

- The license status should change from "Evaluation" to showing your valid license
- All previously grayed-out menu items become fully active
- Navigate to **License Management** and confirm license details show correct core counts and expiration

3.2.5 Ongoing License Usage Reporting (Every 180 Days)

Since the environment is air-gapped, you must manually report usage at least every 180 days:

1. Navigate to **License Management > Registration**
2. Click **Generate Usage File** and save it
3. Transfer the usage file to an internet-connected machine
4. Log in to <https://vcf.broadcom.com>
5. Navigate to **License Management > VCF Operations Registrations**
6. Find your VCF Operations instance, click the vertical ellipsis menu, select **Upload Usage File**
7. Upload the usage file, click **Save and Next**
8. The system generates an updated license file -- click **Download**
9. Click **Finish**
10. Transfer the new license file back and import it via **License Management > Registration > Import License File**

WARNING: If license usage data is not submitted within 180 days, licenses are treated as expired. Hosts are disconnected from vCenter and workload operations are blocked. In a lab environment, set a calendar reminder.

3.3 Fleet Management & Depot Configuration

3.3.1 Fleet Management Appliance Registration

The Fleet Management appliance handles lifecycle management functions formerly in SDDC Manager. If deployed via the VCF Installer, this may already be connected. If not:

Navigation: <https://192.168.1.77/admin/> (the Admin UI, not the main UI)

1. Open a browser and navigate to <https://192.168.1.77/admin/>
2. Log in as **admin** with your VCF Operations admin password
3. Navigate to **System Status > Fleet Management** section
4. Click the **Connect** button
5. **Node Address:** Enter the FQDN of the VCF Operations Fleet Management appliance
6. **Admin Password:** Enter the Fleet Management appliance admin password
7. Click **Test Connection** to verify connectivity
8. Review the security certificate presented by the appliance
9. **Accept the certificate** and click **Next**
10. Enter the **VCF Operations admin password** when prompted
11. Click **Finish**
12. The Fleet Management status should show as **Connected** in the Admin UI

Lab Context: In the lab, Fleet Management was deployed at 192.168.1.78 via the VCF Operations Lifecycle import (not during bringup, which failed). The Cloud Proxy was deployed automatically during this process.

3.3.2 Configure the Offline Depot for VCF Management Components

In VCF 9.0, depot functionality has moved from SDDC Manager to VCF Operations. You must configure the depot before you can download binaries for additional components. Only one depot connection (online OR offline) can be ACTIVE at a time.

Navigation: VCF Operations > **Fleet Management > Lifecycle > VCF Management > Depot Configuration**

1. Navigate to **Fleet Management > Lifecycle > VCF Management > Depot Configuration**
2. Click **Configure** under the Offline Depot widget
3. **Offline Depot Type:** Keep as "Webserver"
4. **Repository URL:** Enter <https://192.168.1.52:8443>
5. **Username:** **admin**
6. **Password:** **admin**
7. Check **"I accept the imported certificate"** after reviewing the certificate details
8. Click **OK**

3.3.3 Verify Depot Connection

1. Navigate to **Binary Management** > **Install Binaries** tab
2. You should see available binaries listed for download (Operations for Logs, Operations for Networks, etc.)
3. Download status should show the binaries available for installation

3.3.4 Configure the Offline Depot for VCF Instance (SDDC Manager)

Navigation: VCF Operations > **Fleet Management** > **Lifecycle** > **VCF Instances** > (select your instance) > **Depot Settings**

1. Navigate to **Fleet Management** > **Lifecycle** > **VCF Instances**
2. Select your VCF Instance from the list
3. Click **Depot Settings**
4. Under **Offline Depot**, select **Set Up**
5. Enter the hostname of your depot server: `192.168.1.52:8443`
6. Click **Save**

Note: Before configuring the SDDC Manager depot, you may need to trust the SSL certificate of your offline depot server. This was already done during the initial bringup (certificate imported into SDDC Manager's Java trust store).

3.3.5 Bundle Management & Update Scheduling

After depot configuration, binaries become available for download and deployment:

1. Navigate to **Fleet Management** > **Lifecycle** > **VCF Management** > **Components**
2. View available components and their current/available versions
3. Click **Add** next to a component to deploy it (e.g., operations-logs, operations-networks)
4. Updates to existing components are surfaced in the **Updates Available** section
5. Schedule updates during maintenance windows by selecting the component and clicking **Schedule Update**

Tip: Binary downloads from depot may intermittently fail. If a download disappears, retry it.

3.4 Data Source Connections (VCF Cloud Account)

This is the critical step that connects VCF Operations to your SDDC Manager, enabling automatic monitoring of all VCF domains including vCenter, NSX, and vSAN.

3.4.1 Add the VMware Cloud Foundation Account

Navigation: VCF Operations > **Administration** > **Integrations** > **Accounts** tab > **Add**

1. In the left navigation, click **Administration**
2. Click **Integrations**
3. Click the **Accounts** tab
4. Click **Add**
5. On the **Account Types** page, select **VMware Cloud Foundation**
6. Fill in the following fields:
 - **Name:** Lab VCF Instance (or any descriptive name)
 - **Description:** Management Domain - Lab Environment
 - **Physical Data Center:** Select existing or create new
7. **Connection Details:**
 - **SDDC Manager FQDN:** sddc-manager.lab.local (use FQDN rather than IP for VCF SSO to work properly)
8. **Credentials:**
 - Click the **Add (+)** icon to create new credentials
 - **Credential Name:** SDDC Manager Admin
 - **Username:** administrator@vsphere.local
 - **Password:** Enter the corresponding password
 - Click **OK** to save the credential
9. **Collector:**
 - Select which VCF Operations collector or collector group manages this account
 - Ensure the SDDC Manager FQDN is reachable from the selected collector
10. Click **Validate Connection**
11. A certificate dialog appears -- review the certificate and click **OK** to accept
12. **Advanced Settings:**
 - **Enable Domain Monitoring on Creation:** Toggle to **True** for automatic data collection on newly discovered domains
 - **Configuration Limits:** Optionally enter the name of a file containing VCF configuration max soft and hard limits
13. **Management Options:**
 - Select the option for monitoring plus license/plugin management
14. Click **Add** to create the account

3.4.2 Start Data Collection

1. On the **Accounts** tab, locate your new VMware Cloud Foundation account
2. Click the **vertical ellipsis** (three dots) menu next to the account
3. Select **Start Collecting All**

3.4.3 What Happens Automatically

After configuration, VCF Operations automatically:

- Discovers all VCF domains
- Creates vCenter adapter instances for each domain's vCenter
- Creates NSX adapter instances for each domain's NSX Manager
- Creates vSAN adapter instances for vSAN-enabled clusters
- Configures system-managed credentials for NSX 9.0 endpoints
- Begins collecting metrics, properties, and alerts from all discovered components

Note: Initial collection takes multiple cycles (standard cycle = 5 minutes). Allow 15-30 minutes for full data population.

3.4.4 Add Individual vCenter Account (If Not Auto-Discovered)

When you add a VCF account, vCenter accounts are normally auto-discovered. If you need to add one manually:

Navigation: VCF Operations > **Administration** > **Integrations** > **Accounts** tab > **Add**

1. Click **Add** on the Accounts tab
2. Select **vCenter** from the Account Types page
3. **Display Name:** `vcenter.lab.local - 192.168.1.69`
4. **Description:** `Management Domain vCenter`
5. **vCenter Field:** `vcenter.lab.local` or `192.168.1.69`
6. **Credentials:** Click **Add (+)** -- enter `administrator@vsphere.local` and password
7. **Collector:** Select the VCF Operations collector
8. Click **Validate Connection** and accept the certificate
9. **Optional Features:**
 - **Activate for Operational Actions:** Check to enable remediation actions
 - **Activate Log Collection:** Check to enable log forwarding (requires VCF Operations for Logs)
 - **Activate Network and Flow:** Check to enable network monitoring
10. Click **Add**
11. On the Accounts tab, click the **vertical ellipsis** menu > **Start Collecting**

Important: vCenter accounts do NOT start monitoring automatically. You must manually initiate data collection.

3.4.5 Verify Data Collection

Navigation: VCF Operations > **Administration** > **Integrations** > **Accounts**

- For each configured account (VCF, vCenter, NSX, vSAN), verify:
 - Collection Status:** Green "Collecting" (not "Stopped" or "No data receiving")
 - Collection State:** "Collecting Data"
- Navigate to **Infrastructure Operations** > **Inventory** and verify:
 - vCenter instances** (vcenter.lab.local)
 - ESXi hosts** (esxi01, esxi02, esxi03, esxi04)
 - Clusters** (management cluster)
 - Datastores** (vSAN datastore)
 - Virtual Machines** (all management VMs)
 - NSX objects** (NSX Manager, transport nodes)
- Navigate to **Infrastructure Operations** > **VCF Health** and verify all components show healthy status

Key Timing Notes:

Metric	Interval
Standard collection cycle	Every 5 minutes
Initial collection (full population)	15-30 minutes
Property-based diagnostic scans	Every 4 hours
Telegraf agent data collection	Every 4 minutes
Cloud proxy registration (first boot)	Up to 20 minutes

3.5 SSO / Identity & Access Management

VCF 9.0 introduces the VCF Identity Broker (VIDB), which provides federated SSO across all VCF components.

3.5.1 Configure VCF Single Sign-On for VCF Operations

Navigation: VCF Operations > **Fleet Management** > **Identity & Access** > **VCF Management** > **Operations Appliance**

1. Navigate to **Fleet Management > Identity & Access > VCF Management**
2. Select **Operations Appliance**
3. Click **Configure**
4. Select the **Identity Broker** instance from the dropdown
5. Accept the role assignment requirements
6. The system validates and displays the Identity Broker on the configuration list after processing

3.5.2 Verify Authentication Source

Navigation: VCF Operations > **Administration > Control Panel > Authentication Sources**

1. Navigate to **Administration > Control Panel > Authentication Sources**
2. Confirm that "**VCF SSO**" now appears in the list of available authentication sources

3.5.3 Import Directory Users and Groups

Navigation: VCF Operations > **Administration > Control Panel > Access Control**

1. Navigate to **Administration > Control Panel > Access Control**
2. Click the **three-dot menu** and select **Import from Source** (do NOT use the standard "Add" button - that creates local groups only)
3. Select **VCF SSO** as the source
4. Search for your Active Directory groups (e.g., `vcf-admins` , `vcf-readonly` , `Domain Admins`)
5. Select the groups to import

3.5.4 Assign Permissions

1. Select the imported groups
2. Click the menu and choose **Edit**
3. Assign:
 - **Role:** The actions users can perform (e.g., Administrator, ReadOnly, ContentAdmin)
 - **Scope:** The objects those actions apply to (e.g., all objects, specific data centers)
4. Click **Save**
5. Test by logging out and logging back in using VCF SSO authentication

3.5.5 Add Active Directory Identity Source in vCenter

To add AD authentication to vCenter separately:

1. Log in to vCenter at `https://192.168.1.69`

2. Navigate to **Administration > Single Sign-On > Configuration**
3. Click **Identity Sources > Add**
4. Select **Active Directory over LDAP** (IWA is removed in vCenter 9.0)
5. Enter your AD domain details:
 - **Domain name:** `lab.local`
 - **Base distinguished name for users:** `DC=lab,DC=local`
 - **Base distinguished name for groups:** `DC=lab,DC=local`
 - **Primary server URL:** `ldap://192.168.1.230:389`
 - **Bind user distinguished name:** (your bind user DN)
 - **Bind password:** (bind user password)
6. Click **Test Connection** to verify
7. Click **Add** to save

Lab Context: The lab has AD/LDAP configured via the embedded identity broker with lab.local domain at 192.168.1.230. Attribute mappings: `userName=sAMAccountName`, `firstName=givenName`, `lastName=sn`, `email=mail`. Domain Admins group synced with nested groups enabled.

3.6 Certificate Management

VCF 9.0 introduces unified, non-disruptive TLS certificate management across all VCF components.

3.6.1 View All Certificates

Navigation: VCF Operations > **Fleet Management > Certificates**

1. Navigate to **Fleet Management > Certificates**
2. Select either **VCF Management** or **VCF Instances** tab
3. View the certificate inventory showing all TLS certificates across your environment
4. Certificates are displayed for: vCenter, ESX hosts, VCF Operations, VCF Automation, Fleet Management, SDDC Manager, NSX local manager
5. Review certificate expiration dates and status alerts

3.6.2 Configure a Certificate Authority -- Microsoft CA

Navigation: VCF Operations > **Fleet Management > Certificates > Configure CA**

1. Navigate to **Fleet Management > Certificates**
2. Select **VCF Management** or **VCF Instances** (and choose a specific instance)

3. Click **Configure CA**
4. Select **Microsoft Certificate Authority**
5. Fill in:
 - **CA Server URL:** Must begin with `https://` and end with `certsrv` (e.g., `https://ca.lab.local/certsrv`)
 - **User Name:** Least-privileged service account (e.g., `svc-vcf-ca`)
 - **Password:** Service account password
 - **Template Name:** The issuing certificate template created in Microsoft CA
6. Click **Save**

Important: VCF management components only support Microsoft CA. VCF Instance components support both Microsoft CA and OpenSSL. You configure the CA separately for management components and instance components.

3.6.3 Configure a Certificate Authority -- OpenSSL

1. Click **Configure CA**
2. Select **OpenSSL**
3. Fill in:
 - **Common Name:** FQDN of SDDC Manager appliance
 - **Country:** Country of registration
 - **Locality Name:** City
 - **Organization Name:** Legal company name
 - **Organization Unit Name:** Department
 - **State:** Full state/province name (unabbreviated)
4. Click **Save**

3.6.4 Replace Default Certificates

After configuring a CA, replace default self-signed certificates with enterprise CA-signed certificates. Certificates eligible for non-disruptive auto-renewal include: ESX SSL, vCenter machine SSL, NSX LM/VIP, SDDC Manager SSL, and VCF Operations certificates.

3.6.5 Enable Automatic Renewal

On the Certificates page, enable **auto-renewal** for supported certificates. This prevents unexpected certificate expiration.

Lab Note: In a lab with no Microsoft CA, you can continue using self-signed certificates. The certificate management

UI will show certificate expiration warnings, which is normal.

3.7 Password Management & Rotation

VCF 9.0 provides unified password management centralized in VCF Operations, replacing the password management previously found in SDDC Manager.

3.7.1 View Password Status

Navigation: VCF Operations > **Fleet Management** > **Passwords**

1. Navigate to **Fleet Management** > **Passwords**
2. Select either **VCF Management** or **VCF Instances** tab
3. Select your domain to view all managed account passwords
4. The dashboard shows:
 - Account names and types (root, admin, backup, consoleuser, support, admin@local, vmware-system-user)
 - Password status (valid, expiring soon, expired)
 - Last modified dates
 - Expiration dates

3.7.2 Managed Components and Accounts

VCF Management Components:

Component
Fleet Management
VCF Automation
VCF Identity Broker
VCF Operations
VCF Operations for Logs
VCF Operations for Networks

VCF Instance/Domain Components:

Component

ESX hosts (esxi01-04)
NSX Manager
vCenter Server
SDDC Manager

3.7.3 Password Functions Reference

Function	When to Use	What It Does
Update	You changed a password outside VCF	Updates VCF database to match the new password on the component
Rotate	Scheduled password change	Changes password on BOTH the component AND the VCF database
Remediate	A rotation failed mid-way	Re-syncs by accepting the current password on the component

3.7.4 Update a Password

1. Navigate to **Fleet Management > Passwords**
2. Select the component and account you want to update
3. Click **Update Password**
4. Enter the new desired password (this lets you specify the exact password, unlike rotation)
5. Confirm the new password
6. Click **Update**

3.7.5 Rotate Passwords

Password rotation generates a randomized password:

1. Navigate to **Fleet Management > Passwords**
2. Select accounts to rotate
3. Click **Rotate**
4. The system generates random passwords meeting complexity requirements
5. Set the rotation interval: **30 days, 60 days, or 90 days**
6. You can also **deactivate** the schedule
7. Only a user with the **ADMIN** role can perform this task

Note: Auto-rotate is automatically enabled for vCenter Server. It may take up to 24 hours to configure the auto-rotate policy for a newly deployed vCenter.

3.7.6 Remediate Passwords

If a password gets out of sync between SDDC Manager and the actual component:

Prerequisites:

- No workflows running or scheduled
- Required permissions: **Fleet Management > Passwords > Manage** and **Fleet Management > Passwords > View**
- One account remediated at a time

Steps:

1. Navigate to **Fleet Management > Passwords**
2. Select either **VCF Management** or **VCF Instances** and choose your domain
3. Select the component showing a password issue
4. Click **Remediate Password**
5. Enter and confirm the manually-set password (the password currently on the component)
6. Click **Remediate Password** to complete

Tip: Password rotation options from VCF 5.x are not fully available in VCF Operations yet. Use the SDDC Manager API as a workaround for some rotation tasks if needed.

WARNING — Credential Rotation Cascade Failure: If a credential update or rotation fails mid-operation (commonly because NSX was temporarily unreachable during a boot storm or maintenance), the component resource can get stuck in **ACTIVATING** state with stale exclusive locks blocking **all** future password operations. Error messages: `"Resources [host] are not available/ready"` or `"Unable to acquire resource Level Lock(s)"`. This requires a database-level fix on SDDC Manager — see [Section 7.2.6](#) for the complete repair procedure.

3.8 Compliance Monitoring

3.8.1 Access Compliance

Navigation: VCF Operations > **Security & Compliance** > **Compliance**

1. Navigate to **Security & Compliance** > **Compliance**
2. Ensure your data sources (vCenter, VCF account) are configured and collecting before proceeding

3.8.2 Activate VMware SDDC Benchmarks

1. On the Compliance page, locate the **VMware SDDC Benchmarks** section
2. Click **Activate** for the benchmark you want to enable
3. Available score cards:
 - **vSphere Security Configuration Guide**
 - **vSAN Security Configuration Guide**
 - **NSX Security Configuration Guide**
4. Select an applicable **policy** when prompted
5. The system activates relevant alert definitions automatically

3.8.3 Activate Regulatory Compliance Benchmarks**Built-in standards (no additional download):**

Standard	Notes
DISA Security Standards	Defense Information Systems Agency STIGs
FISMA Security Standards	Federal Information Security Management Act
HIPAA	Health Insurance Portability and Accountability Act

Standards requiring marketplace download (.PAK file):

Standard	Notes
PCI DSS Compliance Standards	Payment Card Industry Data Security Standard
CIS Security Standards	Center for Internet Security Benchmarks
NIST SP 800-171	Controlled Unclassified Information
NIST SP 800-53 R5	Security and Privacy Controls

For air-gapped environments, install marketplace packs manually:

Navigation: VCF Operations > **Administration** > **Repository**

1. Navigate to **Administration** > **Repository**

2. The **Add Solution** wizard opens
3. **Page 1:** Locate and upload the `.PAK` file
4. **Page 2:** Accept the EULA and install
5. **Page 3:** Review the installation
6. Click **Add Account** to configure the newly installed integration

3.8.4 Configure Drift Detection

Navigation: VCF Operations > **Fleet Management** > **Configuration Drifts** > **Schedule Drift Detection**

1. Navigate to **Fleet Management** > **Configuration Drifts**
2. Click **Schedule Drift Detection**
3. **Step 1 - Configuration Details:** Enter a name and description for the drift check (you can schedule drifts only for vCenter object types)
4. **Step 2 - Define Scope:** Select vCenter instances from the right panel and move them to the left Scope window (you can select a VCF folder as scope to automatically include all VCF instances in that folder)
5. **Step 3 - Preview Scope:** Click **Preview Scope** to validate which vCenter instances will be included
6. **Step 4 - Filtering Criteria:** Apply filters and add criteria specific to the vCenter object type
7. **Step 5 - Schedule:** Set the desired schedule interval and click **Create**
8. The system creates a new job visible in the automation central page

3.9 Alerts, Notifications & Dashboards

3.9.1 Configure Outbound Notification Plug-Ins

Navigation: VCF Operations > **Infrastructure Operations** > **Configurations** > **Outbound Settings**

1. Navigate to **Infrastructure Operations** > **Configurations**
2. Click the **Outbound Settings** tile
3. Click **Add**

3.9.2 Standard Email Plug-In

1. Select **Standard Email Plugin** from the Plug-In Type dropdown
2. **Instance Name:** `Lab Email Notifications`
3. Configure SMTP settings:
 - **Use Secure Connection:** Enable for SSL/TLS
 - **Secure Connection Type:** SSL or TLS

- **Requires Authentication:** Check if your SMTP requires auth
- **SMTP Host:** URL or IP of email server
- **SMTP Port:** 25, 465, or 587
- **Sender Email Address:** vcf-ops@lab.local
- **Sender Name:** VCF Operations
- **Receiver Email Address:** Default recipient

4. Click **Save**

5. Select the instance and click **Activate**

3.9.3 Other Available Plug-Ins

Plug-In	Use Case
Standard Email Plugin	SMTP email notifications
SNMP Trap Plugin	SNMP v1/v2c/v3 traps to network management systems
Webhook Notification Plugin	REST webhooks (supports Basic Auth, Bearer Token, OAuth, X.509, API Key)
Log File	Write alerts to log files
ServiceNow	ITSM integration
Slack	Chat-based alerting
Network Share	Write to network file shares

3.9.4 Create Notification Rules

Navigation: VCF Operations > **Infrastructure Operations** > **Configurations** > **Notifications**

1. Navigate to **Infrastructure Operations** > **Configurations**
2. Click the **Notifications** tile
3. Click **Add** on the toolbar

Step 1 - Basic Details:

- **Name:** Rule identifier (e.g., Critical Host Alerts)
- **Notification Status:** Toggle active/inactive
- **Notification Type:** Select "Alert"

Step 2 - Define Filtering Criteria:

- **Object Scope:** Object Type, specific Objects, Tags, Applications, Tiers (option to include child objects)

- **Alert Scope:** Alert Types/Subtypes, Alert Impact, or Alert Definition
- **Criticality:** Filter by severity (Critical, Immediate, Warning, Information)
- **Notify On:** Trigger conditions (new, updated, canceled)

Step 3 - Select Outbound Method:

- Choose from configured plug-in instances

Step 4 - Payload Template:

- Select default or custom template
- For Email: configure Recipients, Cc/Bcc, Notify Again interval, Max Notifications, Delay to Notify

Step 5 - Test:

- Click **Initiate Process** to test, select an alert definition and object, click **Validate Configuration**

Step 6 - Create:

- Click **Create** to finalize

3.9.5 Key Predefined Dashboards

Navigation: VCF Operations > **Infrastructure Operations** > **Dashboards & Reports**

Dashboard Category	What It Shows
Overview	Geo-map view of VCF instances, inventory sections, diagnostic findings, security risk highlights
Cluster Configuration	vSphere cluster configuration requiring attention
ESXi Configuration	ESXi host configurations needing review
Network Configuration	vSphere distributed switch configurations
VM Configuration	Virtual machine configurations
vSAN Configuration	vSAN configuration details
vSAN OSA Performance	Read/write latency, contention, utilization
vSAN ESA Performance	ESA-specific metrics
Security Operations	User auth, encryption status, CVE advisories, certificate health

Skyline Operational	Proactive monitoring and recommendation dashboard
Energy Efficiency	Virtualization efficiency, idle VM impact

3.9.6 Create a Custom Dashboard

1. From the left menu, click **Dashboards & Reports**
2. Click **New Dashboard**
3. **Dashboard Name:** Enter a name (using `/` in the name creates folder hierarchy, e.g., `Lab/Overview`)
4. The dashboard canvas opens for widget placement
5. Available widget types: Metric Chart, View, Health Chart, Sparkline, Mashup Chart, Rolling View
6. For each widget, click the **pencil icon** to configure data source, metrics, time range, and visual options
7. **Widget Interactions:** Set data from one widget as a filter for another
8. **Share** with user groups, mark as **Favorite**, or set as **Dashboard Home** (up to 5 dashboards on Product Home)

3.10 Backup Configuration

3.10.1 Fleet-Level Backups

Navigation: VCF Operations > **Fleet Management** > **Lifecycle** > **Settings** > **SFTP Settings**

1. Navigate to **Fleet Management** > **Lifecycle** > **Settings**
2. Click **SFTP Settings**
3. Configure the SFTP server details:
 - **SFTP Host:** IP or FQDN of your SFTP server
 - **Port:** Default 22
 - **Username:** SFTP account username
 - **Password:** SFTP account password
 - **Path:** Directory path for backup storage
4. Click **Test Connection** to verify
5. Click **Save**
6. Navigate to **Backup Settings** and configure the backup schedule:
 - **Backup frequency:** Daily, Weekly, or Custom
 - **Retention:** Number of backups to keep

3.10.2 Instance-Level Backups

Navigation: VCF Operations > **Inventory** > **VCF Instance** > **Actions** > **Manage VCF Instance Settings**

1. Navigate to **Inventory** > Select your VCF Instance
2. Click **Actions** > **Manage VCF Instance Settings**
3. Click **Backup Settings**
4. Configure instance-specific backup parameters
5. Click **Save**

3.11 VCF Operations for Logs

VCF Operations for Logs is **not deployed automatically** during initial bringup. It must be deployed as a Day 2 operation. **Status: Deployed.**

Setting	Value
FQDN	logs.lab.local
IP Address	192.168.1.242
VM Name	logs
Node Size	Small
Deployment Method	Fleet Management with custom cert

Known Issue — Self-Signed Certificate SAN Mismatch: The Fleet Management deployment wizard's "Generate self-signed certificate" option may produce a certificate whose SAN entries do not match the node FQDN/IP, causing a precheck error: "Certificate validation for component vrli:vrli-master — The hosts in the certificate doesn't match with the provided/product hosts." The workaround is to generate a custom certificate with OpenSSL and import it. See [Section 3.11.1a](#).

3.11.1 Deploy via Fleet Management

Navigation: VCF Operations > **Fleet Management** > **Lifecycle** > **VCF Management** > **Components**

Prerequisites: Depot must be configured (see Section 3.3) and the operations-logs binary must be downloaded via **Binary Management** > **INSTALL BINARIES** tab. The OVA and PAK files must be in the offline depot under `PROD\COMP\VRLI\`.

1. Navigate to **Fleet Management** > **Lifecycle** > **VCF Management**

2. Under the Components section, click **Add** next to `operations-logs`
3. Select **New Installation**
4. Select deployment type: **Simple** for lab environments
5. **Certificate Configuration:**
 - **Recommended:** Import a custom certificate generated with proper SANs (see [Section 3.11.1a](#))
 - Alternative: Generate self-signed certificate (may fail precheck — see warning above)
6. **VM Location & OS Configuration:**
 - Select vCenter (`vcenter.lab.local`), cluster (`vcenter-cl01`), VM network, and datastore (`vcenter-cl01-ds-vsan01`)
 - Click **Edit Server Selection** to choose DNS (192.168.1.230) and NTP servers
7. **Component Configuration:**
 - Click **Add Password** to set default password (15+ characters, must include special characters `!@#$%^&*`)
 - **Node Size:** Small (for lab)
 - **FIPS Mode:** Disable for lab
 - **VM Compatibility:** Update to latest hardware version
 - **Time Sync:** Select NTP servers
 - **VM Name:** `logs`
 - **FQDN:** `logs.lab.local`
 - **IP Address:** `192.168.1.242`
8. Run **Precheck validation**
9. Click **Deploy**
10. Monitor deployment until completion

3.11.1a Certificate Workaround: Generate Custom Certificate

If the wizard's self-signed certificate fails precheck validation, generate a proper certificate with OpenSSL on SDDC Manager (SSH as `vcf` , then `su -` to root):

Step 1 — Verify DNS resolution:

```
nslookup logs.lab.local 192.168.1.230
nslookup 192.168.1.242 192.168.1.230
ping -c 2 logs.lab.local
```

Step 2 — Create OpenSSL config and generate certificate:

```

cat > /tmp/vrli-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
EOF

openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/vrli.key -out /tmp/vrli.crt \
  -config /tmp/vrli-cert.cnf

```

Step 3 — Verify SANs are correct:

```

openssl x509 -in /tmp/vrli.crt -noout -text | grep -A5 "Subject Alternative Name"
# Expected: DNS:logs.lab.local, DNS:logs, IP Address:192.168.1.242

```

Step 4 — Transfer cert to workstation:

Display the certificate and key, then copy-paste into local files (`vrli.crt` and `vrli.key`):

```
cat /tmp/vrli.crt
cat /tmp/vrli.key
```

Step 5 — Import in Fleet Management wizard:

1. In the deployment wizard's **Certificate** step, select **Import**
2. Upload **vrli.crt** (certificate) and **vrli.key** (private key) — must be PEM format
3. Continue to **Component Configuration** and complete the deployment as in [Section 3.11.1](#)
4. Run **Precheck** — should pass with the custom certificate

Step 6 — Verify deployment:

```
# Check appliance is reachable
curl -sk https://logs.lab.local:9543/api/v2/deployment/new -o /dev/null -w "%{http_code}"

# Check certificate on deployed appliance
openssl s_client -connect logs.lab.local:443 -servername logs.lab.local </dev/null
2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

3.11.2 Integrate with VCF Operations

Navigation: VCF Operations > **Administration** > **Control Panel** > **Log Management**

1. Navigate to **Administration** > **Control Panel** > **Log Management**
2. Enter connection details for the VCF Operations for Logs appliance
3. Click **Validate Connection**
4. Authenticate using admin credentials

3.11.3 Enable Log Collection

1. Navigate to **Administration** > **Integrations** > **Accounts**
2. Find your VCF or vCenter account and click the ellipsis > **Edit**
3. Go to the **Domains** tab (for VCF account) or **Log Operations** section (for vCenter)
4. Click **Activate Log Collection**
5. Repeat for all workload domains
6. Click **Save** and verify the collector status shows healthy

3.11.4 Configure SDDC Manager Log Forwarding (Manual)

As of VCF 9.0, there is no automated way to configure the logs agent on SDDC Manager:

1. Download the `deploy_vcf_ops_logs_agent.sh` script
2. Upload to SDDC Manager appliance (use `ssh vcf@192.168.1.241 "cat > /home/vcf/deploy_vcf_ops_logs_agent.sh" < deploy_vcf_ops_logs_agent.sh`)
3. Ensure port 9543 from SDDC Manager to VCF Operations for Logs is open
4. SSH as root and run the script

Note: The log collection configuration for vCenter adapter instances is NOT included in configuration export/import operations. SCP does not work with SDDC Manager's restricted shell -- use the `ssh cat >` method for file transfers.

3.12 SDDC-to-VCF-Ops Task Migration Reference

The following tasks have moved from SDDC Manager to VCF Operations in VCF 9.0:

Task	VCF 9.0 Location in VCF Operations
DNS/NTP Configuration	Inventory > VCF Instance > Actions > Manage VCF Instance Settings > Network Settings
Workload Domain Creation	Inventory > VCF Instance > Add Workload Domain
Backup Configuration	Fleet Management > Lifecycle > Settings
Certificate Authority	Fleet Management > Certificates > Configure CA
Certificate Management	Fleet Management > Certificates
Password Management	Fleet Management > Passwords
Network Pools	vCenter: Global Inventory > Hosts > Network Pools
Host Commissioning	vCenter: Global Inventory > Unassigned Hosts
Cluster Creation	vCenter: New SDDC Cluster
Licensing	License Management (single file model)

Critical Note: While the SDDC Manager UI is still present in VCF 9.0, performing tasks there does not immediately sync to VCF Operations. Changes depend on scheduled synchronization intervals. Use VCF Operations as the primary interface for all Day 2 operations.

3.12.1 Known Issues (VCF Operations 9.0.1)

#	Issue	Impact
1	Relationships not updated after 2nd collection cycle in management packs built with the Management Pack Builder	Custom management packs may show stale data
2	Custom network adapters do not start after VCF Operations and VCF Operations for Networks are updated to VCF 9.0	Workaround required
3	VCF Operations for Networks stops collecting metrics when NSX is upgraded from 4.2.1 to 9.0	Re-configure after upgrade
4	Manually stopped adapter instances start collecting after a management pack upgrade	Monitor adapter states after upgrades
5	Binary downloads from depot may intermittently fail	Retry the download
6	Fleet Management appliance root password must be 15+ characters	Precheck will fail otherwise
7	Only one VCF Operations for Networks instance supported	Cannot add multiple
8	Log collection configuration for vCenter adapters not included in config export/import	Manually reconfigure after import
9	License expires if usage file not submitted within 180 days (disconnected mode)	Hosts disconnect, workloads blocked
10	Do not configure NTP during OVF deployment (KB 374792)	Configure it in the setup wizard instead
11	Password rotation options from VCF 5.x not fully available	Use SDDC Manager API as workaround
12	After workload domain redeployment, vCenter/vSAN adapter may enter Warning	Reconfigure adapter

13	Infrastructure Health Adapter "no data receiving" — stale SDDC Manager credential	Fix: Integrations → SDDC Mgr → ROTATE or set manually → VALIDATE → SAVE → reboot appliance
14	Adapter log paths changed in 9.x — /storage/log/vcops/log/adapters/<Name>/	Legacy /var/log/vmware/vcops/adapters/ does not exist
15	NSX adapter warnings when NSX is powered off	Expected — clears when NSX is back online
16	NSX adapter PKIX cert trust failure — self-signed cert not trusted	Import NSX cert into /usr/java/jre-vmware-17/lib/security/cacerts (password changeit), reboot
17	NSX System Managed Credential ROTATE fails	Uncheck System Managed, set manually (admin/password), VALIDATE, SAVE
18	Two separate NSX adapters exist — VCF uses VIP, NSX "Aria Admin" uses node FQDN	Both need credentials configured separately
19	Credential Update/Rotate/Remediate cascade failure — stuck tasks and locks	Full PostgreSQL repair required — see Section 7.2.6

3.12.2 Post-Configuration Verification Checklist

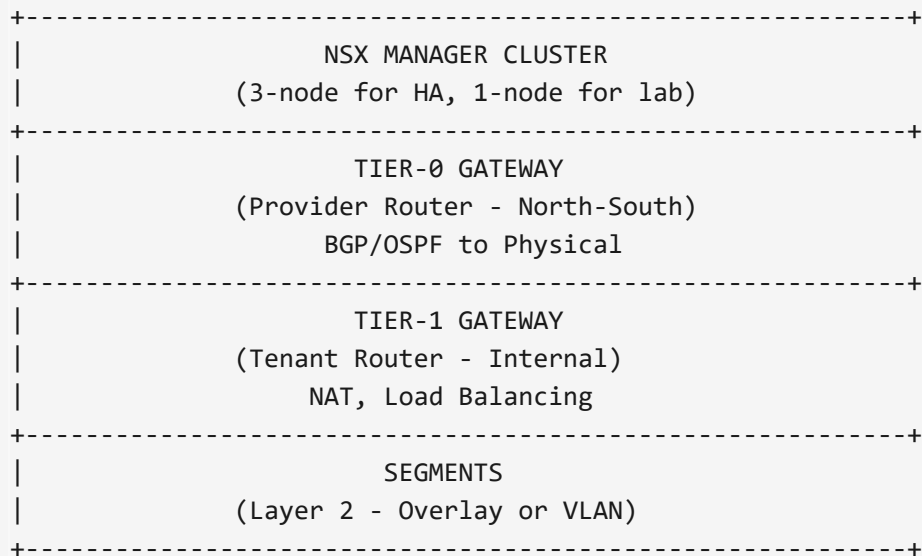
- [] License Management -- license valid, not evaluation mode
- [] Administration > Integrations > Accounts -- all adapters green "Collecting"
- [] Fleet Management dashboard -- all components healthy, Connected
- [] Depot configuration -- connected to offline depot, binaries available
- [] Infrastructure Operations > VCF Instances -- shows VCF instance with all domains
- [] All ESXi hosts (esxi01-04) visible in inventory
- [] VCF Health -- certificates, NTP, DNS checks passing
- [] Security & Compliance -- SDDC benchmarks activated
- [] Fleet Management > Passwords -- all accounts valid
- [] Fleet Management > Certificates -- all certificates visible with expiration dates

PART IV: NSX Networking & Security

4.1 NSX Manager Setup

4.1.1 NSX Architecture Overview

NSX 9.0 provides software-defined networking and security for VCF. In VCF 9.0, NSX is only available as part of the VCF stack -- there is no standalone NSX deployment option.



4.1.2 Deployment Sizing for Nested Environments

RAM Allocation	Result in Nested Lab
16GB	Kernel OOM, constant crashes, console shows <code>sysrq: Show Memory</code>
24GB	Runs initially, but MANAGER/SEARCH services crash under load (e.g., transport node configuration)
32GB (minimum)	Stable operation with 4-host cluster

Resource	Minimum for Nested	Production
RAM	32GB	48GB+
vCPU	6	8+
Deployment Size	small	medium/large

Critical Lesson: NSX Manager `small` deployment needs 32GB RAM and 6 vCPU minimum in nested environments. 16GB causes kernel OOM. 24GB runs but crashes under load. Many VCF Installer validation errors are cascading failures from an unhealthy NSX -- fix NSX health first.

4.1.3 Manual Deployment via ovftool

In nested lab environments, SDDC Manager's automated deployment often times out. Deploy NSX Manager manually using ovftool from the VCF Installer CLI:

```
/usr/bin/ovftool --skipManifestCheck --powerOn --diskMode=thin --acceptAllEulas --
allowExtraConfig --ipProtocol=IPv4 --noSSLVerify --datastore=vcenter-cl01-ds-vsan01
--network=vcenter-cl01-vds01-pg-vm-mgmt --deploymentOption=small --name=nsx-manager
--prop:nsx_role='NSX Manager' --prop:nsx_passwd_0='Success01!0909!!' --
prop:nsx_cli_passwd_0='Success01!0909!!' --
prop:nsx_cli_audit_passwd_0='Success01!0909!!' --prop:nsx_hostname=nsx-
node1.lab.local --prop:nsx_ip_0=192.168.1.71 --prop:nsx_netmask_0=255.255.255.0 --
prop:nsx_gateway_0=192.168.1.1 --prop:nsx_dns1_0=192.168.1.230 --
prop:nsx_domain_0=lab.local --prop:nsx_ntp_0=192.168.1.230 --
prop:nsx_isSSHEnabled=True --prop:nsx_allowSSHRootLogin=True --X:waitForIp --
X:logFile=/tmp/nsx-manager.log --X:logLevel=verbose /nfs/vmware/vcf/nfs-
mount/bundle/028849ee-d3e7-5748-9b90-47d503c6dd3e/028849ee-d3e7-5748-9b90-
47d503c6dd3e/nsx-unified-appliance-9.0.1.0.24952114.ova
"vi://administrator%40vsphere.local:Success01%210909%21%21@vcenter.lab.local/vcenter-
dc01/host/vcenter-cl01"
```

Important: Use single-line commands. Backslash continuation breaks `--noSSLVerify` and other flags with ovftool 5.0.

4.1.4 VIP Configuration

After NSX Manager boots (~15 minutes for all services to stabilize in nested environments):

1. Open browser to `https://192.168.1.71`
2. Log in as `admin`
3. Navigate to **System > Appliances**
4. Click **Set Virtual IP**
5. Enter VIP: `192.168.1.70`
6. Click **Save**

4.1.5 DNS/NTP via Admin CLI

DNS and NTP on NSX are configured via the **admin CLI**, NOT the UI:

```
# SSH to NSX Manager
ssh admin@192.168.1.71

# Configure DNS
set name-servers 192.168.1.230

# Configure NTP
set ntp-servers 192.168.1.230

# Verify DNS
get name-servers

# Verify NTP
get ntp-servers
```

Warning: Do NOT attempt to configure DNS/NTP via the NSX Manager web UI. Use the admin CLI commands above.

4.1.6 Register Compute Manager

NSX must be connected to vCenter as a compute manager:

1. In NSX Manager UI, navigate to **System > Fabric > Compute Managers**
2. Click **Add**
3. Enter:
 - **Name:** `vcenter.lab.local`
 - **FQDN/IP:** `vcenter.lab.local`
 - **Username:** `administrator@vsphere.local`

- **Password:** (vCenter admin password)
4. Accept the certificate thumbprint
 5. Click **Add**
 6. Wait for the connection status to show **Up** and registration status to show **Registered**

4.1.7 Joining with SDDC Manager

After NSX Manager is deployed, it must be registered in SDDC Manager during the VCF Installer bringup process. The bringup wizard validates:

- NSX VIP is configured
- Compute manager is registered
- DNS is configured
- NTP is configured
- NSX cluster is stable
- NSX version meets minimum (9.0.1 > 4.2.1)

If any of these fail, the bringup will not proceed. Fix NSX health first -- many validation errors are cascading failures from an unhealthy NSX.

4.1.8 Initial Verification

```
# SSH to NSX Manager as admin
ssh admin@192.168.1.71

# Check cluster status
get cluster status

# Check all service statuses
get cluster status verbose

# Verify via API
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status
```

Key services that must be UP:

- MANAGER
- SEARCH
- UI
- NODE_MGMT

- CONTROLLER

Tip: In nested environments, NSX services can take 10-15 minutes to stabilize after restart. If the API returns error 101 "Some appliance components are not functioning properly", wait and retry.

4.2 Transport Node Configuration

4.2.1 Transport Zone Architecture

NSX 9.0 creates default transport zones during deployment:

Transport Zone	Type	Purpose
nsx-overlay-transportzone	Overlay	For GENEVE-encapsulated VM-to-VM traffic
nsx-vlan-transportzone-mgmt	VLAN	For direct VLAN connectivity to physical network

4.2.2 Transport Node Profile Creation

Navigation: NSX Manager > **System** > **Fabric** > **Profiles** > **Transport Node Profiles**

1. Click **Add Profile**
2. **Name:** tn-profile-mgmt
3. **Host Switch:**
 - **Type:** VDS (vSphere Distributed Switch)
 - **VDS Name:** vcenter-cl01-vds01
 - **Transport Zone:** nsx-overlay-transportzone
 - **Uplink Profile:** nsx-default-uplink-hostswitch-profile
4. **IPv4 Assignment:** Select **"Use VMkernel Adapter"** (see next section)
5. Click **Save**

4.2.3 vmk0 TEP Configuration ("Use VMkernel Adapter" -- NSX 9.0 Feature)

NSX 9.0 introduces the **"Use VMkernel Adapter"** option for TEP (Tunnel Endpoint) IP assignment. This allows vmk0 (the management VMkernel) to be reused as the TEP interface, eliminating the need for a dedicated TEP VLAN and IP pool. This is ideal for nested environments and simplified lab deployments.

How it works:

- Instead of allocating separate TEP IPs from an IP pool, NSX uses the existing vmk0 management IP as the TEP address
- Overlay (GENEVE) traffic shares the management network
- No additional VLAN or IP pool configuration needed

IPv4 Assignment options in Transport Node Profile:

Option	Description	When to Use
Use IP Pool	Allocate TEP IPs from a pre-configured IP pool	Production with dedicated TEP VLAN
Use DHCP	Obtain TEP IPs via DHCP	Environments with DHCP on TEP VLAN
Use VMkernel Adapter	Reuse vmk0 management IP as TEP	Nested labs, simplified deployments

4.2.4 Apply Transport Node Profile to Cluster

Navigation: NSX Manager > **System** > **Fabric** > **Nodes** > **Host Transport Nodes**

1. Select the cluster tab (e.g., `vcenter-cl01`)
2. Click **Configure NSX** or **Apply Profile**
3. Select `tn-profile-mgmt` from the dropdown
4. Click **Apply**
5. Monitor the configuration progress for each host

Expected result after successful application:

Host	Status	TEP IP (vmk0)
esxi01.lab.local	Success / Up	192.168.1.74
esxi02.lab.local	Success / Up	192.168.1.75
esxi03.lab.local	Success / Up	192.168.1.76
esxi04.lab.local	Success / Up	192.168.1.82

4.2.5 Troubleshooting Transport Node Failures

If transport node configuration fails in nested environments:

1. **Check NSX Manager health** -- MANAGER/SEARCH services must be UP (32GB RAM minimum)
2. **Remove failed profile** from the cluster before retrying

3. Restart management network on affected hosts:

```
# SSH to ESXi host
ssh root@192.168.1.74

# Restart management network
/etc/init.d/hostd restart
/etc/init.d/vpxa restart
```

4. Remove orphaned host state in NSX Manager if hosts show error 500071 (version conflict)

5. Re-apply the Transport Node Profile

Lab Lesson: The initial transport node application failed because NSX at 24GB RAM / 4 vCPU could not handle the deployment load. After increasing to 32GB / 6 vCPU and powering off SDDC Manager to free resources, re-applying the profile succeeded on all 4 hosts.

4.2.6 Verification Commands

On each ESXi host, verify NSX transport node status:

```
# SSH to ESXi host
ssh root@192.168.1.74

# Check NSX proxy status
/etc/init.d/nsx-proxy status

# Check NSX datapath (DFW)
/etc/init.d/nsx-datapath status

# Check NSX operations agent
/etc/init.d/nsx-opsagent status

# List VMkernel interfaces (confirm vmk50 hyperbus exists)
esxcli network ip interface list

# Check TEP connectivity to another host
vmkping 192.168.1.75

# View NSX logs
tail -50 /var/log/nsx-syslog.log
```

```
# Check NSX agent communication (port 1234)
esxcli network ip connection list | grep 1234
```

VMkernel Network Layout (after transport node config):

VMkernel	Subnet	TCP/IP Stack	Purpose
vmk0	192.168.1.0/24	defaultTcpipStack	Management + NSX TEP (overlay)
vmk1	192.168.11.0/24	vmotion	vMotion
vmk2	192.168.12.0/24	defaultTcpipStack	vSAN
vmk50	169.254.0.0/16	hyperbus	NSX Hyperbus (internal, auto-created)

4.3 Segments, Routing & Security Policies

4.3.1 Segment Types

Segment Type	Requires	Use Case
Overlay Segment	Overlay Transport Zone, Tier-1 Gateway, Subnet/Gateway	VM-to-VM east-west traffic across hosts
VLAN-Backed Segment	VLAN Transport Zone, VLAN ID	Direct VLAN connectivity to physical network

4.3.2 Creating an Overlay Segment

Navigation: NSX Manager > **Networking** > **Segments**

1. Click **Add Segment**
2. **Segment Name:** web-segment
3. **Connected Gateway:** Select a Tier-1 gateway
4. **Transport Zone:** nsx-overlay-transportzone
5. **Subnets:** Click **Set Subnets** and enter gateway IP (e.g., 10.10.10.1/24)
6. Click **Save**

4.3.3 Creating a VLAN-Backed Segment

1. Click **Add Segment**
2. **Segment Name:** VLAN-100-Production
3. **Transport Zone:** Select VLAN transport zone
4. **VLAN ID:** 100
5. Leave Subnets empty (physical network handles DHCP/routing)
6. Click **Save**

Note: VLAN-backed segments do NOT require a Tier-1 gateway connection, subnet gateway IP, or DHCP configuration.

4.3.4 Tier-0 and Tier-1 Gateway Concepts

Tier-0 Gateway (Provider Router):

- North-south routing to external/physical networks
- BGP/OSPF peering with physical routers
- Required for internet/external connectivity
- Provider Networking prerequisite for VCF Automation
- Runs on Edge Nodes (not deployed in this lab)

Tier-1 Gateway (Tenant Router):

- Internal routing for application segments
- NAT services
- Load balancing
- Connected to Tier-0 for external access
- Can be distributed (on hosts) or centralized (on Edge)

4.3.5 Distributed Firewall (DFW)

The DFW enforces micro-segmentation at the VM vNIC level for east-west traffic. Rules are processed in this order:

Priority	Category	Purpose
1	Emergency	Critical security policies
2	Infrastructure	Protect infrastructure components
3	Environment	Zone-based policies

4	Application	App-specific micro-segmentation
5	Default	Catch-all rules

Within each category: Rules process TOP to BOTTOM. First match wins.

4.3.6 Creating DFW Rules

Navigation: NSX Manager > **Security** > **Distributed Firewall**

1. Select category (Emergency, Infrastructure, Environment, Application)
2. Click **Add Policy** > **Add Rule**
3. Configure:
 - **Name:** Descriptive rule name
 - **Sources:** Groups, VMs, IPs
 - **Destinations:** Groups, VMs, IPs
 - **Services:** Ports/protocols
 - **Applied To:** Scope (specific groups or DFW)
 - **Action:** Allow, Drop, or Reject
4. Click **Publish** (required for rules to take effect)

4.3.7 Tag-Based Security Groups (Best Practice)

Instead of using IP-based rules (which break when VMs move), use NSX tags:

1. Navigate to **Inventory** > **Groups** > **Add Group**
2. **Name:** Web-Servers
3. Click **Set Members** > **Membership Criteria**
4. Add criteria: **Tag Equals** web-tier
5. Click **Save**

Apply tags to VMs:

1. Navigate to **Inventory** > **Virtual Machines**
2. Select VM(s)
3. Click **Actions** > **Add Tags**
4. Enter tag: web-tier
5. Click **Save**

4.4 NSX Monitoring Integration

4.4.1 Automatic Discovery via VCF Account

When you configure a VCF Cloud Account in VCF Operations (see Section 3.4), NSX adapters are automatically discovered and configured for all domains that have NSX deployed. No manual configuration is needed.

4.4.2 Verify NSX Adapter Status

Navigation: VCF Operations > **Administration** > **Integrations** > **Accounts**

1. Navigate to the Accounts tab
2. Expand the **VMware Cloud Foundation** account
3. Find the NSX adapter listed under the management domain
4. Verify the collection status shows green "Collecting"

4.4.3 NSX Monitoring Features in VCF Operations

The NSX adapter retrieves alerts and findings from NSX into VCF Operations. VCF 9.0 includes enhanced NSX monitoring:

Feature	Description
Enhanced Edge Node Monitoring	New edge node metrics sub-groups
Network Operations Overview	vSphere networking and NSX inventory summary
Network Alert Trends	Visibility into network alerts over time
Transport Node Status	Real-time health of all transport nodes
Segment Health	Overlay and VLAN segment connectivity status

4.4.4 Configure VCF Operations for Networks (Advanced)

For deeper network monitoring capabilities:

Navigation: VCF Operations > **Administration** > **Integrations** > **Repository**

1. Navigate to **Administration** > **Integrations** > **Repository** tab
2. Find the VCF Operations for Networks management pack in **Available Integrations**
3. Click **Activate** on the management pack card
4. After activation, click **Add Account** to configure the adapter instance
5. Enter the connection details for your VCF Operations for Networks instance

Important: Starting from VCF 9.0, only ONE VCF Operations for Networks instance integration is supported. During deployment, VCF Operations Fleet Management integrates VCF Operations and VCF Operations for Networks automatically.

4.4.5 Key NSX Metrics to Monitor

Metric Category	Key Indicators
Transport Node	Configuration state, connection status, TEP reachability
NSX Manager	Service health (MANAGER, SEARCH, UI, CONTROLLER, NODE_MGMT)
DFW	Rule hit counts, dropped packets, policy publish status
Segments	Port count, traffic throughput, MAC learning
Edge Nodes	CPU/memory utilization, throughput, session counts

4.5 NSX Troubleshooting Quick Reference

4.5.1 OOM Issues in Nested Environments

Symptom: NSX Manager console shows repeated `sysrq: Show Memory` messages, all NSX-related validation checks fail.

Diagnosis:

```
# Check NSX Manager memory from vCenter
# VM > Monitor > Performance > Memory

# Check service health via API
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status
```

Resolution:

1. Power off NSX Manager VM in vCenter
2. Edit Settings > increase RAM to 32GB, CPU to 6 vCPU
3. Power on and wait 15 minutes for all services to stabilize

4. Verify via API that all services show **RUNNING**

4.5.2 Transport Node Connectivity Issues

Step 1: Check status in NSX Manager

Navigate to **System > Fabric > Nodes > Host Transport Nodes** and review status (green/yellow/red).

Step 2: Test TEP connectivity from ESXi host

```
# SSH to ESXi host
ssh root@192.168.1.74

# Find TEP VMkernel
esxcfg-vmknics -l | grep -i tep

# For vmk0-as-TEP configuration, test management connectivity
vmkping 192.168.1.75

# Test with MTU 1600 (GENEVE overhead requires 1600+ bytes)
vmkping -d -s 1572 192.168.1.75
```

Step 3: Check NSX agent on host

```
/etc/init.d/nsx-proxy status
/etc/init.d/nsx-datapath status
tail -50 /var/log/nsx-syslog.log
```

Step 4: Resync transport node

In NSX Manager > **System > Fabric > Nodes**, click problematic host > **Actions > Redeploy Node**.

4.5.3 Service Status Checks

NSX Manager CLI (SSH as admin):

```
# Overall cluster status
get cluster status

# Detailed service list
```

```
get cluster status verbose

# Get manager node list
get managers

# Get all transport nodes
get transport-nodes
```

NSX Manager API:

```
# Cluster status
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status

# Transport node status
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/transport-nodes

# Transport node state
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/transport-
nodes/state

# Compute managers
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/fabric/compute-
managers

# List certificates
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/trust-
management/certificates

# Node UUID (from cluster info)
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster
```

Important: NSX shell does NOT support backslash line continuation. All curl commands must be single-line.

4.5.4 NSX Certificate Replacement

The default NSX self-signed certificate may not include proper SAN entries. VDT will report FAIL if the certificate SAN does not include the FQDN that SDDC Manager uses to register NSX.

Step 1: Create OpenSSL config on NSX Manager (SSH as root):

```

cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF

```

Critical: `DNS.3 = nsx-manager.lab.local` is required because SDDC Manager registers NSX using this FQDN. Without it, VDT reports SAN check failure.

Step 2: Generate certificate and key:

```

openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256

```

Step 3: Verify SAN entries:

```
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"
```

Step 4: Create JSON payload using Python (avoids shell PEM escaping issues):

```
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

Step 5: Import certificate into NSX:

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates?action=import" -H "Content-Type: application/json" -d
@/tmp/nsx-import.json
```

Note the certificate ID from the response (e.g., `701d1416-5054-4038-8749-4ac495980ebd`).

Step 6: Get node UUID:

```
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster
```

Step 7: Apply to NSX Manager node:

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<CERT-ID>?
action=apply_certificate&service_type=API&node_id=<NODE-UUID>"
```

Step 8: Apply to cluster VIP:

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<CERT-ID>?"
```

```
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Step 9: Import into SDDC Manager trust stores (SSH to SDDC Manager as root):

```
# Pull active NSX certificate
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# Import into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt

# Import into Java cacerts
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Restart SDDC Manager services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

Services take ~5 minutes to restart. After restart, re-run VDT to confirm NSX cert trust checks pass.

Reference: <https://knowledge.broadcom.com/external/article/316056>

4.5.5 Common NSX Shell Commands Reference

Task	Command
Cluster status	<code>get cluster status</code>
Manager list	<code>get managers</code>
Transport nodes	<code>get transport-nodes</code>
Logical switches (segments)	<code>get logical-switches</code>
Logical routers (gateways)	<code>get logical-routers</code>
VTEP information	<code>get vtep</code>

VTEP table	<code>get vtep-table</code>
Firewall rules	<code>get firewall rules</code>
Firewall status	<code>get firewall status</code>
Interfaces	<code>get interfaces</code>
Set DNS	<code>set name-servers <ip></code>
Set NTP	<code>set ntp-servers <ip></code>

4.5.6 ESXi Host NSX Service Commands

Task	Command
NSX proxy status	<code>/etc/init.d/nsx-proxy status</code>
Restart NSX proxy	<code>/etc/init.d/nsx-proxy restart</code>
NSX datapath status	<code>/etc/init.d/nsx-datapath status</code>
NSX operations agent	<code>/etc/init.d/nsx-opsagent status</code>
View NSX logs	<code>tail -50 /var/log/nsx-syslog.log</code>
Check NSX port 1234 connections	<code>esxcli network ip connection list grep 1234</code>
List VMkernel interfaces	<code>esxcli network ip interface list</code>
List DVS info	<code>esxcli network vswitch dvs vmware list</code>

4.5.7 NSX Port Requirements

Port	Protocol	Purpose
443	TCP	NSX Manager UI and API
1234	TCP	NSX agent communication (host to manager)
1235	TCP	NSX cluster inter-node
6081	UDP	GENEVE overlay encapsulation
8080	TCP	NSX Manager internal API

4.5.8 Traceflow for Network Diagnostics

Navigation: NSX Manager > **Plan & Troubleshoot** > **Traffic Analysis** > **Traceflow**

1. Select Source (VM or IP)
2. Select Destination (VM or IP)
3. Select Protocol (ICMP, TCP, UDP)
4. For TCP/UDP, specify destination port
5. Click **Trace**

Interpreting Results:

Result	Action
Green line	Path working -- check application layer
Red X (DFW rule)	Check firewall rule ordering and policies
Red X (TEP unreachable)	Check physical network, MTU, VLAN configuration
Red X (No route)	Check Tier-0/Tier-1 routing configuration

PART V: vSAN Storage

5.1 vSAN ESA Configuration

vSAN Express Storage Architecture (ESA) is the default storage architecture in VCF 9.0, replacing the older Original Storage Architecture (OSA). ESA eliminates the distinction between cache and capacity tiers, treating all devices as a single flat storage pool with software-managed caching.

5.1.1 ESA vs OSA Comparison

Feature	vSAN OSA	vSAN ESA
Disk Groups	Cache + Capacity tiers	Single flat pool (no disk groups)
Cache Devices	Dedicated SSD for cache	No dedicated cache — software-managed
Capacity Devices	SSD or HDD	NVMe SSDs only (production)
RAID Support	RAID-1/5/6	RAID-1/5/6 with native snapshots
Compression	Dedup + Compression (capacity tier)	Always-on compression
Erasur Coding	Available	Improved efficiency
Performance	Depends on cache tier sizing	Consistent — all devices contribute
Minimum Disks per Host	1 cache + 1 capacity	1 storage device
Nested Lab Support	VMX virtualSSD flag	VMX virtualSSD flag + HCL bypass

5.1.2 vSAN ESA Bypass for Nested Environments

VCF 9.0.1 includes a built-in bypass for vSAN ESA HCL validation, eliminating the need for the mock VIB that was required in earlier versions. This bypass allows virtual SATA disks marked as SSD in the VMX file to be claimed by vSAN ESA.

Step 1: Mark virtual disks as SSD in VMX files

Edit each ESXi VM's `.vmx` file in VMware Workstation (VM must be powered off):

```
# Add to each ESXi VM's VMX file
sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
```

For esxi01 only (has an extra disk):

```
sata0:3.virtualSSD = "1"
```

VMX file locations in this lab:

```
D:\VMs\esxi01.lab.local\esxi01.lab.local.vmx
E:\VMs\esxi02.lab.local\esxi02.lab.local.vmx
E:\VMs\esxi03.lab.local\esxi03.lab.local.vmx
F:\VMs\esxi04.lab.local\esxi04.lab.local.vmx
```

Step 2: Enable the vSAN ESA HCL bypass on the VCF Installer

SSH to the VCF Installer (192.168.1.240) as root:

```
# Add the vSAN ESA HCL bypass property
echo "vsan.esa.sddc.managed.disk.claim=true" >>
/etc/vmware/vcf/domainmanager/application-prod.properties

# Restart the domain manager service to apply
systemctl restart domainmanager

# Verify the property was written
cat /etc/vmware/vcf/domainmanager/application-prod.properties | grep vsan
```

Important: This bypass must be applied *BEFORE* running the VCF Installer wizard. If the wizard has already been started, restart `domainmanager` and refresh the browser.

Step 3: Verify SSD detection on ESXi hosts after power-on

SSH to each ESXi host and confirm disks are recognized as SSD:

```
# Check SSD status for all storage devices
esxcli storage core device list | grep -E "Display Name|Is SSD"
```

```
# Expected output for each disk:
#   Display Name: Local ATA Disk (t10.ATA...)
#   Is SSD: true
```

5.1.3 SSD Detection and SATP Claim Rules

If virtual disks are not detected as SSD even after setting `virtualSSD` in the VMX file, use SATP (Storage Array Type Plugin) claim rules to force SSD detection:

```
# List current SATP rules filtering for SSD
esxcli storage nmp satp rule list | grep enable_ssd

# Add a claim rule to mark a specific device as SSD
esxcli storage nmp satp rule add -s VMW_SATP_LOCAL \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001 \
  -o enable_ssd

# Reclaim the device to apply the new rule
esxcli storage core claiming reclaim \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001

# Verify the device is now marked as SSD
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001 | grep "Is
SSD"
```

Note: SATP claim rules persist across reboots. The VMX `virtualSSD` approach is preferred because it marks the disk at the hardware emulation level before ESXi boots.

5.1.4 Storage Policy Creation

VCF Installer automatically creates a default vSAN storage policy during deployment. For nested labs with only 4 hosts, the default policy uses:

Policy Setting	Value
Failures to Tolerate (FTT)	1
Failure Tolerance Method	RAID-1 (Mirroring)

Object Space Reservation

Thin provisioning

To create a custom storage policy in vCenter:

1. Navigate to `https://vcenter.lab.local` > **Policies and Profiles** > **VM Storage Policies**
2. Click **Create**
3. Name: `vSAN-thin-FTT1`
4. Under **vSAN** rules:
 - Failures to tolerate: 1
 - Failure tolerance method: RAID-1 (Mirroring)
 - Force provisioning: No
5. Select compatible datastores: `vcenter-cl01-ds-vsan01`
6. Click **Finish**

5.1.5 vSAN Datastore Verification

After VCF Installer completes deployment, verify the vSAN datastore:

```
# On any ESXi host, list vSAN storage
esxcli vsan storage list

# Check vSAN cluster membership
esxcli vsan cluster get

# List datastores visible to the host
esxcli storage filesystem list | grep -i vsan

# Verify datastore is accessible in vCenter
# Navigate to: vcenter.lab.local > vcenter-dc01 > vcenter-cl01 > Datastores
# Datastore name: vcenter-cl01-ds-vsan01
```

5.2 Disk Management & Cleanup

5.2.1 Disk Identification Commands

```
# Comprehensive disk query with vSAN eligibility
vdq -iH

# Quick eligibility check
vdq -q

# List all vSAN storage devices and their state
esxcli vsan storage list

# List all storage devices with full details
esxcli storage core device list

# Filter for device name and SSD status
esxcli storage core device list | grep -E "^t10|^naa|Display Name|Is SSD|Size"

# Check partition tables on a specific disk
partedUtil getptbl
/vmfs/devices/disks/t10.ATA VMware Virtual SATA Hard Drive 0300000000000000
```

Sample `vdq -q` output for an eligible disk:

```
{
  "Name":
    "t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001",
  "State": "Eligible for use by VSAN",
  "Reason": "None",
  "IsSSD": "1"
}
```

Sample output for an ineligible disk:

```
{
  "Name":
    "t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001",
  "State": "Ineligible for use by VSAN",
  "Reason": "Has partitions",
  "IsSSD": "1"
}
```

5.2.2 Adding and Removing Disks

Removing a disk from vSAN:

```
# Remove a specific disk from vSAN storage
esxcli vsan storage remove -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000001

# Verify removal
esxcli vsan storage list
vdq -q
```

Cleaning up old vSAN partitions (required after failed deployments):

```
# Check existing partitions
partedUtil getptbl
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000006

# Delete partition 1
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000006
1

# Delete partition 2
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000006
2

# Verify disk is now eligible
vdq -q
```

Warning: Deleting partitions destroys all data on those partitions. Only use this procedure on disks that are being reclaimed for a fresh vSAN deployment.

5.2.3 Disk Group Management (OSA Only)

vSAN ESA does not use disk groups. For environments still running vSAN OSA, disk groups consist of one cache device and one or more capacity devices:

```
# List current disk groups
esxcli vsan storage list

# Remove an entire disk group by specifying the cache disk
esxcli vsan storage remove -d <cache-disk-device-name>
```

5.2.4 Orphaned Object Cleanup

Orphaned vSAN objects can occur after VM deletions or failed migrations:

1. In vCenter, navigate to **Cluster > Monitor > vSAN > Virtual Objects**
2. Filter for objects with status "Inaccessible" or "Orphaned"
3. Select orphaned objects and click **Delete**

From the command line:

```
# List vSAN objects on a host
esxcli vsan debug object list

# Check for inaccessible objects
esxcli vsan debug object health summary get
```

5.3 Storage Migration (Thick to Thin)

5.3.0 Why SDDC Manager Starts on Local Storage (Bootstrap Constraint)

This is a **chicken-and-egg problem** inherent to every VCF deployment:

1. The **VCF Installer OVA** (which is the same OVA as SDDC Manager — dual purpose) must be deployed **before** the bringup process runs
2. **vSAN does not exist yet** at this point — vSAN is created *during* the bringup process when the VCF Installer orchestrates the deployment of vCenter, vSAN, and VDS across the ESXi hosts
3. The only storage available before bringup is the **local datastore** on the ESXi host where you deploy the installer (`esxi01-local` in the lab)
4. After bringup completes, the VCF Installer **transforms into SDDC Manager** — still sitting on local storage where it was originally deployed

This means SDDC Manager is always initially deployed to local storage and must be manually migrated to shared storage (vSAN) afterward. In the lab, this was done during Phase 7 (Feb 10–11) after the management domain bringup was complete.

Resource contention: In a nested lab, this is especially problematic because esxi01 ends up hosting both SDDC Manager and other large VMs (like NSX Manager at 32GB RAM) on its local datastore, with no ability to vMotion until the migration to shared storage is complete.

5.3.1 The Problem

The vCenter migration wizard cannot thin-provision virtual disks when migrating to a vSAN datastore. When you attempt to migrate a thick-provisioned VM using the vCenter storage migration wizard and select "thin provisioning," the disks remain at their full allocated size on vSAN. This is particularly problematic for VMs like SDDC Manager that allocate far more disk space than they actually use.

In this lab, SDDC Manager had 6 disks totaling 914GB allocated but only ~108GB of actual data:

Disk	Allocated	Actual Used
sddc-manager.vmdk	32GB	2.6GB
sddc-manager_1.vmdk	16GB	2.6GB
sddc-manager_2.vmdk	240GB	3.0GB
sddc-manager_3.vmdk	512GB	99.5GB
sddc-manager_4.vmdk	26GB	30MB
sddc-manager_5.vmdk	88GB	64MB
Total	914GB	~108GB

5.3.2 Solution: vmkfstools Per-Disk Migration

The `vmkfstools -i` command with the `-d thin` flag creates a true thin-provisioned copy of each virtual disk. This must be done per-disk from the ESXi shell.

Prerequisites:

- VM must be powered off
- SSH access to the ESXi host where the VM resides
- Sufficient space on the destination datastore for the actual data (not the allocated size)

5.3.3 Step-by-Step Procedure

Step 1: Power off the VM in vCenter

Step 2: SSH to the ESXi host where the VM is registered

```
ssh root@192.168.1.74
```

Step 3: Create the destination directory on vSAN

```
mkdir -p /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
```

Step 4: Clone each disk as thin provisioned

```
# Disk 0 (32GB allocated, 2.6GB actual)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmdk -d thin

# Disk 1 (16GB allocated, 2.6GB actual)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_1.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_1.vmdk -d thin

# Disk 2 (240GB allocated, 3.0GB actual)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_2.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_2.vmdk -d thin

# Disk 3 (512GB allocated, 99.5GB actual) – LARGEST DISK, takes longest
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_3.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_3.vmdk -d thin

# Disk 4 (26GB allocated, 30MB actual)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_4.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_4.vmdk -d thin

# Disk 5 (88GB allocated, 64MB actual)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_5.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_5.vmdk -d thin
```

Warning: Disk 3 (512GB/99.5GB) failed on the first attempt due to a host disconnect during the clone. If a clone fails partway through, delete the partial copy before retrying:

```
vmkfstools -U /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-
manager_3.vmdk
```

Then retry the clone command.

Step 5: Copy configuration files

```
# Copy VMX, NVRAM, and VMSD files
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmx /vmfs/volumes/vcenter-
cl01-ds-vsan01/sddc-manager/
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.nvram
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
cp /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmsd /vmfs/volumes/vcenter-
cl01-ds-vsan01/sddc-manager/
```

5.3.4 Post-Migration Verification

```
# Verify thin provisioned disks on vSAN
ls -la /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/

# Check actual disk usage (thin should show much less)
du -sh /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/
```

5.3.5 VM Reconfiguration After Migration

Step 1: Unregister the old VM from inventory

In vCenter, right-click the VM > **Remove from Inventory** (NOT "Delete from Disk" -- you want to keep the original files as a backup).

Step 2: Register the new VM from vSAN

In vCenter, navigate to **Datastores** > `vcenter-cl01-ds-vsan01` > **Browse Files** > `sddc-manager/` > right-click `sddc-manager.vmx` > **Register VM**.

Step 3: Power on and verify

Power on the VM from vCenter and verify it boots correctly. All services should start normally since the disk contents are identical -- only the provisioning format changed.

Step 4: Clean up the original files (optional, after confirming success)

```
# Only after confirming the migrated VM works correctly
rm -rf /vmfs/volumes/esxi01-local/sddc-manager/
```

5.4 vSAN Monitoring & Health

5.4.1 VCF Operations Integration for vSAN Monitoring

When you configure a VCF Cloud Account or vCenter account in VCF Operations that points to a vSAN-enabled cluster, vSAN monitoring data is automatically collected. No separate configuration is required.

Access vSAN Storage Operations Dashboard:

Navigation: VCF Operations > **Infrastructure Operations** > **Storage Operations**

The centralized storage dashboard shows:

- vSAN Cluster Health Score
- Cluster type classification (ESA/OSA)
- Performance metrics: hover on any cluster pin to see IOPS, Latency, and Throughput
- vSAN Clusters table with detailed cluster information

Predefined vSAN Dashboards:

Navigation: VCF Operations > **Infrastructure Operations** > **Dashboards & Reports**

- **vSAN OSA Performance Dashboard** -- read/write latency, contention, utilization across VMs
- **vSAN ESA Performance Dashboard** -- same metrics for Express Storage Architecture
- **vSAN Configuration Dashboard** -- configuration details and recommendations

Run vSAN Performance Diagnostics:

1. On the Storage Operations page, click **View Diagnostics** or **Run New Diagnostics**
2. Select the cluster (`vcenter-cl01`)
3. Choose diagnostic mode:
 - **Troubleshooting:** For clusters with active workloads
 - **Benchmarking and Optimizing:** For new clusters before deploying workloads
4. Review results: cluster information, diagnostic results, remediation steps

Note: Diagnostic reports are available for the past 7 days only. Diagnostics run on both vSAN OSA and ESA HCI architectures.

5.4.2 vSAN Health Check Commands (ESXi Shell)

```
# Check vSAN cluster health summary
esxcli vsan health cluster list

# Run a specific health check
esxcli vsan health cluster get -t "Network health"

# Check vSAN cluster membership
esxcli vsan cluster get

# List all vSAN storage devices and their state
esxcli vsan storage list

# Check resync status
esxcli vsan debug resync summary get

# Check vSAN object health
esxcli vsan debug object health summary get
```

5.4.3 Key Metrics to Monitor

Metric	Location	Threshold
Network Latency	vSAN Health > Network	< 5ms (will be yellow in nested labs)
Disk Latency	vSAN Health > Physical Disk	< 10ms read, < 20ms write
Congestion	vSAN Health > Performance	< 30 (0-255 scale)

Capacity Utilization	vSAN Capacity	< 80% (warning at 70%)
Component Health	vSAN Health > Data	All objects healthy
Resync Operations	Monitor > vSAN > Resyncing Objects	Should be 0 during steady state

5.4.4 esxtop for Storage Performance

```
# Launch esxtop in disk (storage) mode
esxtop

# Press 'u' to switch to disk device view
# Press 'v' to switch to disk VM view
```

Key esxtop storage metrics:

Column	Meaning
CMDS/s	Total commands per second
READS/s	Read operations per second
WRITES/s	Write operations per second
MBREAD/s	Read throughput in MB/s
MBWRTN/s	Write throughput in MB/s
LAT/rd	Average read latency (ms)
LAT/wr	Average write latency (ms)
KAVG/rd	Kernel average read latency
GAVG/rd	Guest average read latency

5.4.5 vSAN Observer

vSAN Observer provides real-time and historical performance data through a web-based interface. It is available through the Ruby vSphere Console (RVC):

```
# Connect to RVC from vCenter shell
rvc administrator@vsphere.local@localhost
```

```
# Navigate to cluster
cd /vcenter.lab.local/vcenter-dc01/computers/vcenter-cl01

# Start vSAN Observer
vsan.observer . --run-webserver --force
```

The observer starts a web server (typically on port 8010) that can be accessed from a browser.

5.5 vSAN Troubleshooting

5.5.1 Common vSAN Issues in Nested Environments

Network Latency (Expected Yellow)

vSAN health check shows yellow on "Network latency check" -- this is normal and expected for nested ESXi in VMware Workstation. Typical latency values in this lab:

From Host	To Host	Latency (ms)	Threshold (ms)
192.168.12.122	192.168.12.123	6.81	5
192.168.12.123	192.168.12.122	6.32	5
192.168.12.123	192.168.12.120	6.61	5
192.168.12.123	192.168.12.121	6.15	5

Even "passing" pairs average 4.48ms latency, which is high for physical hosts but typical for virtualized NICs. This remains yellow and does not affect functionality.

5.5.2 Disk Not Detected

Symptom: `esxcli storage core device list` shows `Is SSD: false` for virtual disks, or `vdq -q` shows "Ineligible for use by VSAN."

Diagnosis:

```
# Check if disk is seen at all
esxcli storage core device list | grep -E "^t10|Is SSD"
```

```
# Check vSAN eligibility
vdq -q

# Check for stale partitions
partedUtil getptbl /vmfs/devices/disks/<device-name>
```

Resolution (in order of preference):

1. **VMX virtualSSD flag:** Power off the ESXi VM, add `sata0:X.virtualSSD = "1"` to the VMX file, power on
2. **SATP claim rule:** `esxcli storage nmp satp rule add -s VMW_SATP_LOCAL -d <device> -o enable_ssd` then `esxcli storage core claiming reclaim -d <device>`
3. **Clean partitions:** If disk shows "Has partitions," use `partedUtil delete` to remove old partitions

5.5.3 vSAN Network Partition

Symptom: vSAN health shows "Network partition" or hosts appear to be in different sub-clusters.

Diagnosis:

```
# Check vSAN cluster membership
esxcli vsan cluster get

# Check network connectivity between vSAN VMkernel ports
vmkping -I vmk2 192.168.12.120
vmkping -I vmk2 192.168.12.121
vmkping -I vmk2 192.168.12.122
vmkping -I vmk2 192.168.12.123

# Check VMkernel adapter status
esxcli network ip interface list
```

Resolution:

- Verify vmk2 (vSAN VMkernel) is up on all hosts
- Check that vSAN port group VLAN is correct

- In nested environments, ensure VMware Workstation promiscuous mode is enabled:
`ethernet*.noPromisc = "FALSE"`

5.5.4 Object Health Issues

Symptom: vSAN objects show as "Degraded," "Reduced Availability," or "Inaccessible."

```
# Check object health summary
esxcli vsan debug object health summary get

# List objects with issues
esxcli vsan debug object list

# In vCenter: Cluster > Monitor > vSAN > Virtual Objects
# Filter for non-healthy objects
```

Resolution:

- Degraded/Reduced availability: Usually resolves automatically when all hosts are online and resync completes
- Inaccessible: Check that all hosts contributing to the object are online. If a host is permanently lost, data may be unrecoverable

5.5.5 Resync Monitoring

After host maintenance, disk replacement, or policy changes, vSAN resyncs objects:

```
# Check resync summary
esxcli vsan debug resync summary get

# In vCenter: Cluster > Monitor > vSAN > Resyncing Objects
# Shows: Objects resyncing, bytes remaining, ETA
```

Tip: Do not put another host into maintenance mode while resync is in progress. Wait for resync to complete (0 resyncing objects) before proceeding.

5.5.6 vSAN Trace Files and Logging

```
# vSAN trace files location
ls /var/log/vmkernel.log | head
ls /var/log/vobd.log | head

# Search for vSAN-related errors in vmkernel log
grep -i "vsan\|cmm\|clom\|dom\|lsom" /var/log/vmkernel.log | tail -50

# vSAN specific logs
cat /var/log/vsanmgmt.log | tail -50
cat /var/log/vsantraced.log | tail -50

# Check vSAN daemon status
/etc/init.d/vsanmgmt status
/etc/init.d/vsand status
```

Key vSAN log abbreviations:

Abbreviation	Full Name	Purpose
CMMDS	Cluster Monitoring, Membership, and Directory Service	Cluster membership
CLOM	Cluster Level Object Manager	Object placement
DOM	Distributed Object Manager	Object I/O
LSOM	Local Log-Structured Object Manager	Local disk I/O
RDT	Reliable Datagram Transport	vSAN network transport

PART VI: Security, Certificates & Compliance

6.1 Certificate Architecture in VCF

6.1.1 How Certificates Flow in VCF

VCF uses TLS certificates for secure communication between all platform components. In VCF 9.0, certificate management is centralized through VCF Operations (Fleet Management > Certificates), replacing the certificate management previously found in SDDC Manager.

The certificate trust chain works as follows:

1. **SDDC Manager** maintains an inventory of all component certificates and their trust relationships
2. Each component (vCenter, NSX, ESXi, SDDC Manager, VCF Operations) has its own TLS certificate
3. SDDC Manager stores trusted root certificates in two keystores that must both be updated when certificates change
4. VCF Operations queries SDDC Manager's inventory to display certificate status across the fleet

Components and their certificate locations:

Component	Certificate Location	Type
ESXi Hosts	<code>/etc/vmware/ssl/rui.crt</code> and <code>rui.key</code>	Self-signed (auto-generated)
vCenter Server	VMCA-managed (internal)	VMCA-signed
NSX Manager	Internal keystore, managed via API	Self-signed or CA-signed
SDDC Manager	<code>/etc/vmware/vcf/commonsvcs/</code>	Self-signed or CA-signed
VCF Operations	Internal keystore	Self-signed or CA-signed

6.1.2 Self-Signed vs CA-Signed Certs

Aspect	Self-Signed	CA-Signed
Trust	Must be manually imported into trust stores	Automatically trusted if CA root is in trust stores
Complexity	Low — generated locally	Higher — requires CA infrastructure

VDT Validation	Passes if SAN/trust store entries are correct	Passes inherently
Renewal	Manual	Can be automated via VCF Operations
Production Use	Not recommended	Required
Lab Use	Acceptable	Optional

6.1.3 Certificate Lifecycle

Certificates in VCF have the following lifecycle stages:

1. **Generation:** Created during component deployment (self-signed) or issued by CA
2. **Deployment:** Applied to the component's TLS endpoint
3. **Trust Establishment:** Root/issuing CA imported into all consumers' trust stores
4. **Monitoring:** VCF Operations tracks expiration dates and SAN validity via VCF Health
5. **Renewal/Replacement:** Before expiration, certificate is renewed or replaced
6. **Revocation:** If compromised, certificate is revoked and replaced

VCF Operations supports auto-renewal for: ESX SSL, vCenter machine SSL, NSX LM/VIP, SDDC Manager SSL, and VCF Operations certificates.

6.1.4 Which Components Use Which Certs

Communication Path	Certificate Used	Trust Required By
Browser to vCenter	vCenter machine SSL	Browser
Browser to NSX Manager	NSX API certificate	Browser
SDDC Manager to vCenter	vCenter machine SSL	SDDC Manager trust stores
SDDC Manager to NSX	NSX API/VIP certificate	SDDC Manager trust stores
vCenter to ESXi	ESXi rui.crt	vCenter VMCA trust
NSX to ESXi (transport nodes)	ESXi rui.crt + NSX node cert	Mutual trust
VCF Operations to SDDC Manager	SDDC Manager SSL cert	VCF Operations

6.2 NSX Certificate Replacement (CRITICAL -- Full Procedure)

This is the most complex certificate operation in VCF. The default NSX self-signed certificate generated during ovftool deployment uses a wildcard SAN (`*.lab.local`) without specific hostnames or IPs, causing VDT

to report failures. This section documents the complete, lab-tested procedure for replacing the NSX certificate.

Critical: The SAN must include `nsx-manager.lab.local` (the FQDN that SDDC Manager uses to register NSX), not just `nsx-node1.lab.local`. Without it, VDT reports "SAN contains IP but not hostname" because it looks up the registered FQDN and does not find it in the certificate SAN.

6.2.1 OpenSSL Configuration File

SSH to the NSX Manager as root and create the OpenSSL configuration file:

```
ssh root@192.168.1.71
```

```
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF
```

Explanation of each SAN entry:

Entry	Purpose
DNS.1 = nsx-vip.lab.local	NSX Virtual IP FQDN (cluster access point)
DNS.2 = nsx-node1.lab.local	NSX Manager node FQDN (direct node access)
DNS.3 = nsx-manager.lab.local	SDDC Manager's registered FQDN for NSX -- REQUIRED
IP.1 = 192.168.1.70	NSX VIP IP address
IP.2 = 192.168.1.71	NSX Manager node IP address

Important: If you have multiple NSX Manager nodes (HA deployment), add DNS and IP entries for each node (DNS.4, DNS.5, IP.3, IP.4, etc.).

6.2.2 Certificate Generation Commands**Generate a new self-signed certificate and private key:**

```
openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256
```

Verify the certificate SAN entries:

```
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"
```

Expected output:

```
X509v3 Subject Alternative Name:
  DNS:nsx-vip.lab.local, DNS:nsx-node1.lab.local, DNS:nsx-manager.lab.local, IP
  Address:192.168.1.70, IP Address:192.168.1.71
```

Verify the certificate details:

```
# Check subject, issuer, validity period
openssl x509 -in /tmp/nsx.crt -text -noout | head -20

# Check key type and size
openssl x509 -in /tmp/nsx.crt -text -noout | grep "Public-Key"
```

6.2.3 Import via NSX API

The NSX API requires the certificate and private key as a JSON payload with PEM-encoded strings. Shell escaping of PEM data (which contains newlines) is error-prone, so a Python script is used to build the JSON correctly.

Build the JSON payload using Python:

```
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

Why Python? NSX shell does NOT support backslash line continuation. All curl commands must be single-line. Python avoids the shell escaping issues with `\n` characters embedded in PEM data that would break a `curl -d '...' payload`.

Verify the JSON was built correctly:

```
python -c "import json; d=json.load(open('/tmp/nsx-import.json')); print('cert lines:', d['pem_encoded'].count('\n'), 'key lines:', d['private_key'].count('\n'))"
```

Import the certificate into NSX (single-line curl -- mandatory):

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-management/certificates?action=import" -H "Content-Type: application/json" -d
```

```
@/tmp/nsx-import.json
```

The response includes a certificate ID. Example:

```
{
  "results": [
    {
      "id": "701d1416-5054-4038-8749-4ac495980ebd",
      ...
    }
  ]
}
```

Record the certificate ID (`701d1416-5054-4038-8749-4ac495980ebd` in this lab) -- it is needed for the apply step.

Prerequisite: All NSX services must be healthy (MANAGER, SEARCH, UI, NODE_MGMT all UP). If services are DOWN, the API returns error 101: "Some appliance components are not functioning properly." Check service status:

```
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status
```

Services can take 10-15 minutes to stabilize after NSX restart in nested environments.

6.2.4 Apply Certificate

The certificate must be applied in two steps: first to the NSX Manager node (API service), then to the cluster VIP (MGMT_CLUSTER).

Step 1: Get the node UUID from cluster info:

```
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster
```

From the response, extract the node UUID. In this lab: `95493642-ef4a-cb8e-ed7c-5bc20033f2c2`

Step 2: Apply certificate to NSX Manager node (API service):

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/701d1416-5054-4038-8749-4ac495980ebd?
action=apply_certificate&service_type=API&node_id=95493642-ef4a-cb8e-ed7c-
5bc20033f2c2"
```

Expected response: empty body with HTTP 200 -- this means success.

Important: Apply to the node FIRST, then to the VIP. Applying in the wrong order can cause connectivity issues.

Step 3: Apply certificate to the cluster VIP (MGMT_CLUSTER):

```
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/701d1416-5054-4038-8749-4ac495980ebd?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Expected response: empty body with HTTP 200 -- success.

Step 4: Verify the new certificate is active on both endpoints:

```
# Verify node certificate (.71)
openssl s_client -connect 192.168.1.71:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"

# Verify VIP certificate (.70)
openssl s_client -connect 192.168.1.70:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

Both should show:

```
X509v3 Subject Alternative Name:
      DNS:nsx-vip.lab.local, DNS:nsx-node1.lab.local, DNS:nsx-manager.lab.local, IP
Address:192.168.1.70, IP Address:192.168.1.71
```

6.2.5 Trust Store Updates

After replacing the NSX self-signed certificate, the new certificate's root is NOT in SDDC Manager's trust stores. The old NSX cert was pre-trusted during bringup; the new self-signed cert must be explicitly imported into both SDDC Manager keystores.

SSH to SDDC Manager:

```
# Only the vcf user can SSH in (root and admin are rejected)
ssh vcf@192.168.1.241

# Switch to root
su -
```

Note on file transfers: SCP does not work with SDDC Manager due to the restricted shell. Use `ssh vcf@host "cat > file" < localfile` for file transfers.

Step 1: Pull the active NSX certificate:

```
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt
```

Step 2: Verify the certificate is correct:

```
openssl x509 -in /tmp/nsx-root.crt -noout -text | grep -A2 "Subject Alternative
Name"
# Should show: DNS:nsx-vip.lab.local, DNS:nsx-node1.lab.local, DNS:nsx-
manager.lab.local, IP Address:192.168.1.70, IP Address:192.168.1.71
```

Step 3: Import into the VCF trust store:

The VCF trust store password is stored in a `.key` file alongside the store:

```
# Read the trust store password
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)

# Import the NSX certificate
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
```

```
-keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
-storepass "$KEY" -noprompt
```

Step 4: Import into the Java cacerts keystore:

```
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
-keystore /etc/alternatives/jre/lib/security/cacerts \
-storepass changeit -noprompt
```

Step 5: Restart SDDC Manager services:

```
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

Services take approximately 5 minutes to restart. After restart, re-run VDT to confirm NSX cert trust checks pass.

Trust store paths and passwords reference:

Item	Path / Value
VCF trust store	/etc/vmware/vcf/commonsvcs/trusted_certificates.store
VCF trust store password	Contents of /etc/vmware/vcf/commonsvcs/trusted_certificates.key
Java cacerts	/etc/alternatives/jre/lib/security/cacerts
Java cacerts password	changeit
Service restart script	/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh

Reference: [KB 316056 - Trusting Custom Certificates in SDDC Manager](#)

6.3 ESXi Certificate Regeneration

6.3.1 When ESXi Certificate Regeneration Is Needed

ESXi hosts auto-generate self-signed SSL certificates at first boot. Regeneration is required when:

- The ESXi hostname was changed after initial certificate generation
- The certificate has expired
- The certificate SAN does not match the FQDN (e.g., shows `localhost.localdomain`)
- VCF validation reports certificate hostname mismatch

Symptom in VCF Installer/SDDC Manager logs:

```
javax.net.ssl.SSLPeerUnverifiedException: Certificate for <esxi01.lab.local> doesn't
match any of the subject alternative names: [localhost.localdomain]
```

6.3.2 Diagnosis

SSH to the ESXi host and check:

```
# Check current hostname
esxcli system hostname get

# View current certificate SAN
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"

# View full certificate details (subject, issuer, validity)
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout
```

6.3.3 Regeneration Procedure

Run on each ESXi host that needs certificate regeneration:

esxi01.lab.local (192.168.1.74):

```
# Step 1: Ensure hostname is correct
esxcli system hostname set --fqdn=esxi01.lab.local

# Step 2: Verify hostname
esxcli system hostname get
```

```
# Step 3: Backup existing certificates
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak

# Step 4: Generate new certificates
/sbin/generate-certificates

# Step 5: Restart all services to apply new certificate
services.sh restart

# Step 6: Verify new certificate has correct SAN
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"
```

esxi02.lab.local (192.168.1.75):

```
esxcli system hostname set --fqdn=esxi02.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

esxi03.lab.local (192.168.1.76):

```
esxcli system hostname set --fqdn=esxi03.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

esxi04.lab.local (192.168.1.82):

```
esxcli system hostname set --fqdn=esxi04.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

6.3.4 Update Thumbprints After Regeneration

After regenerating ESXi certificates, you must update the thumbprints in VCF. Get the new thumbprints from the VCF Installer or SDDC Manager:

```
# Get SHA-256 thumbprint for each host
echo | openssl s_client -connect 192.168.1.74:443 2>/dev/null | openssl x509 -noout
-fingerprint -sha256
echo | openssl s_client -connect 192.168.1.75:443 2>/dev/null | openssl x509 -noout
-fingerprint -sha256
echo | openssl s_client -connect 192.168.1.76:443 2>/dev/null | openssl x509 -noout
-fingerprint -sha256
echo | openssl s_client -connect 192.168.1.82:443 2>/dev/null | openssl x509 -noout
-fingerprint -sha256
```

Then re-validate the hosts in the VCF Installer UI to update the stored thumbprints.

6.4 Certificate Authority Configuration

6.4.1 Microsoft CA Setup

VCF Operations supports configuring a Microsoft Certificate Authority for automated certificate issuance and renewal.

Navigation: VCF Operations > **Fleet Management** > **Certificates** > **Configure CA**

Configuration Steps:

1. Navigate to **Fleet Management** > **Certificates**
2. Select **VCF Management** or **VCF Instances** (and choose a specific instance)
3. Click **Configure CA**
4. Select **Microsoft Certificate Authority**
5. Fill in:
 - **CA Server URL:** Must begin with `https://` and end with `certsrv` (e.g., `https://ca.lab.local/certsrv`)
 - **User Name:** Least-privileged service account (e.g., `svc-vcf-ca`)
 - **Password:** Service account password
 - **Template Name:** The issuing certificate template created in Microsoft CA

6. Click **Save**

Important: VCF management components (VCF Operations, Fleet Management, VCF Automation) only support Microsoft CA. VCF Instance components (vCenter, NSX, ESXi, SDDC Manager) support both Microsoft CA and OpenSSL.

Microsoft CA Template Requirements:

The certificate template used for VCF must support:

- Key Usage: Digital Signature, Key Encipherment
- Extended Key Usage: Server Authentication (1.3.6.1.5.5.7.3.1)
- Subject Alternative Name: DNS Name and IP Address
- Key Size: 2048-bit RSA minimum
- Validity: Configurable (recommended: 2 years for production, 825 days for lab)

6.4.2 OpenSSL CA Configuration

For environments without Microsoft CA infrastructure, VCF supports OpenSSL as an alternative CA for VCF Instance components.

Configuration Steps:

1. Click **Configure CA**
2. Select **OpenSSL**
3. Fill in:
 - **Common Name:** FQDN of SDDC Manager appliance (e.g., `sddc-manager.lab.local`)
 - **Country:** Country code (e.g., `US`)
 - **Locality Name:** City name
 - **Organization Name:** Organization name (e.g., `lab.local`)
 - **Organization Unit Name:** Department
 - **State:** Full state/province name (unabbreviated)
4. Click **Save**

6.4.3 Certificate Templates

When using Microsoft CA, create a dedicated certificate template:

1. Open the Certificate Authority MMC snap-in on the CA server
2. Right-click **Certificate Templates** > **Manage**

3. Find the **Web Server** template, right-click > **Duplicate Template**
4. On the **General** tab:
 - Template display name: **VCF Web Server**
 - Validity period: 2 years
5. On the **Request Handling** tab:
 - Allow private key to be exported: Yes
6. On the **Extensions** tab:
 - Application Policies: Server Authentication
7. On the **Subject Name** tab:
 - Supply in the request (allow requestor to specify SAN)
8. On the **Security** tab:
 - Grant the VCF service account Read + Enroll permissions
9. Click **OK** to save
10. Back in the CA snap-in, right-click **Certificate Templates** > **New** > **Certificate Template to Issue** > select **VCF Web Server**

Lab Note: In this lab environment, no Microsoft CA is deployed. All certificates are self-signed. The certificate management UI in VCF Operations shows certificate expiration warnings, which is expected and acceptable for lab use.

6.5 Password Management & Rotation

6.5.1 SDDC Manager Password Management

VCF 9.0 centralizes password management in VCF Operations, replacing the password management previously in SDDC Manager.

Navigation: VCF Operations > **Fleet Management** > **Passwords**

The password dashboard shows:

- Account names and types (root, admin, backup, consoleuser, support, admin@local, vmware-system-user)
- Password status (valid, expiring soon, expired)
- Last modified dates
- Expiration dates

Managed VCF Management Components:

Component	Managed Accounts
Fleet Management	root, admin
VCF Automation	root, admin
VCF Identity Broker	root, admin
VCF Operations	root, admin
VCF Operations for Logs	root, admin
VCF Operations for Networks	root, admin

Managed VCF Instance/Domain Components:

Component	Managed Accounts
ESXi Hosts	root
NSX Manager	root, admin, audit
vCenter Server	root, administrator@vsphere.local
SDDC Manager	root, vcf, admin@local

6.5.2 Password Rotation Procedures

Manual password update (specify exact password):

1. Navigate to **Fleet Management > Passwords**
2. Select **VCF Management** or **VCF Instances** tab
3. Select the component and account
4. Click **Update Password**
5. Enter the new desired password
6. Confirm the new password
7. Click **Update**

This changes the password on both the server side (where the account resides) and the client side (where credentials are stored in SDDC Manager).

Automated password rotation (system-generated random password):

1. Navigate to **Fleet Management > Passwords**
2. Select accounts to rotate
3. Click **Rotate**
4. The system generates random passwords meeting complexity requirements

5. Set the rotation interval: **30 days**, **60 days**, or **90 days**
6. You can also **deactivate** the schedule
7. Only a user with the **ADMIN** role can perform this task

Note: Auto-rotate is automatically enabled for vCenter Server. It may take up to 24 hours to configure the auto-rotate policy for a newly deployed vCenter.

WARNING — Credential Rotation Cascade Failure: If a rotation or update fails mid-operation (e.g., NSX unreachable during boot storm), the resource gets stuck in **ACTIVATING** or **ERROR** state in `platform.nsxt`, stale locks fill `platform.Lock`, and unresolved tasks pile up in `platform.task_metadata` (`resolved=false`). Each UI retry adds more stuck tasks. The API cannot cancel these tasks (`TA_TASK_CAN_NOT_BE_RETRIED`). Fix requires direct PostgreSQL repair: fix nsxt status, clear locks, mark task_metadata resolved, clear task_lock, then restart operationsmanager. See [Section 7.2.6](#) for the full 6-step database repair procedure.

Password remediation (when out of sync):

If a password gets out of sync between SDDC Manager's stored credential and the actual component password:

1. Navigate to **Fleet Management > Passwords**
2. Select the component showing a password issue
3. Click **Remediate Password**
4. Enter the password that is currently set on the component
5. Confirm and click **Remediate Password**

Prerequisites:

- No workflows running or scheduled
- Required permissions: Fleet Management > Passwords > Manage and Fleet Management > Passwords > View
- One account remediated at a time

6.5.3 Default Passwords Reference

Component	Account	Default Password	Notes
ESXi Hosts	root	Set during install	Same across all hosts in lab

vCenter Server	administrator@vsphere.local	Set during VCF Installer	SSO administrator
vCenter Server	root	Set during VCF Installer	Appliance shell
NSX Manager	admin	Set during OVF deployment	Web UI + CLI
NSX Manager	root	Set during OVF deployment	Appliance shell
NSX Manager	audit	Set during OVF deployment	Read-only CLI
SDDC Manager	vcf	Set during deployment	SSH login user
SDDC Manager	root	Set during deployment	Via <code>su -</code> from vcf
SDDC Manager	admin@local	Set during deployment	Web UI
VCF Operations	admin	Set during deployment	Web UI
VCF Operations	root	Set during OVF deployment	Appliance shell
Lab password pattern	all	Success01!0909!!	Used across this lab

6.5.4 Password Policy Configuration

VCF enforces password complexity requirements:

- Minimum length: 8 characters (15+ for some components like Fleet Management root)
- Must include: uppercase, lowercase, number, special character
- Special characters supported: `!@#$$%^&*`
- Password history: Cannot reuse recent passwords
- Expiration: Configurable per component (default varies)

6.6 Compliance Monitoring

6.6.1 Available Frameworks

VCF Operations provides built-in and downloadable compliance frameworks:

Built-in (available immediately):

Framework	Coverage
vSphere Security Configuration Guide	ESXi hosts, VMs, vCenter
vSAN Security Configuration Guide	vSAN clusters and configurations
NSX Security Configuration Guide	NSX Manager, transport nodes
DISA Security Standards	Defense Information Systems Agency STIG
FISMA Security Standards	Federal Information Security Management Act
HIPAA	Health Insurance Portability and Accountability Act

Downloadable (requires .PAK file from VMware Marketplace):

Framework	Coverage
PCI DSS Compliance Standards	Payment Card Industry Data Security Standard
CIS Security Standards	Center for Internet Security benchmarks
NIST SP 800-171	Protecting Controlled Unclassified Information
NIST SP 800-53 R5	Security and Privacy Controls

6.6.2 Enabling Compliance in VCF Operations

Navigation: VCF Operations > **Security & Compliance** > **Compliance**

Activate VMware SDDC Benchmarks:

1. Navigate to **Security & Compliance** > **Compliance**
2. Locate the **VMware SDDC Benchmarks** section
3. Click **Activate** for the benchmark you want to enable
4. Select an applicable **policy** when prompted
5. The system activates relevant alert definitions automatically

Install Marketplace Compliance Packs (for air-gapped environments):

1. Download the .PAK file from the VMware Marketplace on an internet-connected machine
2. Transfer the file to a machine that can access VCF Operations
3. Navigate to VCF Operations > **Administration** > **Repository**
4. Click **Add Solution**
5. Upload the .PAK file
6. Accept the EULA and install

7. Click **Add Account** to configure the newly installed integration
8. Return to **Security & Compliance > Compliance** and activate the new benchmark

6.6.3 Compliance Dashboard

After activation, the Compliance dashboard shows:

- **Overall compliance score** per benchmark/framework
- **Per-object compliance** (ESXi hosts, VMs, clusters, NSX components)
- **Failing rules** with severity and remediation guidance
- **Trend analysis** showing compliance improvement over time

Security Operations Dashboard:

Navigation: VCF Operations > **Infrastructure Operations** > **Dashboards & Reports** > **Security Operations**

This dashboard provides:

- User authentication and permissions overview
- Host and VM encryption status
- vSAN cluster encryption status
- CVE violation advisories
- Certificate health monitoring

6.6.4 Remediation Workflows

When compliance checks identify violations:

1. Navigate to the failing rule in the Compliance dashboard
2. Click on the rule to view details and affected objects
3. Review the **Remediation Steps** provided by the benchmark
4. Apply the remediation:
 - **Manual:** Follow the documented steps (ESXi shell commands, vCenter configuration changes)
 - **Automated:** Some rules support automated remediation through VCF Operations actions
5. After remediation, the compliance score updates on the next collection cycle (every 5 minutes for standard metrics, every 4 hours for property-based diagnostic scans)

6.7 Java Keystore Reference

6.7.1 Trust Store Paths and Passwords

Keystore	Path	Password
VCF trust store	<code>/etc/vmware/vcf/commonsvcs/trusted_certificates.store</code>	Contents of <code>/etc/vmware/vcf/commonsvcs</code>
Java cacerts	<code>/etc/alternatives/jre/lib/security/cacerts</code>	<code>changeit</code>
VCF Installer Java	<code>\$JAVA_HOME/lib/security/cacerts</code>	<code>changeit</code>

Note: When replacing any component certificate with a new self-signed cert, the new cert must be imported into BOTH the VCF trust store AND the Java cacerts keystore. Missing either one causes VDT trust check failures.

6.7.2 Common keytool Operations

List all certificates in a keystore:

```
keytool -list -keystore /etc/alternatives/jre/lib/security/cacerts -storepass changeit
```

List certificates with details (verbose):

```
keytool -list -v -keystore /etc/alternatives/jre/lib/security/cacerts -storepass changeit
```

List a specific certificate by alias:

```
keytool -list -alias nsx-selfsigned -keystore
/etc/alternatives/jre/lib/security/cacerts -storepass changeit -v
```

Import a certificate:

```
# Import into Java cacerts
keytool -importcert -alias <alias-name> -file /tmp/cert.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Import into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias <alias-name> -file /tmp/cert.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Delete a certificate:

```
# Delete from Java cacerts
keytool -delete -alias <alias-name> \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit

# Delete from VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -delete -alias <alias-name> \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY"
```

Export a certificate from a keystore:

```
keytool -exportcert -alias <alias-name> \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit \
  -file /tmp/exported-cert.crt -rfc
```

Check if a specific alias exists:

```
keytool -list -alias nsx-selfsigned \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit 2>&1 | head -1
# Returns "nsx-selfsigned, ..." if found, or error if not found
```

Change keystore password:

```
keytool -storepasswd \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit \
  -new <new-password>
```

Download a remote certificate and import in one step:

```
# Pull certificate from a remote server
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/remote-cert.crt

# Verify it is the correct certificate
openssl x509 -in /tmp/remote-cert.crt -noout -subject -issuer -dates

# Import into both keystores
keytool -importcert -alias remote-server -file /tmp/remote-cert.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias remote-server -file /tmp/remote-cert.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Find all Java cacerts files on the system:

```
find / -name "cacerts" -type f 2>/dev/null
```

Restart services after keystore changes:

```
# On SDDC Manager
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh

# On VCF Installer
systemctl restart lcm
systemctl restart domainmanager
```

Tip: Always verify changes with VDT after modifying trust stores. Run VDT from SDDC Manager:

```
cd /home/vcf/vdt-2.2.7_02-05-2026 && python vdt.py
```

VDT report location: `/var/log/vmware/vcf/vdt/vdt-<timestamp>.txt`

PART VII: Troubleshooting & Recovery

7.1 VDT (VMware Deployment Toolkit)

The VCF Diagnostic Tool (VDT) is a standalone Python utility that validates the health and configuration of your VCF environment. It is **NOT pre-installed** on SDDC Manager and must be downloaded separately from Broadcom.

7.1.1 Download from Broadcom

VDT is distributed via Broadcom Knowledge Base article **344917**. Navigate to:

```
https://knowledge.broadcom.com/external/article/344917
```

Download the latest version. In this lab, the version used is `vdt-2.2.7_02-05-2026.zip`.

7.1.2 Upload to SDDC Manager

Warning: SCP does not work with SDDC Manager due to the restricted shell on the `vcf` user. Only the `vcf` user can SSH in (root and admin are rejected). Use the `ssh cat redirect` method for file transfer.

Method 1: SSH cat redirect (recommended)

```
# From your Windows workstation (PowerShell)
ssh vcf@192.168.1.241 "cat > /home/vcf/vdt-2.2.7_02-05-2026.zip" < C:\VCF-Depot\vdt-2.2.7_02-05-2026.zip
```

Method 2: SCP (if it works in your environment)

```
scp C:\VCF-Depot\vdt-2.2.7_02-05-2026.zip vcf@192.168.1.241:/home/vcf/
```

7.1.3 Installation

SSH to SDDC Manager as `vcf` , then extract:

```
ssh vcf@192.168.1.241
cd /home/vcf
unzip vdt-2.2.7_02-05-2026.zip
ls -la vdt-2.2.7_02-05-2026/
```

No additional installation is required. VDT is a Python script that runs directly.

7.1.4 Running VDT

```
cd /home/vcf/vdt-2.2.7_02-05-2026
python vdt.py
```

VDT will prompt for `administrator@vsphere.local` password. It then performs a comprehensive validation of the entire VCF stack.

7.1.5 Interpreting Results

VDT produces a text report and JSON output at:

```
/var/log/vmware/vcf/vdt/vdt-<timestamp>.txt
/var/log/vmware/vcf/vdt/vdt-<timestamp>.json
```

Lab VDT Results Summary (vcf-lab, Feb 12 2026):

Category	Status	Details
SDDC Manager Info	PASS	Version 9.0.1.0.24962180, hostname sddc-manager.lab.local
NTP Service & Server	PASS	192.168.1.230 responding
/etc/hosts	PASS	Properly formatted
SDDC Manager Services	PASS	COMMON_SERVICES, LCM, DOMAIN_MANAGER, OPERATIONS_MANAGER, SDDC_MANAGER_UI -- all ACTIVE

Commonservices API	PASS	HTTP 200 on localhost
Disk Utilization	PASS	Filesystem healthy (space and inodes)
Host/Domain/Cluster Status	PASS	All ACTIVE
vCenter/PSC/NSX Status	PASS	All ACTIVE
SDDC Cert Trust/Expiry/SAN	PASS	717 days remaining
vCenter Cert Trust/Expiry	PASS	725 days remaining
vCenter Cert SAN	WARN	Hostname but not IP in SAN (cosmetic, acceptable for lab)
NSX VIP Cert Trust/Expiry/SAN	PASS	Fixed after cert replacement and trust store import
NSX Manager Cert Trust/Expiry/SAN	PASS	Fixed after cert replacement and trust store import
Deployment/Resource Locks	PASS	No locks detected
Changelog Locks	PASS	All 4 DBs (domainmanager, operationsmanager, lcm, platform)
Service Account Auth	PASS	No authentication issues
NFS Mount Ownership	PASS	Fixed: <code>chown root:vcf /nfs/vmware/vcf/nfs-mount/</code>
Depot Config	PASS	Checks skipped for 9.x+

Note: VDT showed "not found" for Aria Lifecycle, Automation, Operations, Logs, and Workspace One. This is expected when these products were deployed manually outside SDDC Manager's Aria inventory.

7.1.6 Common VDT Failures and Fixes

NFS Mount Ownership: FAIL

```
# Before: owner was nginx instead of root
ls -la /nfs/vmware/vcf/
# drwxrwxr-x nginx vcf nfs-mount/

# Fix:
chown root:vcf /nfs/vmware/vcf/nfs-mount/
```

```
# After: owner is root, group is vcf
# Reference: https://knowledge.broadcom.com/external/article/392923
```

NSX Certificate SAN: FAIL

The default NSX self-signed certificate uses a wildcard SAN (`*.lab.local`) without specific hostnames or IPs. VDT reports "SAN contains neither hostname nor IP." See Section 7.5 for the complete NSX certificate replacement procedure.

NSX Certificate Trust: FAIL

After replacing the NSX self-signed certificate, the new root is not in SDDC Manager's keystores. See Section 7.5 for the trust store import procedure.

Service Properties Ownership: FAIL

```
# Check ownership of service property files
ls -la /opt/vmware/vcf/domainmanager/conf/
ls -la /opt/vmware/vcf/operationsmanager/conf/

# Fix: ensure correct ownership
chown vcf:vcf /opt/vmware/vcf/domainmanager/conf/application-prod.properties
chown vcf:vcf /opt/vmware/vcf/operationsmanager/conf/application-prod.properties
```

7.2 SDDC Manager Troubleshooting

7.2.1 Service Management

SDDC Manager runs multiple services managed via systemd. Here are the key services and their management commands:

Service	Purpose	Command
domainmanager	Domain lifecycle operations	<code>systemctl status domainmanager</code>
lcm	Lifecycle management	<code>systemctl status lcm</code>
operationsmanager	Operations and monitoring	<code>systemctl status operationsmanager</code>

commonsvcs	Shared platform services	<code>systemctl status commonsvcs</code>
postgresql	Internal database	<code>systemctl status postgresql</code>
nginx	Web server / reverse proxy	<code>systemctl status nginx</code>
vcf-services	All VCF services (target)	<code>systemctl status vcf-services</code>

Check all service statuses:

```
systemctl status domainmanager
systemctl status lcm
systemctl status operationsmanager
systemctl status commonsvcs
systemctl status postgresql
systemctl status nginx
```

Restart all VCF services:

```
systemctl restart vcf-services
# Wait 3-5 minutes for all services to start
systemctl status vcf-services
```

Restart individual service:

```
systemctl restart domainmanager
systemctl restart lcm
systemctl restart operationsmanager
```

Full service restart script (recommended for major changes):

```
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
# Takes approximately 5 minutes
```

7.2.2 SDDC Manager Log Locations

```

/var/log/vmware/vcf/
├── domainmanager/
│   ├── domainmanager.log          # Main domain manager log
│   └── domainmanager-gc.log       # Garbage collection log
├── lcm/
│   ├── lcm.log                   # Lifecycle management log
│   ├── lcm-debug.log             # LCM debug (TLS errors show here)
│   └── upgrade/                  # Upgrade-specific logs
├── operationsmanager/
│   ├── operationsmanager.log      # Operations manager log
│   └── operationsmanager-gc.log   # Garbage collection log
├── sos/
│   └── sos.log                   # SoS utility log
├── commonsvcs/
│   └── commonsvcs.log            # Common services log
├── vdt/
│   └── vdt-<timestamp>.txt        # VDT report files
└── sddc-support/
    └── sddc-support.log          # Support bundle log

```

Log analysis commands:

```

# View last 100 lines of domain manager log
tail -100 /var/log/vmware/vcf/domainmanager/domainmanager.log

# Follow log in real-time
tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log

# Search for errors across all VCF logs
grep -ri "error\|exception\|failed"
/var/log/vmware/vcf/domainmanager/domainmanager.log | tail -50

# Search for specific time period
grep "2026-02-12 14:" /var/log/vmware/vcf/domainmanager/domainmanager.log

# Count error occurrences
grep -c "error" /var/log/vmware/vcf/domainmanager/domainmanager.log

# Search for LCM TLS errors
grep -i "tlsfatal\|ssl\|certificate" /var/log/vmware/vcf/lcm/lcm-debug.log | tail
-20

```

7.2.3 Timeout Loop Issues in Nested Environments

Problem: SDDC Manager deployment via VCF Installer enters a timeout loop in nested VMware Workstation environments. The installer waits for SDDC Manager to become responsive, but the appliance takes too long to boot and initialize services on resource-constrained nested hosts.

Symptoms:

- VCF Installer shows deployment progress stuck
- SDDC Manager VM is deployed but services never become ready within the timeout window
- Repeated timeout errors in installer logs

Solution: Bypass the VCF Installer for SDDC Manager deployment. Deploy SDDC Manager manually using `ovftool` with a single-line command (backslash continuation breaks `--noSSLVerify`).

```
# Single-line ovftool command (do NOT use backslash line continuation)
ovftool --acceptAllEulas --noSSLVerify --allowExtraConfig --diskMode=thin --powerOn
--name=sddc-manager --ipProtocol=IPv4 --ipAllocationPolicy=fixedPolicy --
prop:BACKUP_PASSWORD=Success01!0909!! --prop:ROOT_PASSWORD=Success01!0909!! --
prop:VCF_PASSWORD=Success01!0909!! --prop:BASIC_AUTH_PASSWORD=Success01!0909!! --
prop:vami.hostname=sddc-manager.lab.local --prop:vami.ip0.SDDC-Manager-
Appliance=192.168.1.241 --prop:vami.netmask0.SDDC-Manager-Appliance=255.255.255.0 -
--prop:vami.gateway.SDDC-Manager-Appliance=192.168.1.1 --prop:vami.DNS.SDDC-Manager-
Appliance=192.168.1.230 --prop:vami.domain.SDDC-Manager-Appliance=lab.local --
prop:vami.searchpath.SDDC-Manager-Appliance=lab.local --prop:vami.NTP.SDDC-Manager-
Appliance=192.168.1.230 --datastore=esxi01-local --network="VM Network"
vi://root:Success01!0909!!@192.168.1.74 /path/to/sddc-manager.ova
```

Key lesson: `ovftool` on the VCF Installer must use single-line commands. Backslash continuation breaks `--noSSLVerify` and other flags.

7.2.4 NFS Mount Issues

VDT may report NFS mount ownership failures when the mount point owner is incorrect.

```
# Check NFS mount ownership
ls -la /nfs/vmware/vcf/

# Expected: root:vcf ownership on nfs-mount/
# If showing nginx:vcf, fix with:
```

```
chown root:vcf /nfs/vmware/vcf/nfs-mount/

# Verify NFS subdirectories exist
ls -la /nfs/vmware/vcf/nfs-mount/
# Should contain: bundle/, depot/, depot/local/
```

7.2.5 SSH Quirks

Critical: Only the `vcf` user can SSH to SDDC Manager. The `root` and `admin` users are rejected at the SSH level.

```
# SSH to SDDC Manager
ssh vcf@192.168.1.241

# Get root access from vcf session
su -

# File transfer workaround (SCP does not work due to restricted shell)
ssh vcf@192.168.1.241 "cat > /home/vcf/myfile.zip" < localfile.zip

# Transfer file FROM SDDC Manager
ssh vcf@192.168.1.241 "cat /path/to/file" > local_copy
```

Account lockout (faillock):

SDDC Manager uses `faillock` (not `pam_tally2`) to lock accounts after failed SSH attempts. Automated scripts with wrong passwords can quickly lock the `vcf` account.

```
# From SDDC Manager console as root:

# Check lockout status
faillock --user vcf

# Unlock the vcf account
faillock --user vcf --reset
```

```
# Unlock root (if also locked)
faillock --user root --reset
```

If locked out of ALL accounts (root, vcf, admin): Boot into single-user mode via GRUB — reboot the VM, press `e` at the GRUB menu, append `init=/bin/bash` to the `Linux` line, press Ctrl+X. Then: `mount -o remount,rw /` → `faillock --user root --reset` → `faillock --user vcf --reset` → `reboot -f`

7.2.6 Database Access & Credential Cascade Repair

PostgreSQL overview:

SDDC Manager uses PostgreSQL 15 with data directory `/data/pgdata`. It listens on **TCP 127.0.0.1 only** (not Unix sockets — you'll get "No such file or directory" without `-h 127.0.0.1`). Authentication uses `scram-sha-256`.

psql pager trap: When running `psql` queries via Paramiko or remote shell, the default pager (`less` / `more`) captures output and waits for interactive input, corrupting the session. **Always** set `PAGER=cat` before running `psql` commands, or pass it inline: `PAGER=cat psql -h 127.0.0.1 ...`. For Paramiko `invoke_shell()`, also set `height=1000` to prevent terminal-based paging.

```
# Check PostgreSQL status
systemctl status postgresql

# List all databases
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -l"

# Check database disk usage
df -h /
```

Key databases and tables:

Database	Key Tables	Key Columns	Purpose
platform	nsxt	id, status	NSX cluster resource status (ACTIVE/ACTIVATING/ERROR)

platform	lock	resource/lock columns	Exclusive operation locks
platform	task_metadata	resolved (boolean)	Task resolution tracking
platform	task_lock	task-to-lock associations	Task-lock relationships
operationsmanager	task	state (NOT status)	Operation tasks
operationsmanager	execution	execution_status (NOT status)	Execution tracking
operationsmanager	processing_task	status	Active processing queue
operationsmanager	execution_to_task	mapping columns	Execution-task relationships
domainmanager	domain-related tables	—	Domain lifecycle state

Key discovery: The API cannot cancel stuck tasks — `PATCH` returns `TA_TASK_CAN_NOT_BE_RETRIED` and `DELETE` returns HTTP 500. Database repair is the only option for cascade failures.

Accessing PostgreSQL (trust auth workaround):

The PostgreSQL password is not easily discoverable in configuration files. The workaround is to temporarily set trust authentication:

```
# SSH as vcf, then su - to root

# Back up pg_hba.conf (CRITICAL)
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak

# Temporarily allow passwordless local connections
sed -i 's/scram-sha-256/trust/g' /data/pgdata/pg_hba.conf

# Reload postgres (no restart needed)
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Disable psql pager (CRITICAL for scripted/remote sessions)
export PAGER=cat
export PGPAGER=cat
```

```
# Now you can connect without a password
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"
```

CRITICAL: Always restore `pg_hba.conf` immediately after making changes:

```
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"
```

Credential Cascade Failure — Full Diagnosis & 6-Step Repair

Symptoms:

- Password Management → Update Password, Rotate, or Remediate fails for NSX (or other component)
- Task history shows: "Resources [nsx-vip.lab.local] are not available/ready" or "not in ACTIVE state"
- Subsequent attempts fail with: "Unable to acquire resource level lock(s)"
- VCF Operations Fleet Management shows "[2] account(s) has been disconnected"
- SDDC Manager API `/v1/nsxt-clusters` shows empty or non-ACTIVE status
- Dozens of stuck IN_PROGRESS tasks accumulate (visible via `/v1/tasks?status=IN_PROGRESS`)

Root Cause Chain: A failed credential operation (often due to NSX being temporarily unreachable during a boot storm) triggers a cascade:

1. NSX cluster resource gets stuck in `ACTIVATING` or `ERROR` state in `platform.nsxt` table
2. Stale exclusive locks remain in `platform.lock` table, blocking all new operations
3. Failed tasks remain as `IN_PROGRESS` in `platform.task_metadata` (resolved=false), piling up
4. Each retry from the UI creates more stuck tasks and locks
5. Even after NSX recovers, SDDC Manager won't attempt the operation because the status check fails prevalidation

Diagnosis:

```
# 1. Get auth token from SDDC Manager
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
-H "Content-Type: application/json" \
-d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# 2. Check NSX cluster resource state (look for status field)
curl -sk "https://sddc-manager.lab.local/v1/nsxt-clusters" \
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
# If status is "ACTIVATING" or "ERROR" instead of "ACTIVE" → this is the problem

# 3. Check for stale resource locks
curl -sk "https://sddc-manager.lab.local/v1/resource-locks" \
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool

# 4. Check for stuck IN_PROGRESS tasks
curl -sk "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
-H "Authorization: Bearer $TOKEN" | python3 -c \
"import sys,json; d=json.load(sys.stdin); print(f'Stuck tasks:
{len(d.get(\"elements\",[]))}')"

# 5. Verify NSX is actually healthy (from SDDC Manager)
curl -sk -u admin:'Success01!0909!!' --connect-timeout 10 \
https://nsx-vip.lab.local/api/v1/cluster/status
# overall_status should be "STABLE"
```

Fix — Full 6-Step Database Repair:

WARNING: Direct database manipulation is unsupported and should only be done in lab environments. Always back up before modifying.

Step 1: Access PostgreSQL on SDDC Manager

SSH as `vcf`, then `SU -` to root. Enable trust auth (see above), then set pager:

```
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak
sed -i 's/scram-sha-256/trust/g' /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"
export PAGER=cat
```

Step 2: Fix the stuck resource status

The `nsxt` table status can be `ACTIVATING`, `ERROR`, or other non-ACTIVE values:

```
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT id, status
FROM nsxt;\""

# Fix ANY non-ACTIVE status
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt SET
status = 'ACTIVE' WHERE status != 'ACTIVE';\""

```

Step 3: Clear stale resource locks

```
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT count(*) FROM
lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM lock;\""

```

Step 4: Mark stuck tasks as resolved

The `task_metadata` table in the platform DB tracks task resolution state. Unresolved tasks (resolved=false) from failed operations accumulate and can interfere with new operations:

```
# Check unresolved task count
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT resolved,
count(*) FROM task_metadata GROUP BY resolved;\""

# Mark all unresolved tasks as resolved
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE task_metadata
SET resolved = true WHERE resolved = false;\""

# Clear task_lock table if any entries exist
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
task_lock;\""

```

Step 5: Restore pg_hba.conf (CRITICAL — do not skip)

```
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Verify it's back to scram-sha-256
grep -c 'scram-sha-256' /data/pgdata/pg_hba.conf
# Should return 4 or more
```

Step 6: Restart operationsmanager service

```
systemctl restart operationsmanager
# Wait 2-3 minutes for it to fully start
systemctl is-active operationsmanager
```

Verification:

```
# NSX cluster should now show ACTIVE
curl -sk "https://sddc-manager.lab.local/v1/nsxt-clusters" \
  -H "Authorization: Bearer $TOKEN" | python3 -c \
  "import sys,json; [print(f'{c[\"id\"]}: {c[\"status\"]}') for c in
json.load(sys.stdin).get('elements',[])]"

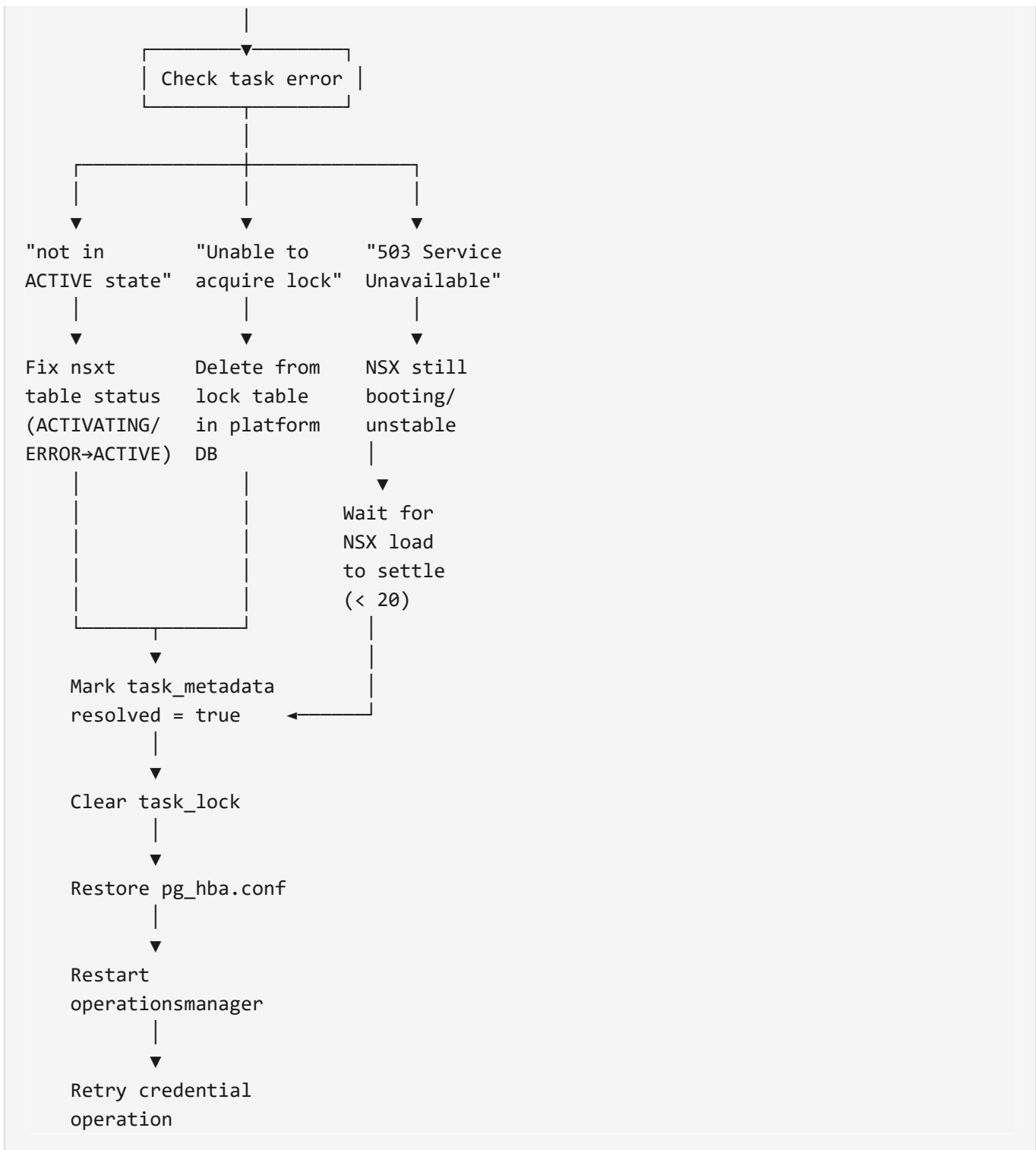
# Resource locks should be empty
curl -sk "https://sddc-manager.lab.local/v1/resource-locks" \
  -H "Authorization: Bearer $TOKEN"

# IN_PROGRESS tasks should be zero or minimal
curl -sk "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
  -H "Authorization: Bearer $TOKEN" | python3 -c \
  "import sys,json; print(f'IN_PROGRESS:
{len(json.load(sys.stdin).get(\"elements\",[]))}')"

# Credential remediate should now succeed via VCF Operations Fleet Management UI
```

Credential Cascade Failure Flowchart:

```
Credential Update/Rotate/Remediate fails
in SDDC Manager or VCF Operations UI
```



Key insight: Three tables in the `platform` database must be cleaned: (1) `nsxt` — resource status, (2) `Lock` — operation locks, (3) `task_metadata` — task resolution tracking (+ `task_lock`). The `operationsmanager` database has separate `task` and `execution` tables (columns: `task.state`,

`execution.execution_status` — NOT `status`). The API won't let you cancel or delete stuck tasks — database repair is required.

General database troubleshooting:

```
# If database connection fails:
# 1. Check PostgreSQL logs
tail -100 /var/log/postgresql/postgresql-*.log

# 2. Restart PostgreSQL
systemctl restart postgresql

# 3. Wait 2 minutes, then restart VCF services
sleep 120
systemctl restart vcf-services
```

Quick SQL reference (for experienced users):

```
-- Connect: su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

-- Fix NSX status (covers ACTIVATING and ERROR)
UPDATE nsxt SET status = 'ACTIVE' WHERE status != 'ACTIVE';

-- Clear stale locks
DELETE FROM lock;

-- Resolve stuck tasks
UPDATE task_metadata SET resolved = true WHERE resolved = false;
DELETE FROM task_lock;
```

Why each repair step is needed:

Step	Table	Action	Why
2	nsxt	Set status to ACTIVE	Stuck ACTIVATING/ERROR makes every new operation fail at prevalidation
3	lock	Delete all rows	Stale exclusive locks block all new operations ("Unable to acquire resource level lock(s)")

4	<code>task_metadata</code>	Set resolved=true	Unresolved tasks accumulate with each UI retry (47 found during initial diagnosis)
4	<code>task_lock</code>	Delete all rows	Orphaned task-lock relationships must be cleared
5	<code>pg_hba.conf</code>	Restore backup	Trust auth is a security risk — restore immediately
6	<code>operationsmanager</code>	Restart service	Service caches DB state in memory — restart forces re-read of cleaned tables

Steps 2-4 must all be done in one session — fixing just the status without clearing locks still fails, and vice versa. All three tables participate in the prevalidation check. The trust auth window should be as short as possible.

Schema discovery notes: None of this is documented by Broadcom. The schema was mapped by exploring databases with `\l`, listing tables with `\dt`, and querying `information_schema.columns`. Key discoveries: `task_metadata` uses `resolved` boolean (not a status field), `operationsmanager.task` uses column `state` (not `status`), and `execution` uses `execution_status` (not `status`). Early script versions failed because of these naming differences. The API's `PATCH /v1/tasks/{id}` returns `TA_TASK_CAN_NOT_BE_RETRIED` and `DELETE` returns HTTP 500 — database repair is the only option.

7.2.7 API Troubleshooting

```
# Get authentication token
curl -k -X POST https://localhost/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"admin@local","password":"Success01!0909!!"}'

# Check task status via API
curl -k -H "Authorization: Bearer <access-token>" \
  https://localhost/v1/tasks/<task-id>

# Cancel a stuck task via API
curl -k -X PATCH https://localhost/v1/tasks/<task-id> \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -d '{"status":"CANCELLED"}'

# Check VCF health via API
```

```
curl -k -H "Authorization: Bearer <access-token>" \
https://localhost/v1/system/health
```

7.2.8 SoS Diagnostic Bundle

SDDC Manager includes the SoS (Supportability and Serviceability) utility for comprehensive log collection:

```
# SSH to SDDC Manager as vcf, then su - to root
ssh vcf@192.168.1.241
su -

# Navigate to SoS directory
cd /opt/vmware/sddc-support/

# Generate log bundle for the management domain
./sos --domain-name mgmt --log-bundle

# Generate with health check included
./sos --domain-name mgmt --log-bundle --health-check

# Include free (unassigned) hosts
./sos --domain-name mgmt --log-bundle --include-free-hosts

# Bundle output location:
# /var/log/vmware/vcf/sddc-support/sos-<timestamp>.tar.gz

# Transfer logs to Broadcom support (VCF 9)
./sos --log-assist --sr-number <support-request-number>
```

7.3 vCenter Troubleshooting

7.3.1 Stuck Deployments

Symptoms:

- Deployment stuck at a percentage (commonly 60%) for more than 30 minutes
- No progress in firstboot logs
- VCF UI shows no timeout error

Diagnostic commands (SSH to vCenter VM):

```
# Check current deployment status
cat /var/log/firstboot/firstbootStatus.json

# Check for running processes
ps aux | grep -E "install|firstboot|postgres|vpqd"

# Check disk I/O (should show activity)
vmstat 1 5

# Check memory usage
free -h

# Check for error logs
tail -50 /var/log/vmware/firstboot/installer.log
grep -i "error\|fail\|exception" /var/log/vmware/firstboot/*.log
```

Monitoring deployment progress from VCF Installer:

```
# Find the latest ci-installer log directory
ls -lt /var/log/vmware/vcf/domainmanager/ | head -5

# Watch the installation log
tail -f /var/log/vmware/vcf/domainmanager/ci-installer-XX-XX-XX-XX-XX-XXX/ci-
installer.log

# Search for errors
grep -i "error\|failed\|exception" /var/log/vmware/vcf/domainmanager/ci-installer-
XX-XX-XX-XX-XX-XXX/ci-installer.log
```

Expected deployment stages:

1. vCenter VM deployment (OVA extraction)
2. First boot -- basic configuration
3. Installing containers (60% mark)
4. Database initialization
5. Service startup
6. vCenter registration with VCF

7.3.2 PostgreSQL Issues

If deployment is stuck at "Installing Containers" (60%), check PostgreSQL:

```
# Check if postgres service exists
ls -la /storage/db/vpostgres/

# Check for postgres config file
ls -la /storage/db/vpostgres/postgresql.conf

# Check postgres user/group
grep postgres /etc/passwd
grep postgres /etc/group

# Check postgres logs
tail -50 /var/log/vmware/vpostgres/*.log
```

Warning: If PostgreSQL never initialized (missing `postgresql.conf` and missing postgres user), the database initialization failed. This is typically **unrecoverable** and requires full redeployment.

Post-deployment PostgreSQL health check:

```
# Check database service
service-control --status vmware-vpostgres

# Check database connections
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT count(*) FROM
pg_stat_activity;"

# If database is unhealthy:
service-control --restart vmware-vpostgres
# Wait 5 minutes, then restart vpxd:
service-control --restart vpxd
```

7.3.3 Service Management

Check all vCenter services:

```
# List all services with status
service-control --status --all

# Alternative: use vmon-cli
vmon-cli --list

# Check specific service
vmon-cli --status vpxd
service-control --status vpxd
```

Expected healthy services (all should show RUNNING):

Service	Purpose
vpxd	Core vCenter daemon
vsphere-ui	vSphere Client web interface
vmware-vpostgres	Embedded PostgreSQL database
rhttpproxy	Reverse proxy
lookupsvc	Lookup service (SSO)
sts	Security Token Service
vlcm	vSphere Lifecycle Manager
content-library	Content Library
eam	ESX Agent Manager

Restart a specific service:

```
service-control --restart vpxd
# Wait 2-3 minutes for service to start
service-control --status vpxd
```

Restart all services (causes brief outage):

```
service-control --restart --all
# Wait 10-15 minutes for all services to start
```

```
service-control --status --all
```

7.3.4 VPXD Issues

```
# Check vpxd status
service-control --status vpxd

# Review vpxd logs
tail -100 /var/log/vmware/vpxd/vpxd.log

# Search for vpxd errors
grep -i "error\|exception\|failed" /var/log/vmware/vpxd/vpxd.log | tail -50

# Check vSphere Client logs
tail -100 /var/log/vmware/vsphere-ui/logs/vsphere_client_virgo.log

# Restart vpxd
service-control --restart vpxd
```

7.3.5 vCenter Deployment Failure Reference Tokens

When vCenter deployment fails, VCF provides a reference token. To find detailed errors:

```
# Search for reference token in logs (example token: 30HCKD)
grep -r "30HCKD" /var/log/vmware/vcf/
grep -B20 -A20 "30HCKD" /var/log/vmware/vcf/domainmanager/*.log
```

7.3.6 Cleanup After Failed Deployment

See Section 7.7 for the complete failed deployment recovery procedure.

7.4 vMotion Troubleshooting

7.4.1 vhv.enable Ghost Setting

Problem: The `vhv.enable` setting can persist in a VM's runtime DICT (vmware.log) even when it is not present in the VMX file. This causes vMotion to fail with:

```
Migration failed after VM memory precopy. Configuration mismatch:
The virtual machine cannot be restored because the snapshot was taken with VHV enabled.
```

Root cause (lab-tested): The vCenter UI showed "Expose hardware assisted virtualization" unchecked, and the VMX file had no `vhv.enable` entry. However, the VM runtime logs revealed `vhv.enable = "TRUE"` inherited from the original deployment environment.

Diagnostic steps:

```
# SSH to the ESXi host running the VM
ssh root@192.168.1.74

# Search VM logs for vhw references
grep -i vhw /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/*

# Check the VMX file directly
grep -i vhw /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmx
```

Fix: Add an explicit `vhv.enable = "FALSE"` to the VMX file, even if the setting does not currently appear:

```
# Power off the VM first, then:
echo 'vhv.enable = "FALSE"' >> /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-
manager/sddc-manager.vmx

# Power the VM back on
```

Key lesson: The absence of `vhv.enable` in the VMX file does NOT mean it is disabled. The setting can persist in runtime/logs from a previous environment. Always add an **explicit** `vhv.enable = "FALSE"` to fix vMotion failures related to VHV mismatch.

7.4.2 Memory Convergence Failures in Nested Environments

Problem: Hot vMotion fails in nested VMware Workstation environments because memory convergence cannot complete within the timeout window.

Error message:

```
Migration was canceled because the amount of changing memory was greater
than the available network bandwidth
```

Root cause: Nested environments have limited network throughput and higher memory change rates, making it difficult for vMotion to converge the memory state between source and destination hosts.

Workarounds:

1. **Reduce VM memory activity** -- quiesce the workload before migration
2. **Increase vMotion timeout** -- via advanced vCenter settings (not always effective)
3. **Use cold migration** (recommended fallback):

```
# Cold migration procedure:
1. Power off the VM (graceful shutdown)
2. Right-click VM in vCenter -> Migrate
3. Select "Change both compute resource and storage"
4. Select destination host and datastore
5. Complete the migration
6. Power the VM back on
```

In the lab, SDDC Manager was successfully relocated from esxi01 to esxi03 using cold migration after hot vMotion failed.

7.4.3 EVC Compatibility Issues

Problem: DRS cannot migrate VMs between hosts with different CPU generations.

Diagnostic steps:

```
# Check CPU model on each host (from vCenter or ESXi SSH)
esxcli hardware cpu global get

# Check EVC status on cluster
# In vSphere Client: Cluster -> Configure -> VMware EVC
```

EVC mode hierarchy (Intel):

```
Newest  -> Intel "Cascade Lake" Generation
          Intel "Skylake" Generation
          Intel "Broadwell" Generation
          Intel "Haswell" Generation
          Intel "Ivy Bridge" Generation
Oldest  -> Intel "Sandy Bridge" Generation
```

EVC mode must be set to the **lowest** CPU generation in the cluster. All VMs may need to be powered off before changing EVC mode.

7.4.4 Network Troubleshooting for vMotion

```
# Check vMotion VMkernel adapter exists
esxcfg-vmknic -l | grep -i vmotion

# Test vMotion network connectivity between hosts
vmkping -I vmk1 192.168.100.11

# Check vMotion is enabled on the VMkernel adapter
esxcli network ip interface tag get -i vmk1

# Verify MTU settings (1500 for nested, do NOT use 9000)
esxcfg-vmknic -l

# Check vMotion port (TCP 8000) connectivity
nc -z 192.168.100.11 8000
```

Network	VLAN	Subnet	Gateway	MTU
vMotion	100	192.168.100.0/24	192.168.100.1	1500

Warning: Do NOT use jumbo frames (MTU 9000) in nested VMware Workstation environments. Use MTU 1500 for all networks.

7.5 NSX Troubleshooting

7.5.1 OOM in Nested Environments

Problem: NSX Manager deployed with the `small` option (16GB RAM) crashes with kernel OOM (Out of Memory) in nested environments. Console shows repeated `sysrq: Show Memory` messages.

Impact: All NSX-related validation checks in VCF Installer fail, and services cannot stabilize.

Sizing requirements for nested environments:

RAM	vCPU	Result
16GB	4	Kernel OOM, constant crashes
24GB	4	Runs, but MANAGER/SEARCH services crash under load (transport node config)
32GB	6	Required for stable operation with 4-host cluster

Resolution:

```
# Power off NSX Manager VM
# In vCenter: right-click NSX Manager VM -> Power -> Shut Down Guest OS

# Edit VM settings:
# - Memory: 32 GB
# - CPU: 6 vCPU

# Power on NSX Manager VM
# Wait 10-15 minutes for all services to stabilize
```

Key lesson: Many VCF Installer validation errors are cascading failures from an unhealthy NSX. Fix NSX health **first** before troubleshooting other validation failures.

7.5.2 Transport Node Issues

Symptoms:

- Transport node shows "Disconnected" or "Failed" in NSX Manager
- VMs cannot communicate across hosts on overlay segments

Diagnostic commands on ESXi host:

```
# Check NSX proxy agent status
/etc/init.d/nsx-proxy status

# Start NSX proxy if not running
/etc/init.d/nsx-proxy start

# Check NSX datapath status
/etc/init.d/nsx-datapath status

# Check connectivity to NSX Manager (port 1234)
esxcli network ip connection list | grep 1234

# Review NSX agent logs
tail -50 /var/log/nsx-syslog.log

# Find TEP VMkernel adapter
esxcfg-vmknics -l | grep -i tep

# Test TEP-to-TEP connectivity
vmkping <other-host-tep-ip>
```

Transport node recovery steps:

1. Remove the failed transport node profile from the cluster
2. Restart the management network on affected hosts
3. Re-apply the transport node profile
4. Wait for all hosts to show Success/Up

In the lab, transport node configuration initially failed when NSX had only 24GB RAM. After increasing to 32GB/6vCPU:

1. Removed failed profile from cluster
2. Restarted management network on all hosts
3. Re-applied tn-profile-mgmt
4. All 4 hosts configured successfully -- vmk0 used as TEP

Force resync from NSX Manager UI:

1. Navigate to System -> Fabric -> Nodes -> Host Transport Nodes
2. Click on the problematic host

3. Click Actions -> Redeploy Node
4. Wait 5-10 minutes for resync

7.5.3 Certificate Problems

NSX certificate issues are the most common VDT failures. Two types of problems occur:

Problem 1: SAN Missing Hostnames/IPs

The default NSX self-signed certificate uses a wildcard SAN (`*.lab.local`) without specific hostnames or IPs. VDT reports "SAN contains neither hostname nor IP."

Step 1: Create OpenSSL config on NSX Manager (SSH as root):

```
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF
```

Critical: `DNS.3 = nsx-manager.lab.local` is required because SDDC Manager registers NSX using this FQDN. Without it, VDT reports "SAN contains IP but not hostname."

Step 2: Generate certificate and build JSON payload:

```
# Generate cert (single-line, no backslash continuation)
openssl req -x509 -nodes -days 825 -newkey rsa:2048 -keyout /tmp/nsx.key -out
/tmp/nsx.crt -config /tmp/nsx-cert.conf -sha256

# Verify SAN entries
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"

# Build JSON payload using Python (avoids shell PEM escaping issues)
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

Warning: NSX shell does NOT support backslash line continuation. All curl commands must be single-line. Use Python to build JSON payloads containing PEM data.

Step 3: Import and apply certificate via NSX API:

```
# Import cert (single-line)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates?action=import" -H "Content-Type: application/json" -d
@/tmp/nsx-import.json
# Note the certificate ID from response (e.g., 701d1416-5054-4038-8749-
4ac495980ebd)

# Get node UUID
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster
# Note the node UUID (e.g., 95493642-ef4a-cb8e-ed7c-5bc20033f2c2)

# Apply to node (API service)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/701d1416-5054-4038-8749-4ac495980ebd?
action=apply_certificate&service_type=API&node_id=95493642-ef4a-cb8e-ed7c-
```

```
5bc20033f2c2"
```

```
# Apply to VIP (MGMT_CLUSTER)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/701d1416-5054-4038-8749-4ac495980ebd?
action=apply_certificate&service_type=MGMT_CLUSTER"

# Verify on both endpoints
openssl s_client -connect 192.168.1.71:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
openssl s_client -connect 192.168.1.70:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

Prerequisite: All NSX services must be healthy (MANAGER, SEARCH, UI, NODE_MGMT all UP). If services are DOWN, the API returns error 101. Wait 10-15 minutes after NSX restart in nested environments.

Problem 2: Certificate Trust Failure

After replacing the NSX certificate, VDT reports "NSX VIP Cert Trust: FAIL" because the new self-signed cert root is not in SDDC Manager's keystores.

Step 1: Pull the NSX certificate (SSH to SDDC Manager as root):

```
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# Verify it is the correct cert
openssl x509 -in /tmp/nsx-root.crt -noout -text | grep -A2 "Subject Alternative
Name"
```

Step 2: Import into VCF trust store:

```
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Step 3: Import into Java cacerts:

```
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
-keystore /etc/alternatives/jre/lib/security/cacerts \
-storepass changeit -noprompt
```

Step 4: Restart SDDC Manager services:

```
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
# Wait ~5 minutes, then re-run VDT
```

Key trust store paths:

Item	Path/Value
VCF trust store	<code>/etc/vmware/vcf/commonsvcs/trusted_certificates.store</code>
VCF trust store password	Contents of <code>/etc/vmware/vcf/commonsvcs/trusted_certificates.key</code>
Java cacerts	<code>/etc/alternatives/jre/lib/security/cacerts</code>
Java cacerts password	<code>changeit</code>
Service restart script	<code>/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh</code>

Reference: <https://knowledge.broadcom.com/external/article/316056>

7.5.4 Service Status Checks

```
# SSH to NSX Manager as admin
ssh admin@192.168.1.71

# Check cluster status
get cluster status

# Check all service status (from root shell)
/etc/init.d/proton-manager status
```

```
/etc/init.d/corfu_server status
```

```
# Check NSX API health
```

```
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status
```

NSX Manager critical services:

Service	Purpose
MANAGER	NSX Management plane
SEARCH	Search/indexing service
UI	NSX Manager web interface
NODE_MGMT	Node management
proton	Core NSX engine
corfu	Distributed datastore

7.5.5 NSX Manager Cluster Issues

For single-node NSX deployments (common in nested labs):

```
# Check cluster health via API
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status

# DNS/NTP configured via admin CLI (NOT the UI)
ssh admin@192.168.1.71
set name-servers 192.168.1.230
set ntp-servers 192.168.1.230
get name-servers
get ntp-servers
```

7.5.6 NSX Traceflow for Network Debugging

1. Log in to NSX Manager: <https://nsx-vip.lab.local>
2. Navigate to Plan & Troubleshoot -> Traffic Analysis -> Traceflow
3. Configure source VM and destination VM/IP
4. Select protocol (ICMP, TCP, UDP)
5. Click "Trace" and review results:

- Green line = packet delivered successfully
- Red X = packet dropped (shows WHERE and by which rule)
- Yellow triangle = packet received but not forwarded

7.6 Offline Depot Troubleshooting

7.6.1 TLS/FIPS Compatibility

Problem: VCF 9.0.1 uses BouncyCastle FIPS TLS implementation which has strict certificate validation. Connection to offline depot with self-signed certificate fails.

Symptoms:

Secure protocol communication error, check logs for more details

LCM debug logs show:

```
org.bouncycastle.tls.TlsFatalAlert caught when processing request to {s}-
>https://192.168.1.160:8443
```

Diagnostic commands on VCF Installer / SDDC Manager:

```
# Test SSL connectivity
openssl s_client -connect 192.168.1.160:8443

# Test with TLS 1.2 specifically
openssl s_client -connect 192.168.1.160:8443 -tls1_2

# Check cipher negotiation
openssl s_client -connect 192.168.1.160:8443 -tls1_2 </dev/null 2>&1 | grep -E
"Cipher|Protocol|Verify"

# View certificate details
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-text -noout

# Get certificate fingerprint
```

```
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-noout -fingerprint -sha256
```

Fix: Import the depot certificate into the Java truststore:

```
# Download certificate from depot server
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-outform PEM > /tmp/depot.crt

# Verify certificate was downloaded
cat /tmp/depot.crt

# Find Java truststore
echo $JAVA_HOME
# Output: /usr/lib/jvm/openjdk-java17-headless.x86_64

# Delete old certificate if exists
keytool -delete -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit

# Import new certificate
keytool -import -trustcacerts -alias offline-depot -file /tmp/depot.crt -keystore
$JAVA_HOME/lib/security/cacerts -storepass changeit -noprompt

# Verify import
keytool -list -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit

# Restart LCM service
systemctl restart lcm

# Wait 2 minutes, verify LCM is ready
systemctl status lcm
tail -f /var/log/vmware/vcf/lcm/lcm-debug.log | grep -i "started\|ready"
```

7.6.2 404 Errors

Problem: SDDC Manager requests files that do not exist in the depot structure.

Symptoms in HTTPS server log:

```
192.168.1.125 - "HEAD /PROD/COMP/VCENTER/VMware-VCSA-all-9.0.1.0.24957454.iso HTTP/1.1"
404 -
```

Fix: Check the HTTPS server logs to identify the exact path requested. Place the file at the correct location:

```
C:\VCF-Depot\PROD\COMP\<COMPONENT>\<filename>
```

Reference: [Broadcom KB 413848](#)

7.6.3 Missing Catalog Entries

Problem: "Product Version Catalog (PVC) does not exist"

Cause: The `productVersionCatalog.json` was not extracted from the official `vcf-9.0.1.0-offline-depot-metadata.zip`, or the LCM-specific copy is missing.

Fix:

1. Extract metadata from the official zip file
2. Copy `productVersionCatalog.json` to:
`PROD\COMP\SDDC_MANAGER_VCF\lcm\productVersionCatalog\`

7.6.4 Certificate Mismatch

```
# Verify the depot server certificate matches what is in the truststore
# Get server certificate fingerprint
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-noout -fingerprint -sha256

# Get truststore certificate fingerprint
keytool -list -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit

# If fingerprints don't match, re-import the correct certificate
keytool -delete -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit
keytool -import -trustcacerts -alias offline-depot -file /tmp/depot.crt -keystore
$JAVA_HOME/lib/security/cacerts -storepass changeit -noprompt
systemctl restart lcm
```

7.6.5 Python HTTPS Server Issues

The offline depot uses a Python HTTPS server on the Windows host at 192.168.1.160:8443.

Starting the server:

```
cd C:\VCF-DEPOT
python https_server.py
```

Generating certificates (if needed):

```
cd C:\VCF-DEPOT
python generate_cert.py
# Then start the server
python https_server.py
```

Certificate requirements for FIPS compliance:

- RSA 2048-bit minimum
- SHA-256 signature
- TLS 1.2+ support
- SAN must include IP address of the depot server (192.168.1.160)
- BasicConstraints: CA:TRUE (for self-signed)

Monitoring depot requests:

Watch the HTTPS server console window during depot operations. Successful requests show **200** status codes. Any **404** indicates a file SDDC Manager expects but cannot find.

7.6.6 PVC Missing

If SDDC Manager reports "Product Version Catalog does not exist":

1. Verify the metadata zip was fully extracted
2. Check that **productVersionCatalog.json** exists at:

```
C:\VCF-Depot\PROD\COMP\SDDC_MANAGER_VCF\lcm\productVersionCatalog\productVersionCatalog.json
```

3. Check the HTTPS server can serve the file:

```
curl -k -u admin:admin  
https://192.168.1.160:8443/PROD/COMP/SDDC_MANAGER_VCF/lcm/productVersionCatalog
```

7.7 Recovery Procedures

7.7.1 SDDC Manager Recovery

Database Corruption:

```
# 1. Stop VCF services  
systemctl stop vcf-services  
  
# 2. Check disk space  
df -h  
  
# 3. Check memory  
free -m  
  
# 4. Restore PostgreSQL from backup (backup location varies)  
# Consult your backup documentation for restore procedure  
  
# 5. Restart services  
systemctl start vcf-services  
  
# 6. Verify services are running  
systemctl status vcf-services
```

Service Won't Start:

```
# 1. Check specific service logs  
tail -100 /var/log/vmware/vcf/<service>/<service>.log  
  
# 2. Check disk space (services fail if disk is full)  
df -h
```

```
# 3. Check memory
free -m

# 4. Restart individual service
systemctl restart <service-name>

# 5. If still failing, restart all services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

SDDC Manager UI Inaccessible:

```
# 1. Verify VM is powered on (check via vCenter or ESXi)
# 2. Verify network connectivity
ping 192.168.1.241

# 3. SSH as vcf user
ssh vcf@192.168.1.241
su -

# 4. Check Nginx
systemctl status nginx
nginx -t
systemctl restart nginx

# 5. Check all VCF services
systemctl status vcf-services

# 6. Restart all services if needed
systemctl restart vcf-services
# Wait 3-5 minutes
```

7.7.2 vCenter Recovery

From VAMI Backup:

1. Deploy a new vCenter appliance
2. During deployment wizard, select "Restore" instead of "Install"
3. Provide backup location (NFS/SMB/HTTP/SFTP) and credentials
4. Complete the restore wizard
5. Verify services start correctly: `service-control --status --all`

Service Recovery (no backup needed):

```
# SSH to vCenter
ssh root@vcenter.lab.local

# Check all services
service-control --status --all

# Restart a single failed service
service-control --restart <service-name>

# Or restart all services (causes outage)
service-control --restart --all

# Wait 10-15 minutes
```

7.7.3 NSX Manager Recovery**Single Node Failure (3-node cluster):**

1. Cluster continues operating on 2 nodes
2. Deploy a replacement NSX Manager appliance
3. Add the new node to the existing cluster
4. Wait for cluster synchronization

Single Node Recovery (lab with 1 node):

```
# Check NSX services
ssh admin@192.168.1.71
get cluster status

# If services are unhealthy, restart NSX Manager VM
# Power off, wait 30 seconds, power on
# Wait 10-15 minutes for all services to stabilize

# Verify via API
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status
```

Complete Cluster Recovery:

1. Restore from NSX backup
2. Reconfigure transport nodes if needed

3. Verify all host connectivity

7.7.4 Full Environment Cleanup and Redeployment

VCF does **NOT** provide a rollback mechanism for failed management domain deployments. A failed deployment requires manual cleanup:

Step 1: Delete Failed vCenter VM

```
# From the ESXi host running the vCenter VM
vim-cmd vmsvc/getallvms
# Find the vCenter VM ID (look for vcenter.lab.local)

# Power off if running
vim-cmd vmsvc/power.off <vmid>

# Unregister the VM
vim-cmd vmsvc/unregister <vmid>

# Delete VM files from datastore (if needed)
rm -rf /vmfs/volumes/<datastore>/vcenter.lab.local/
```

Step 2: Clean Up VDS (Distributed Switch)

```
# List current distributed switches
esxcli network vswitch dvs vmware list

# Remove VMkernel ports from VDS
esxcli network ip interface remove -i vmk1 # vMotion
esxcli network ip interface remove -i vmk2 # vSAN
```

Step 3: Clean Up vSAN Configuration (run on EACH ESXi host)

```
# List current vSAN storage
esxcli vsan storage list

# Remove vSAN disk groups
esxcli vsan storage remove -d
t10.ATA____VMware_Virtual_SATA_Hard_Drive_____03000000000000000001
esxcli vsan storage remove -d
```

```
t10.ATA____VMware_Virtual_SATA_Hard_Drive_____04000000000000000001

# Delete partitions from cache disk
partedUtil getptbl
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____03000000000000000000
partedUtil delete
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____03000000000000000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____03000000000000000000
2

# Delete partitions from capacity disk
partedUtil getptbl
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____04000000000000000000
partedUtil delete
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____04000000000000000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA____VMware_Virtual_SATA_Hard_Drive_____04000000000000000000
2

# Verify disks are now eligible
vdq -q
```

Common error: If you see `cache disk/s are in an invalid state...available size is 0.0 GB`, disks still have partitions. Use `partedUtil` to delete them.

Step 4: Verify Hosts Are Ready

```
# On each ESXi host, verify:
esxcli system hostname get
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"
vim-cmd hostsvc/runtimeinfo | grep ssh
vdq -q
esxcli network vswitch dvs vmware list
```

Step 5: Remove Depot Connection in VCF UI

1. Log in to VCF Installer UI (<https://192.168.1.240:8443>)

2. Navigate to Settings or Configuration
3. Remove the existing offline depot connection
4. Re-add the depot connection with certificate

Step 6: Restart VCF Services

```
systemctl restart lcm
systemctl restart domainmanager
sleep 120
systemctl status lcm
systemctl status domainmanager
```

Step 7: Retry Deployment

7.7.5 ESXi Host Recovery

Disconnected from vCenter:

```
# SSH to the host
ssh root@<esxi-host-ip>

# Check vpxa agent (vCenter agent)
/etc/init.d/vpxa status

# Restart vpxa
/etc/init.d/vpxa restart

# Restart all management agents
services.sh restart

# If still disconnected, force reconnect from vCenter UI:
# Right-click host -> Connection -> Disconnect
# Wait 30 seconds
# Right-click host -> Connection -> Connect
```

Rebuilding Host:

1. Install ESXi from ISO
2. Configure management network (IP, DNS, NTP, hostname)
3. Enable SSH: `vim-cmd hostsvc/enable_ssh && vim-cmd hostsvc/start_ssh`
4. Commission into VCF

5. Add to workload domain

7.7.6 Backup Recommendations

Component	Backup Method	Frequency
SDDC Manager	VM snapshot + PostgreSQL dump	Before any upgrade
vCenter	VAMI file-based backup (NFS/SFTP)	Daily
NSX Manager	NSX built-in backup to remote store	Daily
ESXi Configuration	Host profile / auto-backup.sh	After changes

ESXi auto-backup:

```
/sbin/auto-backup.sh
```

vCenter backup configuration:

1. Open VAMI: <https://vcenter.lab.local:5480>
2. Navigate to Backup
3. Configure backup schedule (protocol, location, credentials)
4. Schedule: Daily recommended

7.8 Troubleshooting Flowcharts

7.8.1 Deployment Failure Flowchart

```
START: VCF Deployment Failed
|
+---> Note reference token from error message
|       +---> Search logs: grep -r "TOKEN" /var/log/vmware/vcf/
|
+---> Delete failed vCenter VM
|       +---> vim-cmd vmsvc/getallvms
|       +---> vim-cmd vmsvc/power.off <vmid>
|       +---> vim-cmd vmsvc/unregister <vmid>
```

```

|
+---> Clean up vSAN on EACH host
|     +---> esxcli vsan storage remove -d <device>
|     +---> partedUtil delete ... (both partitions)
|     +---> vdq -q (verify eligible)
|
+---> Clean up VDS (if configured)
|     +---> esxcli network ip interface remove ...
|
+---> Remove depot connection in VCF UI
|     +---> Re-add with certificate
|
+---> Verify SSH enabled on all hosts
|     +---> vim-cmd hostsvc/enable_ssh
|
+---> Retry deployment

```

7.8.2 Certificate Issue Flowchart

```

START: VDT reports NSX cert FAIL (Trust or SAN)
|
+---> Check which check failed
|     +---> SAN FAIL: Certificate missing hostnames/IPs
|     +---> Trust FAIL: Certificate root not in SDDC Manager keystores
|
+---> If SAN FAIL:
|     +---> SSH to NSX Manager as root
|     +---> Create OpenSSL config with all SANs:
|         |     DNS.1 = nsx-vip.lab.local
|         |     DNS.2 = nsx-node1.lab.local
|         |     DNS.3 = nsx-manager.lab.local <-- SDDC Manager registered FQDN
|         |     IP.1 = 192.168.1.70 (VIP)
|         |     IP.2 = 192.168.1.71 (node)
|     +---> Generate cert: openssl req -x509 ...
|     +---> Build JSON: python (avoid shell PEM escaping)
|     +---> Import via API: POST /api/v1/trust-management/certificates?
action=import
|     +---> Apply to node: ?action=apply_certificate&service_type=API&node_id=
<uuid>
|     +---> Apply to VIP: ?action=apply_certificate&service_type=MGMT_CLUSTER
|
+---> If Trust FAIL (after cert replacement):
|     +---> SSH to SDDC Manager as vcf, then su - to root
|     +---> Pull cert: openssl s_client ... > /tmp/nsx-root.crt

```

```

|      +---> Import to VCF store: keytool -importcert ... trusted_certificates.store
|      +---> Import to Java cacerts: keytool -importcert ... cacerts
|      +---> Restart services: sddcmanager_restart_services.sh
|
+---> Re-run VDT after ~5 minutes
      +---> Expected: NSX cert checks all PASS

```

7.8.3 Offline Depot Connection Failure Flowchart

```

START: "Secure protocol communication error"
|
+---> Test connectivity: ping 192.168.1.160
|      +---> FAIL: Check network/firewall
|
+---> Test SSL: openssl s_client -connect 192.168.1.160:8443
|      +---> FAIL: Check depot server is running (python https_server.py)
|
+---> Check certificate: View cert details
|      +---> Wrong hostname/IP: Regenerate certificate (python generate_cert.py)
|
+---> Import certificate to Java truststore
|      +---> keytool -import -trustcacerts -alias offline-depot ...
|
+---> Verify fingerprints match
|      +---> MISMATCH: Re-import correct certificate
|
+---> Restart LCM service
|      +---> systemctl restart lcm
|      +---> Wait 2 minutes, retry connection

```

7.8.4 Service Failure Flowchart

```

START: VCF Component Service Not Responding
|
+---> Identify which component is affected
|      +---> SDDC Manager: https://sddc-manager.lab.local
|      +---> vCenter: https://vcenter.lab.local
|      +---> NSX: https://nsx-vip.lab.local
|
+---> Verify VM is powered on (check via vCenter or ESXi)
|      +---> Powered Off: Power on, wait 5-10 min
|

```

```

+---> SSH to the appliance
|
|   +---> SDDC Manager: ssh vcf@192.168.1.241 -> su -
|   +---> vCenter: ssh root@192.168.1.69
|   +---> NSX: ssh admin@192.168.1.71
|
+---> Check services
|
|   +---> SDDC Manager: systemctl status vcf-services
|   +---> vCenter: service-control --status --all
|   +---> NSX: get cluster status
|
+---> Restart failed services
|
|   +---> SDDC Manager: systemctl restart <service>
|   +---> vCenter: service-control --restart <service>
|   +---> NSX: Power cycle VM (wait 10-15 min in nested env)
|
+---> Check logs for errors
|
|   +---> SDDC Manager: /var/log/vmware/vcf/<service>/<service>.log
|   +---> vCenter: /var/log/vmware/vpxd/vpxd.log
|   +---> NSX: /var/log/proton/nsxapi.log
|
+---> Check database health
|
|   +---> SDDC Manager: systemctl status postgresql
|   +---> vCenter: service-control --status vmware-vpostgres
|
+---> If still not resolved:
|   +---> Collect SoS bundle: /opt/vmware/sddc-support/sos --log-bundle
|   +---> Open Broadcom support case

```

7.8.5 vSAN Issue Flowchart

```

START: vSAN Health Warning or Error
|
+---> Check vSAN Skyline Health
|   +---> vSphere Client -> Cluster -> Monitor -> vSAN -> Skyline Health
|
+---> Identify failure category
|   +---> Cluster health
|   +---> Network connectivity
|   +---> Data / object health
|   +---> Disk health
|   +---> Capacity limits
|
+---> If SSD Detection Failure (nested env):
|   +---> esxcli storage core device list | grep "Is SSD"

```

```

|      +---> If "Is SSD: false":
|      |      +---> Shut down ESXi VM in Workstation
|      |      +---> Edit VMX: sata0:X.virtualSSD = 1
|      |      +---> Power on, verify: esxcli storage core device list
|      +---> If "Has partitions":
|      |      +---> esxcli vsan storage remove -d <device>
|      |      +---> partedUtil delete ... (all partitions)
|      |      +---> vdq -q (verify eligible)
|
+---> If Object Degraded:
|      +---> Monitor -> vSAN -> Resyncing Components
|      +---> Allow rebuild to complete (ensure 30% free capacity)
|      +---> Do NOT make changes during rebuild
|
+---> If Disk Failed:
|      +---> Identify disk (serial number, slot)
|      +---> Remove from disk group
|      +---> Replace physically (hot-swap if supported)
|      +---> Add new disk to vSAN
|      +---> Monitor rebuild
|
+---> If Network Health Warning (nested env):
|      +---> Latency warnings are expected in nested environments
|      +---> Verify MTU is 1500 (NOT 9000)
|      +---> Test vSAN network: vmkping -I vmk2 <other-host-vsan-ip>

```

7.8.6 ESXi Certificate Mismatch Flowchart

```

START: "Certificate doesn't match subject alternative names"
|
+---> Check current cert SAN
|      +---> openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1
"Subject Alternative Name"
|
+---> Set correct hostname
|      +---> esxcli system hostname set --fqdn=esxi01.lab.local
|
+---> Backup old certificates
|      +---> mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
|      +---> mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
|
+---> Generate new certificates
|      +---> /sbin/generate-certificates
|

```

```

+---> Restart services
|      +---> services.sh restart
|
+---> Update thumbprints in VCF
|      +---> Re-validate hosts in UI
|      +---> Get new thumbprints:
|              echo | openssl s_client -connect 192.168.1.74:443 2>/dev/null | openssl
x509 -noout -fingerprint -sha256

```

7.8.7 vCenter Deployment Stuck Flowchart

```

START: vCenter deployment stuck at percentage
|
+---> Wait 30 minutes (large downloads may be slow)
|
+---> SSH to vCenter VM (ssh root@vcenter.lab.local, password: vmware)
|
+---> Check firstboot status
|      +---> cat /var/log/firstboot/firstbootStatus.json
|
+---> Check for activity
|      +---> vmstat 1 5 (disk I/O)
|      +---> tail -f /var/log/vmware/firstboot/installer.log
|
+---> If stuck at 60% "Installing Containers":
|      +---> Check postgres: ls /storage/db/vpostgres/
|      +---> Missing postgresql.conf: Database failed to init
|      +---> UNRECOVERABLE: Must redeploy
|
+---> Check services: vmon-cli --list
|      +---> Services not started: Check individual logs
|
+---> If unrecoverable:
|      +---> Delete vCenter VM (vim-cmd vmsvc/unregister)
|      +---> Clean up vSAN on all hosts
|      +---> Reset depot connection
|      +---> Retry deployment

```

7.8.8 vLCM Host Seeding Failure Flowchart

```

START: "Extraction of image from host failed"
|

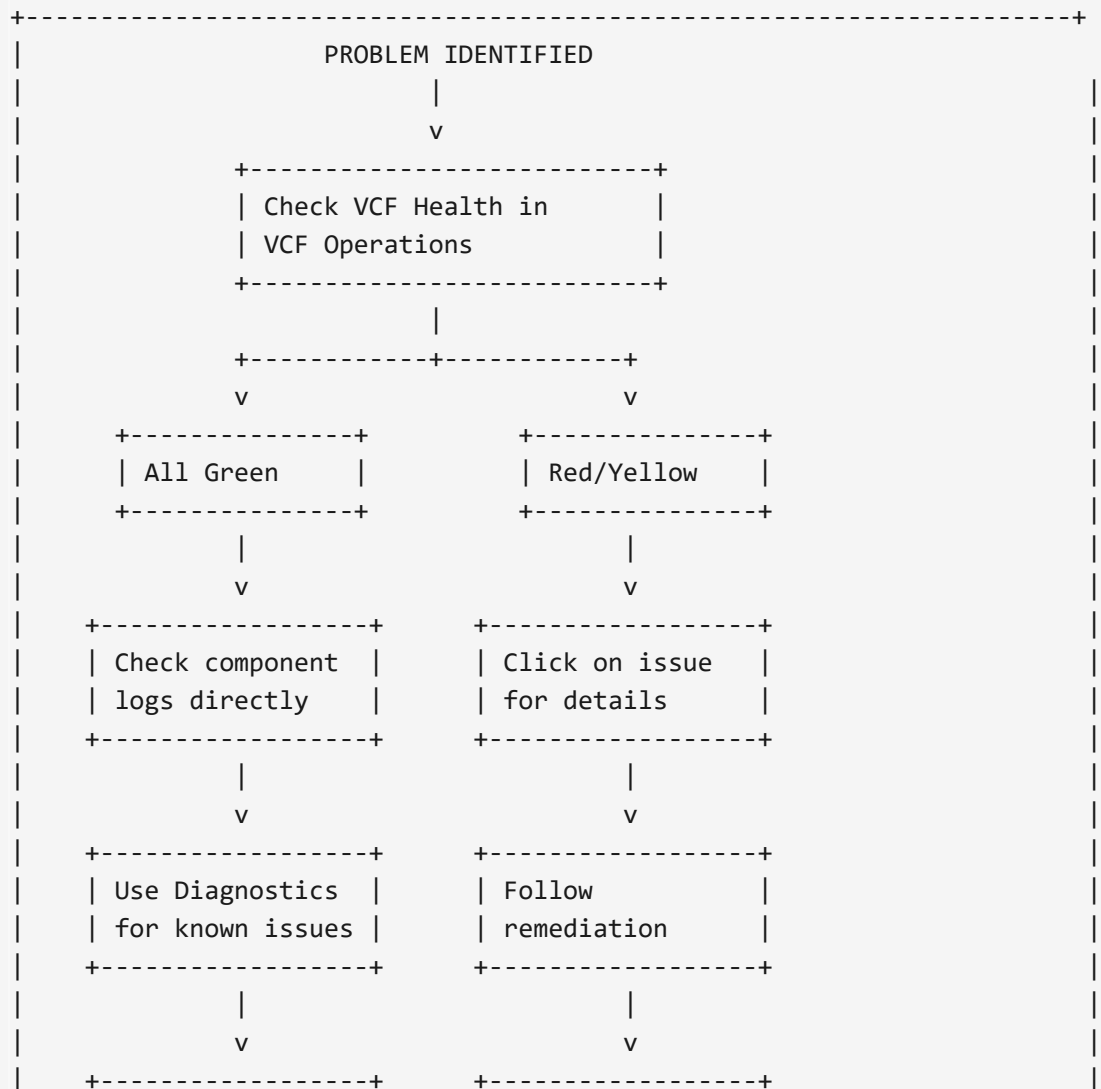
```

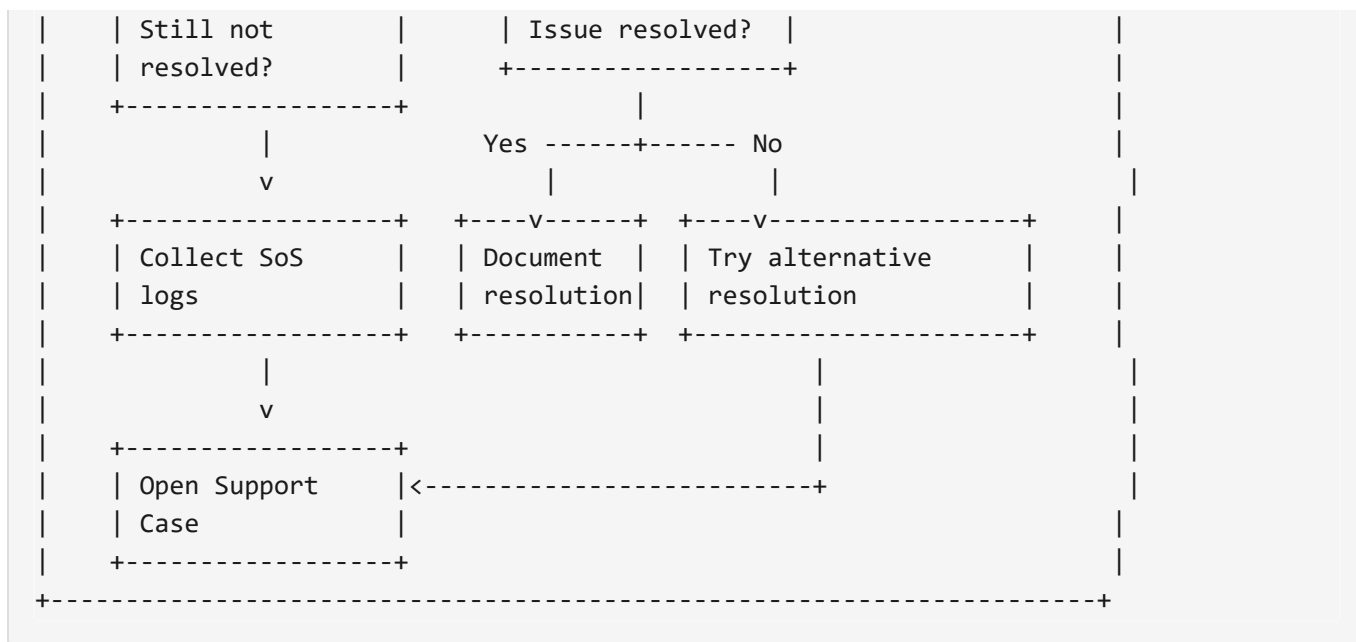
```

+---> Check SSH status on ESXi host
|     +---> vim-cmd hostsvc/runtimeinfo | grep ssh
|
+---> SSH Disabled?
|     +---> vim-cmd hostsvc/enable_ssh
|     +---> vim-cmd hostsvc/start_ssh
|
+---> Verify SSH on ALL hosts (esxi01-04)
|     +---> esxcli system ssh set --enable=true
|     +---> esxcli system ssh get
|
+---> Retry vCenter deployment

```

7.8.9 General Problem Identification Decision Tree





7.8.10 Common Error Messages Quick Reference

Error	Cause	Resolution
"Secure protocol communication error"	Self-signed cert not trusted	Import cert to Java truststore, restart LCM
"Certificate doesn't match subject alternative names"	ESXi cert has wrong hostname	Regenerate cert: <code>/sbin/generate-certificates</code>
"Found zero SSD devices"	VMX missing virtualSSD flag	Edit VMX: <code>sata0:X.virtualSSD = 1</code>
"Migration failed...VHV enabled"	Ghost vhv.enable in runtime	Add explicit <code>vhv.enable = "FALSE"</code> to VMX
"Memory convergence timeout"	Nested env bandwidth limit	Use cold migration as fallback
"Password out of sync"	Password changed outside VCF	Use Update Password in SDDC Manager
"Transport node disconnected"	TEP connectivity issue	Check VTEP, MTU, NSX proxy on host
"vSAN degraded"	Disk or host failure	Allow rebuild, replace failed components
"Task failed - prerequisite not met"	Missing dependency	Complete prerequisite first, retry
"503 Service Unavailable" (vCenter)	vCenter services down	<code>service-control --restart --all</code>

"NSX Manager unavailable"	NSX OOM or service crash	Check RAM (need 32GB nested), restart
"SAN contains neither hostname nor IP" (VDT)	NSX cert uses wildcard SAN	Replace cert with explicit SANs
"Product Version Catalog does not exist"	PVC file missing in depot	Extract metadata, copy to correct path
"Extraction of image from host failed"	SSH disabled on ESXi	Enable SSH: <code>vim-cmd hostsvc/enable_ssh</code>

7.8.11 Log Locations Quick Reference

Component	Log Path
SDDC Manager (all)	<code>/var/log/vmware/vcf/</code>
SDDC Manager Domain Manager	<code>/var/log/vmware/vcf/domainmanager/domainmanager.log</code>
SDDC Manager LCM	<code>/var/log/vmware/vcf/lcm/lcm.log</code>
SDDC Manager LCM Debug	<code>/var/log/vmware/vcf/lcm/lcm-debug.log</code>
SDDC Manager Ops Manager	<code>/var/log/vmware/vcf/operationsmanager/operationsmanager.log</code>
VDT Reports	<code>/var/log/vmware/vcf/vdt/vdt-<code><timestamp></code>.txt</code>
SoS Bundles	<code>/var/log/vmware/vcf/sddc-support/sos-<code><timestamp></code>.tar.gz</code>
vCenter vpxd	<code>/var/log/vmware/vpxd/vpxd.log</code>
vCenter vSphere UI	<code>/var/log/vmware/vsphere-ui/logs/vsphere_client_virgo.log</code>
vCenter PostgreSQL	<code>/var/log/vmware/vpostgres/postgresql-*.log</code>
vCenter firstboot	<code>/var/log/firstboot/firstbootStatus.json</code>
NSX Manager	<code>/var/log/proton/nsxapi.log</code>
NSX Syslog (on ESXi)	<code>/var/log/nsx-syslog.log</code>
ESXi hostd	<code>/var/log/hostd.log</code>
ESXi vpxa	<code>/var/log/vpxa.log</code>
ESXi vmkernel	<code>/var/log/vmkernel.log</code>

vSAN health

`/var/log/vmware/vsan-health/`

7.8.12 Critical Port Numbers

Service	Port	Protocol
SDDC Manager UI	443	HTTPS
vCenter Server	443	HTTPS
NSX Manager	443	HTTPS
ESXi Management	443, 902	HTTPS, VMware
SSH	22	TCP
vSAN	2233	TCP
vMotion	8000	TCP
NSX Manager Cluster	1234	TCP
Offline Depot	8443	HTTPS

PART VIII: Complete Command Reference

8.1 ESXi Commands

8.1.1 esxcli Commands

esxcli system -- System administration and configuration.

```
# Display hostname, FQDN, and domain
esxcli system hostname get

# Set fully qualified domain name
esxcli system hostname set --fqdn=esxi01.lab.local

# Set short hostname only
esxcli system hostname set --host=esxi01

# Set domain only
esxcli system hostname set --domain=lab.local

# Get ESXi version and build number
esxcli system version get

# Enter maintenance mode (no vSAN data evacuation)
esxcli system maintenanceMode set -e true -m noAction

# Enter maintenance mode (evacuate all vSAN data)
esxcli system maintenanceMode set -e true -m evacuateAllData

# Exit maintenance mode
esxcli system maintenanceMode set -e false

# Check maintenance mode status
esxcli system maintenanceMode get

# Get system time
esxcli system time get
```

esxcli network -- VMkernel, vSwitch, IP, and firewall management.

```
# List all VMkernel interfaces
esxcli network ip interface list

# Get IPv4 configuration for a specific VMkernel interface
esxcli network ip interface ipv4 get -i vmk0

# Set IPv4 address on VMkernel interface (static)
esxcli network ip interface ipv4 set -i vmk2 -I 192.168.12.74 -N 255.255.255.0 -t
static

# Add a new VMkernel interface
esxcli network ip interface add -i vmk1 -p "vMotion"

# List all standard vSwitches with uplinks and portgroups
esxcli network vswitch standard list

# Add uplink NIC to vSwitch
esxcli network vswitch standard uplink add -u vmnic3 -v vSwitch0

# Remove uplink NIC from vSwitch
esxcli network vswitch standard uplink remove -u vmnic3 -v vSwitch0

# Get failover policy (active, standby, unused adapters)
esxcli network vswitch standard policy failover get -v vSwitch0

# Set adapter as active in failover policy
esxcli network vswitch standard policy failover set -v vSwitch0 -a vmnic3

# Get security policy for a vSwitch
esxcli network vswitch standard policy security get -v vSwitch0

# Get security policy for a specific portgroup
esxcli network vswitch standard portgroup policy security get -p "VM Network"

# List distributed virtual switches
esxcli network vswitch dvs vmware list

# List all physical NICs with link status and speed
esxcli network nic list

# Get detailed NIC information
esxcli network nic get -n vmnic0
```

```
# Get NIC traffic statistics
esxcli network nic stats get -n vmnic0

# Filter NIC stats for packet and byte counts
esxcli network nic stats get -n vmnic0 | grep -E "Packets|Bytes"

# Show ARP table entries
esxcli network ip neighbor list

# Filter ARP for specific subnet
esxcli network ip neighbor list | grep 192.168.12

# Show IPv4 routing table
esxcli network ip route ipv4 list

# List active network connections
esxcli network ip connection list

# Filter connections for NSX Manager communication (port 1234)
esxcli network ip connection list | grep 1234

# List firewall rulesets and their enabled/disabled status
esxcli network firewall ruleset list

# Filter firewall for SSH rules
esxcli network firewall ruleset list | grep -i ssh
```

esxcli storage -- Device, adapter, and filesystem management.

```
# List all storage devices with capacity, vendor, model, SSD status
esxcli storage core device list

# Filter for SSD detection status
esxcli storage core device list | grep -E "Display Name|Is SSD"

# Rescan all storage adapters for new devices
esxcli storage core adapter rescan --all

# Rescan a specific adapter
esxcli storage core adapter rescan --adapter=vmhba0

# List all storage adapters
esxcli storage core adapter list
```

```
# List all mounted filesystems and VMFS datastores
esxcli storage filesystem list

# List VMFS extents
esxcli storage vmfs extent list

# Rescan VMFS filesystems
esxcli storage filesystem rescan
```

esxcli vsan -- vSAN cluster, storage, health, and network operations.

```
# Get vSAN cluster status (member count, node state, health)
esxcli vsan cluster get

# Force host to leave vSAN cluster (CAUTION)
esxcli vsan cluster leave

# List unicast agents (all cluster members)
esxcli vsan cluster unicastagent list

# List vSAN storage devices and disk groups
esxcli vsan storage list

# Disable automatic disk claiming
esxcli vsan storage automode set --enabled=false

# Enable automatic disk claiming
esxcli vsan storage automode set --enabled=true

# Add storage to vSAN (cache + capacity tier)
esxcli vsan storage add -s <cache-device> -d <capacity-device>

# Remove device from vSAN
esxcli vsan storage remove -s <device>

# List vSAN health checks and their status
esxcli vsan health cluster list

# Get specific health test results
esxcli vsan health cluster get -t "vSAN Health"

# List vSAN network adapters
esxcli vsan network list
```

```
# Add VMkernel interface to vSAN traffic
esxcli vsan network ip add -i vmk1

# Remove VMkernel interface from vSAN traffic
esxcli vsan network ip remove -i vmk1

# Show vSAN resync status and progress
esxcli vsan debug resync summary

# List vSAN objects for debugging
esxcli vsan debug object list
```

esxcli software -- VIB and software depot management.

```
# List installed VIBs
esxcli software vib list

# Install a VIB from a local path
esxcli software vib install -v /path/to/vib.vib

# Remove a VIB
esxcli software vib remove -n <vib-name>

# Show installed software profile
esxcli software profile get

# Add software depot
esxcli software sources profile list -d /path/to/depot.zip
```

8.1.2 vmkfstools Commands

```
# Display VMDK metadata and lock information
vmkfstools -D "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmdk"

# Clone VMDK from one datastore to another (thick to thin conversion)
# Lab-tested: Used to migrate SDDC Manager from local to vSAN (914GB thick -> 108GB thin)
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmdk
/vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmdk -d thin
```

```
# Clone as thin provisioned (per-disk for large VMs)
vmkfstools -i <source-vmdk> <destination-vmdk> -d thin

# Clone as thick lazy zeroed
vmkfstools -i <source-vmdk> <destination-vmdk> -d zeroedthick

# Clone as thick eager zeroed
vmkfstools -i <source-vmdk> <destination-vmdk> -d eagerzeroedthick

# Delete a VMDK file (use when cleaning failed clones)
vmkfstools -U /vmfs/volumes/<datastore>/<vm>/<disk>.vmdk

# Create a new VMDK (50GB thin)
vmkfstools -c 50G -d thin /vmfs/volumes/<datastore>/<vm>/newdisk.vmdk

# Extend an existing VMDK to 100GB
vmkfstools -X 100G /vmfs/volumes/<datastore>/<vm>/disk.vmdk

# Get disk geometry information
vmkfstools -g /vmfs/volumes/<datastore>/<vm>/disk.vmdk
```

Disk format types:

Flag	Format	Description
-d thin	Thin provisioned	Allocates space on demand (saves storage)
-d zeroedthick	Thick lazy zeroed	Allocates full space, zeros on first write
-d eagerzeroedthick	Thick eager zeroed	Allocates and zeros all space immediately

8.1.3 Other ESXi Commands

vdq -- Disk qualification for vSAN:

```
# List all eligible disks for vSAN
vdq -qH

# Detailed disk qualification query
vdq -q -d <device-name>
```

esxtop -- Real-time performance monitoring:

```
# Launch interactive performance monitor
esxtop

# Batch mode: capture to CSV (5-second intervals, 10 samples)
esxtop -b -d 5 -n 10 > /tmp/esxtop.csv
```

Interactive view keys:

Key	View	Key Columns
c	CPU	%USED, %RDY, %CSTP, %MLMTD
m	Memory	MCTLSZ (balloon), SWCUR (swap), CACHEUSD
n	Network	MbTX/s, MbRX/s, %DRPTX, %DRPRX
d	Disk/Storage	DAVG (device latency), KAVG (kernel latency), GAVG (guest latency)
v	VM view	Per-VM resource utilization
u	Disk device	Per-device I/O statistics

vim-cmd -- VM management from ESXi shell:

```
# List all registered VMs with VMIDs
vim-cmd vmsvc/getallvms

# Get power state of a VM
vim-cmd vmsvc/power.getstate <vmid>

# Power on a VM
vim-cmd vmsvc/power.on <vmid>

# Power off a VM (hard power off)
vim-cmd vmsvc/power.off <vmid>

# Graceful shutdown (requires VMware Tools)
vim-cmd vmsvc/power.shutdown <vmid>

# Reset (hard reboot) a VM
vim-cmd vmsvc/power.reset <vmid>

# Register a VM from its VMX file
```

```
vim-cmd solo/registervm "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmx"

# Unregister a VM (does not delete files)
vim-cmd vmsvc/unregister <vmid>

# List all devices attached to a VM
vim-cmd vmsvc/device.getdevices <vmid>

# Force VM into BIOS/EFI on next boot
vim-cmd vmsvc/setboot.options <vmid> enterBIOSSetup=true

# Enter maintenance mode
vim-cmd hostsvc/maintenance_mode_enter

# Exit maintenance mode
vim-cmd hostsvc/maintenance_mode_exit
```

localcli -- Bypass hostd for direct VMkernel operations:

```
# Useful when hostd is unresponsive
localcli network ip interface list
localcli storage core device list
localcli system hostname get
```

dcli -- vCenter REST API client on ESXi:

```
# List VMs via vCenter API from ESXi shell
dcli +server vcenter.lab.local +username administrator@vsphere.local com vmware
vcenter vm list
```

esxcfg- -- Legacy network configuration commands:*

```
# List all VMkernel interfaces with IP, MTU, and enabled services
esxcfg-vmknics -l

# List all virtual switches with portgroups and uplinks
esxcfg-vswitch -l
```

```
# List physical NICs with driver, link state, speed, duplex
esxcfg-nics -l
```

vmkping -- VMkernel stack ping utility:

```
# Basic ping
vmkping 192.168.12.75

# Ping from specific VMkernel interface
vmkping -I vmk2 192.168.12.75

# MTU test with Don't Fragment flag (1600 byte total for overlay networks)
vmkping -d -s 1572 192.168.12.75

# Ping with count
vmkping -c 10 192.168.12.75
```

vscsiStats -- Storage I/O statistics:

```
# List VMs available for storage statistics
vscsiStats -l

# Start collecting stats for a VM
vscsiStats -s -w <world-id>

# Print storage statistics
vscsiStats -p all -w <world-id>
```

vsish -- VMkernel System Information Shell:

```
# List vsish nodes
vsish -e ls /

# Get memory statistics
vsish -e get /memory/comprehensive

# Get network portset info
vsish -e get /net/portsets/
```

Partition utilities:

```
# Display partition table of a disk
partedUtil getptbl /dev/disks/<device-name>

# Create fresh GPT label (DESTROYS ALL DATA)
partedUtil mklabel /dev/disks/<device-name> gpt
```

ESXi service control scripts:

```
# Restart ALL management services (causes brief outage)
services.sh restart

# Host daemon (hostd) control
/etc/init.d/hostd restart
/etc/init.d/hostd status

# vCenter agent (vpxa) control
/etc/init.d/vpxa restart
/etc/init.d/vpxa status

# SSH service control
/etc/init.d/SSH status
/etc/init.d/SSH start
/etc/init.d/SSH stop

# NSX proxy agent on ESXi
/etc/init.d/nsx-proxy status
/etc/init.d/nsx-proxy restart

# NSX operations agent
/etc/init.d/nsx-opsagent status

# NSX datapath (distributed firewall)
/etc/init.d/nsx-datapath status

# Regenerate ESXi SSL certificates (run after FQDN change)
/sbin/generate-certificates

# Persist configuration changes across reboots
/sbin/auto-backup.sh
```

8.2 vCenter Commands

```
# Check status of ALL vCenter services
service-control --status --all

# Check status of a specific service
service-control --status vpxd

# Start all services
service-control --start --all

# Stop all services (causes vCenter outage)
service-control --stop --all

# Restart a specific service
service-control --restart vpxd
service-control --restart vsphere-client
service-control --restart vmware-vpostgres
service-control --restart vsphere-ui

# Restart all services (causes brief outage)
service-control --restart --all
```

Critical vCenter services:

Service	Purpose
vpxd	Core vCenter Server daemon
vsphere-ui	vSphere Client web interface
vmware-vpostgres	Embedded PostgreSQL database
vmcad	Certificate Authority daemon
vmmdir	Directory Service (vmmdir)
vmafdd	Authentication Framework daemon
vmware-sps	Profile-Driven Storage
vlcm	vSphere Lifecycle Manager
eam	ESX Agent Manager

lookupsvc	Lookup Service
appliancegmt	Appliance Management

vCenter database operations:

```
# Connect to vCenter PostgreSQL database
/opt/vmware/vpostgres/current/bin/psql -U postgres

# Test database connection
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT 1;"

# Check active database connections
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT count(*) FROM
pg_stat_activity;"
```

vCenter certificate management:

```
# Launch certificate manager wizard
/usr/lib/vmware-vmca/bin/certificate-manager

# List certificates in VECS stores
for store in MACHINE_SSL_CERT TRUSTED_ROOTS; do
    /usr/lib/vmware-vmca/bin/vecs-cli entry list --store $store
done
```

SSO management (cmsso-util):

```
# Repoint vCenter to external Platform Services Controller (legacy)
cmsso-util repoint --repoint-partner <psc-fqdn>

# List SSO domain information
/opt/vmware/bin/dir-cli service list --login administrator@vsphere.local
```

Appliance management:

```
# Get appliance version
vamicli version --appliance
```

```
# Check for available updates
vamicli update --check

# VAMI login shell
/opt/vmware/share/vami/vami_login
```

8.3 SDDC Manager Commands

VCF service management (systemctl):

```
# Check all VCF services status
systemctl status vcf-services

# Restart all VCF services
systemctl restart vcf-services

# Start all VCF services
systemctl start vcf-services

# Stop all VCF services
systemctl stop vcf-services
```

Individual SDDC Manager services:

Service Name	systemctl Command
Domain Manager	<code>systemctl status domainmanager / systemctl restart domainmanager</code>
Lifecycle Manager	<code>systemctl status lcm / systemctl restart lcm</code>
Operations Manager	<code>systemctl status operationsmanager / systemctl restart operationsmanager</code>
NGINX (reverse proxy)	<code>systemctl status nginx / systemctl restart nginx</code>
PostgreSQL (database)	<code>systemctl status postgresql / systemctl restart postgresql</code>
SDDC Manager UI	<code>systemctl restart sddc-manager-ui-app.service</code>

Common Services

`systemctl status commonsvcs`**Service discovery:**

```
# List all VCF-related systemd service units
systemctl list-units --type=service | grep vcf
```

SOS utility (Supportability and Serviceability):Path: `/opt/vmware/sddc-support/sos`

```
# Collect comprehensive log bundle for VMware support
/opt/vmware/sddc-support/sos --log-bundle

# Run health check on SDDC Manager and all components
/opt/vmware/sddc-support/sos --health-check

# Collect logs for a specific workload domain
/opt/vmware/sddc-support/sos --domain-name mgmt

# Get inventory of all VCF components
/opt/vmware/sddc-support/sos --get-inventory

# Clean up old log bundles to free disk space
/opt/vmware/sddc-support/sos --cleanup-logs

# Retrieve current passwords (requires authentication)
/opt/vmware/sddc-support/sos --get-passwords

# Backup SDDC Manager configuration
/opt/vmware/sddc-support/sos --backup-config
```

SDDC Manager database (PostgreSQL):

Always use `PAGER=cat` when running `psql` on SDDC Manager to prevent pager traps in remote/scripted sessions.

```
# Connect to SDDC Manager database (use -h 127.0.0.1, NOT localhost or Unix
sockets)
```

```

su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

# Test database connection
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -c 'SELECT 1;'"

# List all databases
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -l"

# Backup SDDC Manager database
su - postgres -c "pg_dump -h 127.0.0.1 platform > /tmp/platform_backup.sql"

# Full cascade repair (quick reference)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt SET
status = 'ACTIVE' WHERE status != 'ACTIVE';\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE task_metadata
SET resolved = true WHERE resolved = false;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
task_lock;\""
# See Section 7.2.6 for full procedure with diagnosis and verification

```

psql internal commands:

Command	Description
\dt	List all tables
\l	List databases
\d <table>	Describe table columns
\q	Exit psql
\?	Help

Configuration file locations on SDDC Manager:

File	Purpose
/etc/vmware/vcf/domainmanager/application-prod.properties	Domain Manager configuration
/etc/vmware/vcf/commonsvcs/trusted_certificates.store	VCF trust store (password in .key file)
/etc/vmware/vcf/commonsvcs/trusted_certificates.key	VCF trust store password

<code>/etc/alternatives/jre/lib/security/cacerts</code>	Java cacerts trust store (password: <code>changeit</code>)
<code>/etc/resolv.conf</code>	DNS configuration
<code>/nfs/vmware/vcf/nfs-mount/bundle/</code>	VCF bundle depot directory

File transfer workaround (SCP does not work with restricted shell):

```
# SDDC Manager only allows SSH as 'vcf' user (root/admin rejected for SSH)
# SCP fails due to restricted shell; use ssh cat method instead:
ssh vcf@192.168.1.241 "cat > /home/vcf/file.zip" < localfile.zip

# Root access: su - from vcf session
ssh vcf@192.168.1.241
su -
```

SDDC Manager service restart script (alternative):

```
# Full service restart with proper sequencing
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

8.4 NSX Manager Commands

8.4.1 NSX CLI Commands

All NSX CLI commands are run from the NSX Manager console or SSH session as `admin` . NSX shell does NOT support backslash line continuation -- all commands must be single-line.

```
# Get cluster status (controller cluster health)
get cluster status

# List NSX Manager nodes
get managers

# Get cluster node details
get cluster nodes
```

```
# Get certificate information
get certificate api

# List all transport nodes
get transport-nodes

# Get transport node status by UUID
get transport-node <uuid> status

# List all logical switches (segments)
get logical-switches

# List all logical routers (gateways)
get logical-routers

# List all interfaces
get interfaces

# Show VTEP (Tunnel Endpoint) information
get vtep

# Display VTEP table entries
get vtep-table

# List all distributed firewall rules
get firewall rules

# Check DFW status
get firewall status

# Get details of a specific firewall rule
get firewall rule <rule-id>

# Start a traceflow for network debugging
start traceflow --src-port <port-id> --dst-ip <ip>

# Get traceflow results
get traceflow <traceflow-id>

# Set DNS servers (admin CLI, NOT the UI)
set name-servers 192.168.1.230

# Set NTP servers (admin CLI)
set ntp-servers 192.168.1.230

# Restart a specific NSX service
```

```
restart service <service-name>
```

```
# Check NSX service status
get service <service-name>
```

8.4.2 NSX API Commands (curl)

All curl commands to NSX must be single-line. No backslash continuation in NSX shell.

```
# Check NSX cluster status
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status

# Get full cluster information (includes node UUIDs)
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster

# List all certificates
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/trust-
management/certificates

# Import a certificate (use Python to build JSON payload for PEM data)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates?action=import" -H "Content-Type: application/json" -d
@/tmp/nsx-import.json

# Apply certificate to NSX Manager node (API service)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=API&node_id=<node-uuid>"

# Apply certificate to cluster VIP (management cluster)
curl -k -u admin:'Success01!0909!!' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=MGMT_CLUSTER"

# List transport nodes via API
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/transport-nodes

# List segments via API
curl -k -u admin:'Success01!0909!!'
https://192.168.1.71/policy/api/v1/infra/segments

# Get transport zone list
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/transport-zones
```

```
# List compute managers
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/fabric/compute-managers
```

Building JSON payload for certificate import (Python method):

```
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

This avoids shell escaping issues with `\n` characters in PEM data.

8.5 Certificate Commands

8.5.1 OpenSSL Commands

```
# Generate a self-signed certificate and private key (basic)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes -subj '/CN=hostname'

# Generate with Subject Alternative Names (SANs)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes \
  -subj "/CN=192.168.1.52/O=VCF-Depot/C=US" \
  -addext "subjectAltName=IP:192.168.1.52,DNS:localhost" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth"

# Generate private key separately
openssl genrsa -out server.key 2048

# Generate CSR (Certificate Signing Request)
openssl req -new -key server.key -out server.csr -subj "/CN=hostname/O=Org/C=US"

# Sign CSR with CA certificate
```

```
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
server.crt -days 365
```

```
# Generate certificate using config file (lab-tested for NSX)
```

```
openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256
```

```
# View full certificate details
```

```
openssl x509 -in cert.crt -text -noout
```

```
# View Subject Alternative Names only
```

```
openssl x509 -in cert.crt -text -noout | grep -A1 'Subject Alternative Name'
```

```
# View certificate validity dates
```

```
openssl x509 -in cert.crt -noout -dates
```

```
# View expiration date only
```

```
openssl x509 -in cert.crt -noout -enddate
```

```
# View certificate subject
```

```
openssl x509 -in cert.crt -noout -subject
```

```
# View certificate issuer
```

```
openssl x509 -in cert.crt -noout -issuer
```

```
# Verify certificate against CA
```

```
openssl verify -CAfile ca.crt server.crt
```

```
# View remote server certificate (connect and display chain)
```

```
openssl s_client -connect vcenter.lab.local:443 -showcerts
```

```
# Pull remote certificate and save to file
```

```
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt
```

```
# Check certificate fingerprint (SHA-256)
```

```
openssl x509 -in cert.crt -noout -fingerprint -sha256
```

```
# Convert PEM to DER format
```

```
openssl x509 -in cert.pem -outform der -out cert.der
```

```
# Convert DER to PEM format
```

```
openssl x509 -in cert.der -inform der -outform pem -out cert.pem
```

8.5.2 Keytool Commands

```
# Import certificate into a Java truststore
keytool -import -trustcacerts -alias <name> -file <cert> -keystore <cacerts> -
storepass changeit -noprompt

# Example: import into Cloud Builder / SDDC Manager Java cacerts
keytool -import -trustcacerts -alias vcf-depot \
  -file /tmp/depot.crt \
  -keystore /usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts \
  -storepass changeit -noprompt

# List all certificates in a keystore (summary)
keytool -list -keystore /etc/alternatives/jre/lib/security/cacerts -storepass
changeit

# List certificates with full details (verbose)
keytool -list -v -keystore /etc/alternatives/jre/lib/security/cacerts -storepass
changeit

# Delete a certificate from keystore
keytool -delete -alias <name> -keystore /etc/alternatives/jre/lib/security/cacerts
-storepass changeit

# Export a certificate from keystore
keytool -export -alias <name> -keystore <cacerts> -storepass changeit -file
exported.crt
```

Common VCF keystores:

Keystore Path	Password	Purpose
/etc/alternatives/jre/lib/security/cacerts	changeit	Java default trust store
/etc/vmware/vcf/commonsvcs/trusted_certificates.store	Contents of .key file	VCF common services trust store
/usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts	changeit	Java 17 trust store

Lab-tested: Import NSX self-signed cert into SDDC Manager trust stores:

```
# Step 1: Pull the active NSX certificate
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# Step 2: Import into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvc/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvc/trusted_certificates.store \
  -storepass "$KEY" -noprompt

# Step 3: Import into Java cacerts
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Step 4: Restart SDDC Manager services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

8.6 Windows / Depot Commands

PowerShell commands for depot and certificate management:

```
# Disable Hyper-V (required for nested virtualization in VMware Workstation)
bcdedit /set hypervisorlaunchtype off
Disable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All -
NoRestart
Disable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -
NoRestart
Disable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-
Linux -NoRestart

# Verify hypervisor is off after reboot
bcdedit /enum | findstr hypervisor

# Check Device Guard / VBS status (VirtualizationBasedSecurityStatus should be 0)
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace
root\Microsoft\Windows\DeviceGuard
```

```
# Check VMX file settings from Windows
type "D:\VMs\esxi01.lab.local\esxi01.lab.local.vmx" | findstr /i "vhv vpmc vvtd"
```

certutil commands (Windows certificate management):

```
# View certificate details
certutil -dump cert.crt

# Verify certificate chain
certutil -verify cert.crt

# Import certificate into Windows trust store
certutil -addstore Root cert.crt

# Export certificate from Windows store
certutil -exportPFX -p "password" Root cert.pfx

# Hash a file (verify download integrity)
certutil -hashfile file.zip SHA256
```

DNS management (Windows Server):

```
# Add forward DNS record (A record)
Add-DnsServerResourceRecordA -Name "vcenter" -ZoneName "lab.local" -IPv4Address
"192.168.1.69"

# Add reverse DNS record (PTR record)
Add-DnsServerResourceRecordPtr -Name "69" -ZoneName "1.168.192.in-addr.arpa" -
PtrDomainName "vcenter.lab.local"

# Verify DNS resolution
nslookup vcenter.lab.local

# Verify reverse DNS
nslookup 192.168.1.69

# List all DNS records in a zone
Get-DnsServerResourceRecord -ZoneName "lab.local"
```

8.7 API Quick Reference

SDDC Manager API endpoints:

Method	Endpoint	Purpose
POST	/v1/tokens	Get authentication bearer token
GET	/v1/system	System information
GET	/v1/hosts	List all commissioned hosts
GET	/v1/domains	List all workload domains
GET	/v1/tasks	List all tasks
PATCH	/v1/tasks/<id>	Cancel a stuck task
GET	/v1/clusters	List all clusters
GET	/v1/nsxt-clusters	List NSX clusters
GET	/v1/vcenters	List all vCenter instances
GET	/v1/credentials	List all managed credentials
GET	/v1/bundles	List available bundles
POST	/v1/bundles	Upload a bundle

```
# Authenticate and get bearer token
curl -k -X POST https://sddc-manager.lab.local/v1/tokens -H "Content-Type:
application/json" -d '{"username":"admin@local","password":"Success01!0909!!"}'

# Get system information
curl -k -X GET https://sddc-manager.lab.local/v1/system -H "Authorization: Bearer
<token>"

# List all hosts
curl -k -X GET https://sddc-manager.lab.local/v1/hosts -H "Authorization: Bearer
<token>"

# List all domains
curl -k -X GET https://sddc-manager.lab.local/v1/domains -H "Authorization: Bearer
<token>"
```

```
# List all tasks
curl -k -X GET https://sddc-manager.lab.local/v1/tasks -H "Authorization: Bearer <token>"

# Cancel a stuck task
curl -k -X PATCH https://sddc-manager.lab.local/v1/tasks/<task-id> -H
"Authorization: Bearer <token>" -H "Content-Type: application/json" -d
'{"status":"CANCELLED"}'
```

NSX API endpoints:

Method	Endpoint	Purpose
GET	/api/v1/cluster/status	Cluster health status
GET	/api/v1/cluster	Cluster info with node UUIDs
GET	/api/v1/transport-nodes	List transport nodes
GET	/api/v1/transport-zones	List transport zones
GET	/api/v1/trust-management/certificates	List all certificates
POST	/api/v1/trust-management/certificates?action=import	Import certificate
POST	/api/v1/trust-management/certificates/<id>?action=apply_certificate	Apply certificate
GET	/api/v1/fabric/compute-managers	List compute managers
GET	/policy/api/v1/infra/segments	List segments (Policy API)
GET	/policy/api/v1/infra/tier-0s	List Tier-0 gateways
GET	/policy/api/v1/infra/tier-1s	List Tier-1 gateways

vCenter API endpoints:

Method	Endpoint	Purpose
POST	/api/session	Create session (Basic auth)
GET	/api/vcenter/vm	List all VMs
GET	/api/vcenter/host	List all hosts

GET	/api/vcenter/cluster	List all clusters
GET	/api/vcenter/datastore	List all datastores
GET	/api/vcenter/network	List all networks

Authentication patterns:

```
# SDDC Manager: Bearer token authentication
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens -H "Content-Type: application/json" -d '{"username":"admin@local","password":"Success01!0909!!"}' | python -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")
curl -k -H "Authorization: Bearer $TOKEN" https://sddc-manager.lab.local/v1/system

# NSX Manager: Basic authentication
curl -k -u admin:'Success01!0909!!' https://192.168.1.71/api/v1/cluster/status

# vCenter: Session-based authentication
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session -u 'administrator@vsphere.local:Success01!0909!!' | tr -d ' ')
curl -sk -H "vmware-api-session-id: $SESSION" https://vcenter.lab.local/api/vcenter/vm
```

API status codes:

Code	Meaning
200	Success
201	Created
202	Accepted (async operation started)
400	Bad Request (malformed JSON or invalid parameters)
401	Unauthorized (bad credentials or expired token)
403	Forbidden (insufficient permissions)
404	Not Found
409	Conflict (resource already exists)
500	Internal Server Error

PART IX: Disaster Recovery & Health Checks

9.1 Windows Update Crash — Incident Summary

On March 13, 2026, the Windows host running the nested VCF 9.0 lab environment was force-rebooted by Windows Updates. This caused an unclean shutdown of all nested VMs simultaneously, including all four ESXi hosts, vCenter, SDDC Manager, NSX Manager, and the VCF management components that were in the process of being deployed.

Impact Assessment

Impact Area	Description
All VMs	Powered off ungracefully
vSAN Cluster	Entered partitioned state — datastore inaccessible
NSX Manager	Services became unstable, crash loop
SDDC Manager	CPU soft lockups from resource contention
VCF Management Components	Deployment task interrupted mid-deploy at step 25 of 28
Fleet (vRSLCM)	CPU soft lockups
VCF Operations	Cluster stuck in <code>INITIALIZATION_FAILED</code> state

Recovery Duration: Approximately 48 hours across multiple troubleshooting sessions

Outcome: Full recovery achieved — all VCF components operational

Environment Reference (Post-Deployment)

Component	Hostname	IP Address	VM ID	vCPU	RAM
ESXi Host 1	esxi01.lab.local	192.168.1.201	—	8	48 GB
ESXi Host 2	esxi02.lab.local	192.168.1.202	—	8	48 GB
ESXi Host 3	esxi03.lab.local	192.168.1.203	—	8	48 GB

ESXi Host 4	esxi04.lab.local	192.168.1.204	—	8	48 GB
vCenter Server	vcenter.lab.local	192.168.1.69	vm-18	2	16 GB
SDDC Manager	sddc-manager.lab.local	192.168.1.241	vm-68	4	16 GB
NSX Manager	nsx-manager.lab.local	192.168.1.71	vm-58	6	30 GB
NSX VIP	nsx-vip.lab.local	192.168.1.70	—	—	—
Fleet (vRSLCM)	fleet.lab.local	192.168.1.78	vm-4014	4	12 GB
VCF Operations	vcf-ops.lab.local	192.168.1.77	vm-4015	8	32 GB
Collector	collector.lab.local	192.168.1.79	vm-4016	4	16 GB
Logs	—	—	vm-69	4	8 GB

Total nested VM resources: 32 vCPU, 130 GB RAM (management VMs only, excluding ESXi hosts)

Key IDs and References

Item	ID
SDDC Manager UUID	90ffb005-52c9-4d35-b254-0217f5305b59
Fleet Environment ID	df6d02bb-692a-4c44-a0d3-99e29c672bd0
Fleet Request ID	be0221fd-e620-48f3-8543-eb67b26616b0
Deployment Task ID	a48065d5-1ead-48ea-9d1e-113ae80732d2
VCF Ops Admin User ID	6df57f67-9573-47a8-a9d4-e9efa841a2ba
vCenter GUID	92109cf0-ad3b-4ffa-8972-a77bb7fadacf
NSX Cluster ID	6c55d856-ab96-4190-8495-3cc8cb23450c

9.2 Phase 1: vSAN Cluster Recovery

9.2.1 Symptoms & Diagnosis

After the Windows host rebooted, the vSAN datastore was inaccessible and the vSAN cluster showed a partitioned state:

- vSAN health showed "Cluster partition detected"
- VMs on the vSAN datastore were unresponsive
- `esxcli vsan cluster get` showed hosts in separate sub-clusters
- vmkping between hosts on the vSAN network (192.168.12.x) failed

Root Cause: The ungraceful shutdown caused the vSwitch failover policies for the vSAN portgroup to revert to using an incorrect NIC teaming configuration, preventing vSAN traffic between hosts.

Diagnosis steps on each host:

```
# Check vSAN cluster membership
esxcli vsan cluster get

# Test vSAN VMkernel connectivity from esxi01
vmkping -I vmk2 192.168.12.75
vmkping -I vmk2 192.168.12.76
vmkping -I vmk2 192.168.12.82

# Check vSwitch NIC teaming – look for "Unused Adapters"
esxcli network vswitch standard policy failover get --vswitch-name=vSwitch0
```

9.2.2 vSwitch Failover Policy Fix

The vSAN portgroup failover policy needed to be corrected on **all four ESXi hosts**:

```
# Fix the failover policy for the vSAN VMkernel portgroup
esxcfg-vswitch -p "vSAN" -N vmnic0 vSwitch0

# Verify fix
esxcli network vswitch standard policy failover get --vswitch-name=vSwitch0
# Should show: Active Adapters: vmnic3 (or appropriate NIC)
# Should show: Unused Adapters: (empty)
```

After correcting the failover policy on all hosts, vSAN traffic resumed and the cluster reformed.

9.2.3 vSAN Object Resync Verification

```
# Monitor vSAN resync progress
esxcli vsan debug resync summary get
```

```
# Verify cluster health
esxcli vsan cluster get
esxcli vsan health cluster list
```

Note: vSAN object resync took approximately 30-45 minutes after the cluster reformed. All objects returned to a compliant state.

9.3 Phase 2: NSX Manager Recovery

9.3.1 Service Restart Procedure

After vSAN recovery, NSX Manager was reachable but unstable — UI intermittently available, SDDC Manager reported NSX as "UNSTABLE", and services were in a crash loop.

Root Cause: The ungraceful shutdown corrupted some NSX service state. Services needed a clean restart.

```
# SSH to NSX Manager
ssh admin@192.168.1.71

# Check service status
get service

# Restart critical services
restart service manager
restart service proton
restart service corfu

# Wait 5-10 minutes, then verify
get cluster status
```

9.3.2 NSX Verification

```
# Verify via SDDC Manager API
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
```

```
-d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])"

curl -sk -H "Authorization: Bearer $TOKEN" \
https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool
```

Expected: "status": "ACTIVE"

9.4 Phase 3: SDDC Manager Recovery

9.4.1 CPU Soft Lockup Diagnosis

Symptom on VM console:

```
watchdog: BUG: soft lockup - CPU#0 stuck for 22s! [java:12345]
```

The SDDC Manager VM console showed a CPU soft lockup — the Java-based Spring Boot services consumed all available CPU, preventing the Linux kernel scheduler from running other processes.

Root Cause: Resource contention — with all management VMs running simultaneously (32 vCPU, 130 GB RAM in nested VMs), the physical host couldn't provide enough CPU time.

9.4.2 Hard Reset via vCenter API

SSH was unresponsive due to the soft lockup. The VM had to be hard-reset through the vCenter REST API:

```
# Get vCenter API session
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
-H "Authorization: Basic $(echo -n 'administrator@vsphere.local:Success01!0909!!' \
| base64)")

# Hard reset the SDDC Manager VM
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-68/power?
action=reset" \
-H "vmware-api-session-id: $SESSION"
```

Warning: Hard reset is destructive and should only be used when SSH and console are completely unresponsive due to

soft lockups. Always prefer graceful restart first.

9.4.3 Spring Boot Service Startup

After hard reset, SDDC Manager takes significantly longer to start under resource contention:

Service	Port	Normal Startup	Under Load (Nested)
domainmanager	7200 (HTTP)	2-3 min	~ 37 min
operationsmanager	7300	2-3 min	~ 30 min
lcm	7400	2-3 min	~ 25 min

```
# SSH to SDDC Manager (once responsive)
ssh vcf@192.168.1.241

# Check if domainmanager port is bound
ss -tlnp | grep 7200

# Check service status
systemctl status domainmanager
systemctl status operationsmanager

# Watch SDDC Manager API health
curl -sk https://localhost/v1/system/health
```

Critical Note: The `domainmanager` service uses **HTTP** on port 7200 (not HTTPS). Using `curl -sk https://localhost:7200` will fail with "wrong version number". Always use `http://localhost:7200` for direct service health checks.

9.4.4 Service Health Verification

```
# Verify all SDDC Manager services are running
systemctl list-units --type=service --state=running | grep -E
'domain|operations|lcm|common'

# Verify API is responsive
curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
```

```
-H "Content-Type: application/json" \
-d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json; t=json.load(sys.stdin); print('Token:',
t['accessToken'][:20]+'...')"
```

9.5 Phase 4: VCF Management Components Deployment Recovery

9.5.1 Deployment Task Status & Fleet CPU Soft Lockup

The VCF Management Components deployment (Fleet, VCF Operations, Collector) was interrupted at step 25 of 28 when the Windows crash occurred.

```
# Check management components status
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/vcf-management-components | python3 -m
json.tool
```

Task status showed:

- Steps 1-25: Successful
- Step 26: FAILED — "Retrieve the status for VCF Operations with VCF Operations collector Deployment request"
- Steps 27-28: Not attempted

Fleet (vm-4014) also experienced a CPU soft lockup and required a hard reset:

```
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4014/power?
action=reset" \
-H "vmware-api-session-id: $SESSION"
```

Fleet startup time: Port 8080 took approximately **48 minutes** to become available after hard reset.

9.5.2 Resource Contention Mitigation

With all management VMs running, the total resource demand caused severe contention. The solution was to temporarily power off non-essential VMs:

```
# Power off Collector VM (already crashed)
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# Power off Logs VM (not needed for recovery)
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-69/power?action=stop"
\
  -H "vmware-api-session-id: $SESSION"
```

Resources freed: 8 vCPU + 24 GB RAM

Lesson Learned: In nested environments with limited resources, prioritize which VMs need to run simultaneously. Power off non-essential VMs during recovery to prevent CPU soft lockups.

9.5.3 Fleet Database Investigation

After Fleet came back online, its API returned HTTP 500 errors for the deployment request. PostgreSQL investigation revealed the request had already completed:

```
ssh root@192.168.1.78
sudo -u postgres psql -d vr1cm

# Check the request status
SELECT id, state, requesttype, created, completed
FROM vm_rs_request
WHERE id = 'be0221fd-e620-48f3-8543-eb67b26616b0';
```

Result: The request was already in **COMPLETED** state — Fleet's crash recovery had processed it during its long startup.

9.5.4 SDDC Manager Task Retry

With Fleet reporting the request as completed, the SDDC Manager deployment task was retried:

```
curl -sk -X PATCH "https://sddc-manager.lab.local/v1/tasks/a48065d5-1ead-48ea-9d1e-
113ae80732d2" \
  -H "Authorization: Bearer $TOKEN" \
```

```
-H "Content-Type: application/json" \
-d '{"status":"IN_PROGRESS"}'
```

After approximately 60 seconds, the task progressed through steps 26, 27, and 28 — all successful. **Final status: 28/28 subtasks completed successfully.**

```
{
  "vcfOperationsFleetManagement": "SUCCEEDED",
  "vcfOperations": "SUCCEEDED",
  "vcfOperationsCollector": "SUCCEEDED"
}
```

9.6 Phase 5: VCF Operations Cluster Initialization

9.6.1 HSQLDB Reset Procedure

VCF Operations (vcf-ops.lab.local) was stuck in **INITIALIZATION_FAILED** state. The CASA API confirmed:

```
curl -sk https://192.168.1.77/casa/cluster/status
# Showed: "state": "INITIALIZATION_FAILED"
```

Root Cause: The unclean shutdown left the Gemfire distributed cache and HSQLDB in an inconsistent state.

Reset procedure:

```
# SSH to VCF Operations node
ssh root@192.168.1.77

# Stop services
systemctl stop vmware-casa
systemctl stop vmware-vcops-watchdog

# Backup HSQLDB
cp /storage/db/casa/webapp/hsqldb/casa.db.script \
  /storage/db/casa/webapp/hsqldb/casa.db.script.bak
```

```
# Edit HSQLDB – change initialization state
vi /storage/db/casa/webapp/hsqldb/casa.db.script
# Find: "initialization_state":"FAILED"
# Replace with: "initialization_state":"NONE"

# Clear HSQLDB log file
> /storage/db/casa/webapp/hsqldb/casa.db.log
```

9.6.2 Admin Password Reset

The admin password hash may have become invalid after the crash:

```
cat > /storage/vcops/user/conf/adminuser.properties << 'EOF'
#Properties for vCops user 'admin'
username=admin
hashed_password=
EOF
```

After cluster initialization, the system regenerates the password hash from the password configured during initial setup.

9.6.3 Cluster Initialization & Verification

```
# Get the SHA1 thumbprint of the local certificate
THUMBPRINT=$(openssl x509 -in /storage/vcops/user/conf/ssl/cert.pem -noout -
fingerprint -sha1 \
| sed 's/SHA1 Fingerprint=//')

# Restart services
systemctl start vmware-casa
systemctl start vmware-vcops-watchdog

# Wait for CASA to start, then trigger initialization
curl -sk -X POST https://localhost/casa/cluster/init \
-H "Content-Type: application/json"

# Verify cluster status
curl -sk https://localhost/casa/cluster/status
# Expected: "cluster_state": "INITIALIZED"
```

```
# Verify slice is online
curl -sk https://localhost/casa/sysadmin/slice/online_state
# Expected: "onlineState":"ONLINE"
```

9.7 Phase 6: VCF Operations Admin Roles & Adapter Configuration

9.7.1 Admin Role Fix via Suite-API

After cluster initialization, both users showed empty roles. Investigation revealed:

- The `admin` user is a **super admin** with implicit full access — `roleNames: []` is by design
- The `administrator@vsphere.local` user needed the Administrator role explicitly assigned

```
# Get authentication token
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# Assign Administrator role (CRITICAL: single object, NOT array)
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/auth/users/<user-id>/permissions" \
-H "Authorization: vRealizeOpsToken $TOKEN" \
-H "Content-Type: application/json" \
-H "Accept: application/json" \
-d '{
  "roleName": "Administrator",
  "allowAllObjects": true,
  "traversal-spec-instances": []
}'
```

Critical: The request body must be a **single JSON object** with `roleName`. Using `{"permissions": [{"roleName": "Administrator"}]}` will fail with "Role with name: null cannot be found".

Critical: VCF Operations Suite-API uses the auth header format `vRealizeOpsToken <token>` — NOT `Bearer` .

User	roleNames	Actual Access	Notes
admin	[] (empty)	Full admin	Built-in super admin — implicit access by design
administrator@vsphere.local	["Administrator"]	Full admin	Explicitly assigned via permissions API

9.7.2 Collector Recovery & Adapter Restart

The Collector VM (vm-4016) was powered off during resource contention mitigation. After other components stabilized:

```
# Power on collector
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?
action=start" \
-H "vmware-api-session-id: $SESSION"
```

Collector startup observations:

Phase	Duration
Boot to SSH responsive	~4 minutes
Load average during startup	15.14 on 4 vCPUs
Load stabilization	~30 minutes
CASA service fully initialized	~30 minutes

After the collector came online, adapters showed `COLLECTOR_DOWN` status. They needed stop/start cycles:

```
# For each adapter assigned to the collector (collectorId=2):
# Stop the adapter
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/stop" \
  -H "Authorization: vRealizeOpsToken $TOKEN"
```

```
# Start the adapter
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start" \
  -H "Authorization: vRealizeOpsToken $TOKEN"
```

Important: After stopping and starting an adapter, wait for the collector to actually be responsive. Starting adapters while the collector JVM is still initializing will leave them in a **STOPPED** state.

9.7.3 NSX Adapter Manual Creation

The NSX adapter was never auto-created because the VCF adapter's initial auto-discovery had already run before the crash. Manual creation was required.

Step 1: Create NSX credential:

```
curl -sk -X POST "https://192.168.1.77/suite-api/api/credentials" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "nsx-vip.lab.local",
    "adapterKindKey": "NSXTAdapter",
    "credentialKindKey": "NSXTCREDENTIAL",
    "fields": [
      {"name": "USERNAME", "value": "admin"},
      {"name": "PASSWORD", "value": "Success01!0909!!"}
    ]
  }'
```

Note: The credential field names are **USERNAME** and **PASSWORD** (uppercase). Using **USER** will fail with "USERNAME is mandatory".

Step 2: Create NSX adapter instance:

```
curl -sk -X POST "https://192.168.1.77/suite-api/api/adapters" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
```

```
-d '{
  "name": "nsx-vip.lab.local",
  "description": "NSX Manager",
  "adapterKindKey": "NSXTAdapter",
  "resourceIdentifiers": [
    {"name": "NSXTHOST", "value": "nsx-vip.lab.local"},
    {"name": "AUTO_DISCOVERY", "value": "true"},
    {"name": "ENABLE_ALERTS_FROM_NSX", "value": "false"},
    {"name": "VCURL", "value": "vcenter.lab.local"},
    {"name": "VMEntityVCID", "value": "<vcenter-guid>"},
    {"name": "NSX_CLUSTER_ID", "value": "<nsx-cluster-id>"}
  ],
  "credential": {"id": "<credential-id>"},
  "collectorId": 2
}'
```

Step 3: Start the adapter and verify (within 60 seconds):

```
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start" \
  -H "Authorization: vRealizeOpsToken $TOKEN"

# Verify
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>"
# Expected: numberOfResourcesCollected > 0
```

9.7.4 Final Adapter Status

Adapter	Status	Health	Resources
vcenter (VMWARE)	DATA_RECEIVING	GREEN	33
nsx-vip.lab.local (NSXTAdapter)	DATA_RECEIVING	GREEN	1+
lab (VcfAdapter)	DATA_RECEIVING	ORANGE	2
Container	DATA_RECEIVING	GREEN	43
VCF Operations API (vcf-ops)	DATA_RECEIVING	GREEN	1
VCF Operations Adapter (vcf-ops)	DATA_RECEIVING	GREEN	13

VCF Operations Adapter (collector)	DATA_RECEIVING	GREEN	7
Infrastructure Health (vcf-ops)	DATA_RECEIVING	GREEN	59
Infrastructure Health (collector)	DATA_RECEIVING	GREEN	3
Infrastructure Management (vcf-ops)	DATA_RECEIVING	GREEN	5
Infrastructure Management (collector)	DATA_RECEIVING	GREEN	7
Configuration Management (collector)	DATA_RECEIVING	GREEN	0
Diagnostics (vcf-ops)	DATA_RECEIVING	GREEN	7
Diagnostics (collector)	DATA_RECEIVING	GREEN	2
Application Monitoring (collector)	DATA_RECEIVING	GREEN	1
Log Assist (collector)	ERROR	ORANGE	1

Note: Log Assist adapter shows ERROR because the Logs VM was powered off. This resolves when the Logs VM is powered back on.

9.8 VCF Environment Health Check Procedure

This section provides a comprehensive, **reusable** health check procedure that can be applied to any VCF environment. Each subsection covers a specific component with the exact commands and expected outputs.

See also: The standalone document **VCF-Environment-Health-Check.md** provides this same procedure as a portable runbook.

9.8.1 Pre-Check: Physical/Virtual Host

Before checking VCF components, verify the underlying platform:

```
# For VMware Workstation nested labs – check host resources
# (run on the Windows host)
systeminfo | findstr /C:"Total Physical Memory" /C:"Available Physical Memory"
wmic cpu get NumberOfCores,NumberOfLogicalProcessors

# For bare metal – check IPMI/iLO/iDRAC for hardware alerts
```

```
# For ESXi standalone – check hardware status
esxcli hardware platform get
esxcli system version get
```

9.8.2 ESXi Host Health

Run on each ESXi host via SSH:

```
# 1. Basic host info
esxcli system version get
esxcli system hostname get

# 2. Uptime and boot time
esxcli system stats uptime get

# 3. CPU and memory
esxcli hardware cpu global get
esxcli hardware memory get

# 4. NIC status – all NICs should show "Link Status: Up"
esxcli network nic list

# 5. VMkernel interfaces – verify IPs on management, vMotion, vSAN
esxcli network ip interface ipv4 list

# 6. vSwitch health – verify uplinks are assigned
esxcli network vswitch standard list

# 7. Failover policy – ensure no "Unused Adapters"
esxcli network vswitch standard policy failover get -v vSwitch0

# 8. Routing table – verify routes for all subnets
esxcli network ip route ipv4 list

# 9. Services
esxcli system settings advanced list -o /UserVars/SuppressShellWarning
```

Expected healthy state:

- All NICs: Link Status = Up
- vmk0 (mgmt), vmk1 (vMotion), vmk2 (vSAN) all have correct IPs
- vSwitch uplinks: Active Adapters populated, Unused Adapters empty

- Routes exist for all three subnets

9.8.3 vSAN Health

Run on any ESXi host in the cluster:

```
# 1. Cluster membership – all hosts should be in one sub-cluster
esxcli vsan cluster get

# Key: Sub-Cluster Member Count should equal total host count
# Key: Local Node Health State should be HEALTHY

# 2. Cluster health
esxcli vsan health cluster list

# 3. Unicast agents – should list all cluster members
esxcli vsan cluster unicastagent list

# 4. Disk status
esxcli vsan storage list

# 5. vSAN network connectivity – ping other hosts from vmk2
vmkping -I vmk2 192.168.12.75 -c 3
vmkping -I vmk2 192.168.12.76 -c 3
vmkping -I vmk2 192.168.12.82 -c 3

# 6. Resync status (should show 0 resyncing objects)
esxcli vsan debug resync summary get

# 7. Object health
esxcli vsan debug object health summary get
```

Expected healthy state:

- Sub-Cluster Member Count = total host count (e.g., 4)
- All health checks pass (green)
- vmkping succeeds between all hosts
- 0 objects resyncing
- All objects compliant

9.8.4 vCenter Health

Via REST API from any machine with network access:

```
# 1. Get API session
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n 'administrator@vsphere.local:Success01!0909!!' \
  | base64)")

# 2. Check vCenter health status
curl -sk -H "vmware-api-session-id: $SESSION" \
  https://vcenter.lab.local/api/appliance/health/system

# 3. Check individual health components
for component in applmgmt database load mem softwarepackages storage swap; do
  echo -n "$component: "
  curl -sk -H "vmware-api-session-id: $SESSION" \
    "https://vcenter.lab.local/api/appliance/health/$component"
  echo
done

# 4. List all VMs and their power states
curl -sk -H "vmware-api-session-id: $SESSION" \
  https://vcenter.lab.local/api/vcenter/vm | python3 -m json.tool

# 5. Check services (SSH to vCenter appliance)
ssh root@vcenter.lab.local
vmon-cli --list
```

Expected healthy state:

- System health: "green"
- All health components: "green"
- All expected VMs present and in expected power states
- All vmon services running

9.8.5 NSX Manager Health**Via NSX CLI (SSH):**

```
ssh admin@nsx-vip.lab.local

# 1. Cluster status
get cluster status
```

```
# 2. Service status
get service

# 3. Interface status
get interface

# 4. Certificate status
get certificate api
```

Via NSX API:

```
# 1. Cluster status
curl -sk -u admin:'Success01!0909!!' https://nsx-
vip.lab.local/api/v1/cluster/status

# 2. Transport node status
curl -sk -u admin:'Success01!0909!!' https://nsx-vip.lab.local/api/v1/transport-
nodes/state

# 3. Alarms
curl -sk -u admin:'Success01!0909!!' https://nsx-vip.lab.local/api/v1/alarms
```

Expected healthy state:

- Cluster status: STABLE
- All services: running
- All transport nodes: SUCCESS state
- No critical alarms

9.8.6 SDDC Manager Health

Via REST API:

```
# 1. Get auth token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
-H "Content-Type: application/json" \
-d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# 2. System health
```

```

curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system | python3 -m json.tool

# 3. Component status
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool

curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/vcenters | python3 -m json.tool

# 4. Check for stuck tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" | python3 -m
json.tool

# 5. Check for resource locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -m json.tool

# 6. VCF Management Components
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/vcf-management-components | python3 -m
json.tool

```

Via SSH:

```

ssh vcf@sddc-manager.lab.local

# Service status
systemctl list-units --type=service --state=running | grep -E
'domain|operations|lcm|common'

# Check ports
ss -tlnp | grep -E '7200|7300|7400|443'

# Check disk space
df -h

```

Expected healthy state:

- All components: ACTIVE status
- No stuck tasks (IN_PROGRESS)
- No stale resource locks

- All 4 services running (domainmanager, operationsmanager, lcm, commonsvcs)
- Management components: all SUCCEEDED

9.8.7 VCF Operations Health

Via Suite-API:

```
# 1. Get token
TOKEN=$(curl -sk -X POST https://vcf-ops.lab.local/suite-api/api/auth/token/acquire \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"Success01!0909!!","authSource":"local"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# 2. Node status
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
https://vcf-ops.lab.local/suite-api/api/deployment/node/status | python3 -m json.tool

# 3. Collector status
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
https://vcf-ops.lab.local/suite-api/api/collectors | python3 -m json.tool

# 4. Adapter status – check all adapters for health
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
https://vcf-ops.lab.local/suite-api/api/adapters | python3 -m json.tool

# 5. Cluster status (CASA API, from localhost)
ssh root@vcf-ops.lab.local
curl -sk https://localhost/casa/cluster/status
curl -sk https://localhost/casa/sysadmin/slice/online_state
```

Expected healthy state:

- Node status: ONLINE
- All collectors: UP
- All adapters: DATA_RECEIVING with GREEN health
- Cluster: INITIALIZED
- Slice: ONLINE

9.8.8 Fleet (vRSLCM) Health

Via API:

```
# 1. Authentication
FLEET_TOKEN=$(curl -sk -X POST https://fleet.lab.local:8080/lcm/authzn/api/login \
-H "Content-Type: application/json" \
-d '{"username":"admin@local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# 2. Environment status
curl -sk -H "Authorization: Bearer $FLEET_TOKEN" \
https://fleet.lab.local:8080/lcm/lcops/api/v2/environments | python3 -m json.tool

# 3. Health check
curl -sk -H "Authorization: Bearer $FLEET_TOKEN" \
https://fleet.lab.local:8080/lcm/health | python3 -m json.tool
```

Via SSH:

```
ssh root@fleet.lab.local

# Service status
systemctl status nginx
systemctl status vmware-lcm

# Database status
sudo -u postgres pg_isready

# Port check
ss -tlnp | grep 8080
```

Expected healthy state:

- Environments: all COMPLETED
- Health: UP
- nginx and vmware-lcm services: running
- PostgreSQL: accepting connections
- Port 8080: listening

9.8.9 Complete Health Check Script

A ready-to-use Python script that checks all components in one pass:

```
#!/bin/bash
# VCF Environment Health Check Script
# Usage: bash vcf-health-check.sh
# Prerequisites: curl, python3, SSH access to all components

VCENTER="vcenter.lab.local"
SDDC="sddc-manager.lab.local"
NSX_VIP="nsx-vip.lab.local"
VCF_OPS="vcf-ops.lab.local"
FLEET="fleet.lab.local"
USER="administrator@vsphere.local"
PASS="Success01!0909!!"
ADMIN_PASS="Success01!0909!!" # VCF Ops admin password

echo "=====
echo "VCF Environment Health Check"
echo "Date: $(date)"
echo "=====

# 1. vCenter Health
echo -e "\n--- vCenter Health ---"
SESSION=$(curl -sk -X POST "https://$VCENTER/api/session" \
  -H "Authorization: Basic $(echo -n "$USER:$PASS" | base64)" 2>/dev/null | tr -d
  '')
if [ -n "$SESSION" ] && [ "$SESSION" != "null" ]; then
  HEALTH=$(curl -sk -H "vmware-api-session-id: $SESSION" \
    "https://$VCENTER/api/appliance/health/system" 2>/dev/null | tr -d '')
  echo "vCenter System Health: $HEALTH"
else
  echo "vCenter: UNREACHABLE"
fi

# 2. SDDC Manager Health
echo -e "\n--- SDDC Manager Health ---"
TOKEN=$(curl -sk -X POST "https://$SDDC/v1/tokens" \
  -H "Content-Type: application/json" \
  -d '{"username\":\"$USER\",\"password\":\"$PASS\"}' 2>/dev/null \
  | python3 -c "import
sys,json;print(json.load(sys.stdin).get('accessToken','FAILED'))" 2>/dev/null)
if [ "$TOKEN" != "FAILED" ] && [ -n "$TOKEN" ]; then
  echo "SDDC Manager API: HEALTHY (token acquired)"
  # Check components
  curl -sk -H "Authorization: Bearer $TOKEN" \
```

```

    "https://$SDDC/v1/vcf-management-components" 2>/dev/null \
    | python3 -c "
import sys,json
d=json.load(sys.stdin)
for k,v in d.items():
    if isinstance(v,str): print(f' {k}: {v}')
" 2>/dev/null
else
    echo "SDDC Manager: UNREACHABLE"
fi

# 3. NSX Health
echo -e "\n--- NSX Manager Health ---"
NSX_STATUS=$(curl -sk -u "admin:$PASS" \
    "https://$NSX_VIP/api/v1/cluster/status" 2>/dev/null \
    | python3 -c "import
sys,json;d=json.load(sys.stdin);print(d.get('control_cluster_status',
{}).get('status','UNKNOWN'))" 2>/dev/null)
echo "NSX Cluster Status: $NSX_STATUS"

# 4. VCF Operations Health
echo -e "\n--- VCF Operations Health ---"
OPS_TOKEN=$(curl -sk -X POST "https://$VCF_OPS/suite-api/api/auth/token/acquire" \
    -H "Content-Type: application/json" \
    -d "
{\"username\":\"admin\", \"password\":\"$ADMIN_PASS\", \"authSource\":\"local\"}"
2>/dev/null \
    | python3 -c "import sys,json;print(json.load(sys.stdin).get('token','FAILED'))"
2>/dev/null)
if [ "$OPS_TOKEN" != "FAILED" ] && [ -n "$OPS_TOKEN" ]; then
    echo "VCF Operations API: HEALTHY (token acquired)"
    # Check adapters
    curl -sk -H "Authorization: vRealizeOpsToken $OPS_TOKEN" \
        "https://$VCF_OPS/suite-api/api/adapters" 2>/dev/null \
        | python3 -c "
import sys,json
d=json.load(sys.stdin)
adapters=d.get('adapterInstancesInfoDto',[])
green=sum(1 for a in adapters if a.get('resourceKey',
{}).get('adapterKindKey','')!='')
print(f' Total Adapters: {len(adapters)}')
for a in adapters:
    name=a.get('resourceKey',{}).get('name','?')
    cs=a.get('adapter-status',{}).get('adapterStatus','?')
    print(f' {name}: {cs}')
" 2>/dev/null

```

```

else
    echo "VCF Operations: UNREACHABLE"
fi

# 5. Fleet Health
echo -e "\n--- Fleet (vRSLCM) Health ---"
FLEET_TOKEN=$(curl -sk -X POST "https://$FLEET:8080/lcm/authzn/api/login" \
    -H "Content-Type: application/json" \
    -d "{\"username\":\"admin@local\", \"password\":\"$PASS\"}" 2>/dev/null \
    | python3 -c "import sys,json;print(json.load(sys.stdin).get('token','FAILED'))"
2>/dev/null)
if [ "$FLEET_TOKEN" != "FAILED" ] && [ -n "$FLEET_TOKEN" ]; then
    echo "Fleet API: HEALTHY (token acquired)"
else
    echo "Fleet: UNREACHABLE"
fi

echo -e "\n=====
echo "Health Check Complete"
echo "=====

```

Customization: Replace the hostname/IP variables at the top of the script with values for your environment.

9.9 Key Learnings & Common Pitfalls

9.9.1 Service Startup Times Under Load

In a nested lab environment with resource contention, Java-based services take significantly longer to start:

Service	Normal Startup	Under Load (Nested)	Port
SDDC Manager domainmanager	2-3 min	37 min	7200 (HTTP)
SDDC Manager operationsmanager	2-3 min	30 min	7300
Fleet LCM backend	3-5 min	48 min	8080
VCF Operations CASA	2-3 min	10-15 min	443
VCF Operations Collector CASA	2-3 min	5-10 min	443

NSX Manager services	3-5 min	10-15 min	443
----------------------	---------	------------------	-----

Rule of Thumb: In nested environments, expect startup times to be **5-10x longer** than normal. Do not assume a service has failed — check CPU load and be patient.

9.9.2 API Gotchas Reference

Pitfall	Wrong	Correct
VCF Ops auth header	<code>Authorization: Bearer <token></code>	<code>Authorization: vRealizeOpsToken <token></code>
SDDC Manager internal port	<code>https://localhost:7200</code>	<code>http://localhost:7200</code>
VCF Ops permissions body	<code>{"permissions": [{"roleName": "Admin"}]}</code>	<code>{"roleName": "Administrator", "allowAllObjects"}</code>
NSX credential field	<code>{"name": "USER", "value": "admin"}</code>	<code>{"name": "USERNAME", "value": "admin"}</code>
Bash <code>!</code> in passwords	<code>password="Success01!"</code>	Use heredoc or single quotes
Gemfire cache after init	Querying roles immediately	Wait 5-10 minutes for cache to populate

APPENDICES

Appendix A: Environment Quick Reference

A.1.1 Complete IP Address Table

Component	IP Address	FQDN	Role
DNS / AD Server	192.168.1.230	dc.lab.local	DNS, NTP, Active Directory (lab.local)
vCenter Server	192.168.1.69	vcenter.lab.local	vSphere management
NSX VIP	192.168.1.70	nsx-vip.lab.local	NSX Manager cluster VIP
NSX Node 1	192.168.1.71	nsx-node1.lab.local	NSX Manager node
ESXi Host 1	192.168.1.74	esxi01.lab.local	Compute host
ESXi Host 2	192.168.1.75	esxi02.lab.local	Compute host
ESXi Host 3	192.168.1.76	esxi03.lab.local	Compute host
VCF Operations	192.168.1.77	vcf-ops.lab.local	Monitoring / Fleet Management UI
Fleet (Cloud Proxy)	192.168.1.78	fleet.lab.local	VCF Operations data collector
Collector	192.168.1.79	collector.lab.local	Operations Collector
ESXi Host 4	192.168.1.82	esxi04.lab.local	Compute host
Automation	192.168.1.90	automation.lab.local	VCF Automation (if deployed)
Aria Lifecycle	192.168.1.94	aria-lifecycle.lab.local	Lifecycle Manager
SDDC Manager	192.168.1.241	sddc-manager.lab.local	VCF orchestration and lifecycle
NSX Manager (SDDC registered)	192.168.1.70	nsx-manager.lab.local	FQDN used by SDDC Manager for NSX

A.1.2 DNS Records (Forward and Reverse)

Forward (A) records required in lab.local zone:

vcenter	A	192.168.1.69
nsx-vip	A	192.168.1.70
nsx-node1	A	192.168.1.71
nsx-manager	A	192.168.1.70
esxi01	A	192.168.1.74
esxi02	A	192.168.1.75
esxi03	A	192.168.1.76
vcf-ops	A	192.168.1.77
fleet	A	192.168.1.78
collector	A	192.168.1.79
esxi04	A	192.168.1.82
automation	A	192.168.1.90
aria-lifecycle	A	192.168.1.94
sddc-manager	A	192.168.1.241

Reverse (PTR) records required in 1.168.192.in-addr.arpa zone:

69	PTR	vcenter.lab.local.
70	PTR	nsx-vip.lab.local.
71	PTR	nsx-node1.lab.local.
74	PTR	esxi01.lab.local.
75	PTR	esxi02.lab.local.
76	PTR	esxi03.lab.local.
77	PTR	vcf-ops.lab.local.
78	PTR	fleet.lab.local.
82	PTR	esxi04.lab.local.
241	PTR	sddc-manager.lab.local.

Entries NOT needed for Simple Mode: nsx-node2, nsx-node3, vcf-ops-rep, vcf-ops-data, vcf-ops-lb, automation-node1/2/3/4, automation-upgrade.

A.1.3 Default Credentials Reference

Component	Username	Password / Notes
ESXi Hosts	root	Set during installation
vCenter SSO	administrator@vsphere.local	Set during deployment

SDDC Manager UI	admin@local	Set during deployment
SDDC Manager SSH	vcf	Only user that can SSH; root via <code>su -</code>
NSX Manager admin	admin	Set during OVA deployment
NSX Manager audit	audit	Set during OVA deployment
NSX Manager root	root	Set during OVA deployment
VCF Operations	admin	Set during OVA deployment
Java Keystore	N/A	<code>changeit</code>
VCF Trust Store	N/A	Contents of <code>/etc/vmware/vcf/commonsvcs/trusted_certificates.key</code>
Cloud Builder SSH	root	<code>vmware</code> (default)

A.1.4 VM Resource Specifications

VM	vCPU	RAM	Storage (Actual)	Deployed By
ESXi Host (x4)	32	48 GB	~400 GB each (local)	VMware Workstation
NSX Manager	6	32 GB	vSAN (thin)	Manual (ovftool)
vCenter Server	4	19 GB	vSAN	VCF Installer
SDDC Manager	4	16 GB	vSAN (thin, ~108 GB used)	VCF Installer bringup
VCF Operations	2	8 GB	vSAN (thin)	Manual (ovftool)
Fleet (Cloud Proxy)	2	4 GB	vSAN (thin)	VCF Operations Lifecycle

Physical host: Dell Precision 7920, 35-core CPU, 192 GB RAM, D: 2TB SSD, E: 2TB SSD, 2x 4TB HDD.

A.1.5 VMkernel Layout

VMkernel	Subnet	TCP/IP Stack	Purpose
vmk0	192.168.1.0/24	defaultTcpipStack	Management + NSX TEP (overlay)
vmk1	192.168.11.0/24	vmotion	vMotion
vmk2	192.168.12.0/24	defaultTcpipStack	vSAN
vmk50	169.254.0.0/16	hyperbus	NSX Hyperbus (internal, auto-created)

Per-host VMkernel IP addresses:

Host	vmk0 (Mgmt/TEP)	vmk1 (vMotion)	vmk2 (vSAN)
esxi01	192.168.1.74	192.168.11.121	192.168.12.121
esxi02	192.168.1.75	192.168.11.120	192.168.12.120
esxi03	192.168.1.76	192.168.11.122	192.168.12.122
esxi04	192.168.1.82	192.168.11.123	192.168.12.123

Appendix B: Port Reference

B.1 Management Ports

Port	Protocol	Source	Destination	Description
22	TCP	Admin workstation	ESXi, vCenter, SDDC Mgr, NSX	SSH access
53	TCP/UDP	All components	DNS server	DNS resolution
80	TCP	Browsers	vCenter	HTTP redirect to HTTPS
123	UDP	All components	NTP server	Time synchronization
443	TCP	Browsers, SDDC Mgr	vCenter, NSX, ESXi, SDDC Mgr	HTTPS management UI and API
902	TCP	vCenter	ESXi hosts	VMware authentication / NFC
5480	TCP	Admin workstation	vCenter	VAMI (appliance management)
5432	TCP	SDDC Mgr (internal)	PostgreSQL	Database connectivity

B.2 vSAN Ports

Port	Protocol	Source	Destination	Description
2233	TCP	ESXi hosts	ESXi hosts	vSAN transport
12345-23451	UDP	ESXi hosts	ESXi hosts	vSAN cluster service (CMMDS, RDT)

B.3 NSX Ports

Port	Protocol	Source	Destination	Description
443	TCP	Admin, SDDC Mgr	NSX Manager	NSX UI and API
1234	TCP	ESXi hosts	NSX Manager	NSX agent to manager communication
1235	TCP	NSX Manager	NSX Manager	NSX cluster inter-node
6081	UDP	ESXi hosts	ESXi hosts	GENEVE overlay encapsulation
8080	TCP	NSX Manager	NSX Manager	Internal cluster HTTP

B.4 VCF Operations Ports

Port	Protocol	Source	Destination	Description
443	TCP	Browsers	VCF Operations	Operations UI and API
443	TCP	Cloud Proxy	VCF Operations	Fleet management data

B.5 SDDC Manager Ports

Port	Protocol	Source	Destination	Description
443	TCP	Browsers, VCF Ops	SDDC Manager	SDDC Manager UI and API
22	TCP	Admin workstation	SDDC Manager	SSH (vcf user only)
5432	TCP	Internal	SDDC Manager	PostgreSQL database

B.6 vMotion and Other Ports

Port	Protocol	Source	Destination	Description
------	----------	--------	-------------	-------------

8000	TCP	ESXi hosts	ESXi hosts	vMotion traffic
8443	TCP	SDDC Manager	Offline depot	Custom HTTPS offline depot
111	TCP	ESXi hosts	NFS server	NFS portmapper
2049	TCP	ESXi hosts	NFS server	NFS file system

Appendix C: Log File Matrix

C.1 Log Files by Component

SDDC Manager logs:

Log Path	Description
<code>/var/log/vmware/vcf/domainmanager/domainmanager.log</code>	Domain Manager main log (deployments, tasks, domain operations)
<code>/var/log/vmware/vcf/domainmanager/domainmanager-gc.log</code>	Domain Manager garbage collection log
<code>/var/log/vmware/vcf/lcm/lcm.log</code>	Lifecycle Manager log (upgrades, patching, bundles)
<code>/var/log/vmware/vcf/lcm/upgrade/</code>	Upgrade-specific logs directory
<code>/var/log/vmware/vcf/operationsmanager/operationsmanager.log</code>	Operations Manager log
<code>/var/log/vmware/vcf/operationsmanager/operationsmanager-gc.log</code>	Operations Manager GC log
<code>/var/log/vmware/vcf/sos/sos.log</code>	SoS utility log
<code>/var/log/vmware/vcf/commonsvcs/commonsvcs.log</code>	Common services log (certificates, trust store)
<code>/var/log/vmware/vcf/sddc-support/sddc-support.log</code>	Support bundle collection log
<code>/var/log/vmware/vcf/vdt/vdt-<timestamp>.txt</code>	VCF Diagnostic Tool results
<code>/var/log/nginx/error.log</code>	NGINX reverse proxy error log
<code>/var/log/nginx/access.log</code>	NGINX access log

<code>/var/log/postgresql/postgresql-*.log</code>	PostgreSQL database logs
---	--------------------------

vCenter Server logs:

Log Path	Description
<code>/var/log/vmware/vpxd/vpxd.log</code>	Main vCenter Server daemon log
<code>/var/log/vmware/vsphere-client/logs/vsphere_client_virgo.log</code>	vSphere Client (legacy) log
<code>/var/log/vmware/vsphere-ui/logs/vsphere_client_virgo.log</code>	vSphere UI log
<code>/var/log/vmware/vpostgres/postgresql*.log</code>	vCenter PostgreSQL database logs
<code>/var/log/vmware/sso/vmware-sts-idmd.log</code>	SSO / Lookup service log
<code>/var/log/vmware/eam/eam.log</code>	ESX Agent Manager log
<code>/var/log/vmware/content-library/cls.log</code>	Content Library service log
<code>/var/log/vmware/vlcm/vlcm.log</code>	vSphere Lifecycle Manager log

ESXi host logs:

Log Path	Description
<code>/var/log/vmkernel.log</code>	VMkernel log (storage, network, hardware events)
<code>/var/log/hostd.log</code>	Host daemon log (management operations, VM power)
<code>/var/log/vpxa.log</code>	vCenter agent log (host-to-vCenter communication)
<code>/var/log/nsx-syslog.log</code>	NSX agent log on ESXi hosts
<code>/var/log/fdm.log</code>	Fault Domain Manager (HA) log
<code>/var/log/vobd.log</code>	VMkernel Observation log (events, alarms)
<code>/var/log/esxupdate.log</code>	ESXi patching and update log
<code>/var/log/vmkwarning.log</code>	VMkernel warning messages
<code>/var/log/shell.log</code>	ESXi shell command history
<code>/var/log/auth.log</code>	Authentication and SSH log

NSX Manager logs:

Log Path	Description
/var/log/proton/nsxapi.log	NSX API service log
/var/log/proton/nsx-management-plane.log	NSX management plane log
/var/log/corfu/corfu.log	Corfu distributed database log
/var/log/syslog	General system log
/config/cluster-manager/	Cluster manager configuration and certificates

VCF Operations logs:

Log Path	Description
/storage/log/vcops/	VCF Operations main log directory
/storage/log/vcops/web/	Web UI logs
/storage/log/vcops/analytics/	Analytics engine logs

C.2 Log Files by Issue Type

Issue Category	Primary Logs to Check	Secondary Logs
VCF Task Failures	domainmanager.log , lcm.log	operationsmanager.log
Deployment Issues	domainmanager.log , lcm.log	commonsvcs.log
vCenter Connectivity	vpxd.log , vpxa.log	hostd.log
VM Power Issues	hostd.log	vpxd.log , vmkernel.log
Network / Connectivity	vmkernel.log , nsx-syslog.log	vpxa.log
vSAN Storage	vmkernel.log (grep vsan)	hostd.log
Certificate Errors	commonsvcs.log	vpxd.log , domainmanager.log
Authentication / SSO	vmware-sts-idmd.log	vpxd.log
NSX Transport Nodes	nsx-syslog.log , nsxapi.log	vmkernel.log
Bundle Download / LCM	lcm.log	nginx/error.log

Database Issues	postgresql-*.log	domainmanager.log
VCF Diagnostic Tool	/var/log/vmware/vcf/vdt/vdt- <timestamp>.txt	N/A

Log analysis commands:

```
# Real-time log monitoring
tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log

# Search for errors in a log file
grep -i error /var/log/vmware/vcf/domainmanager/domainmanager.log | tail -50

# Search for exceptions
grep -i exception /var/log/vmware/vcf/lcm/lcm.log | tail -20

# Filter by date
grep "2026-02-12" /var/log/vmware/vcf/domainmanager/domainmanager.log

# Search compressed/rotated logs
zgrep -i error /var/log/vmware/vcf/domainmanager/domainmanager.log.gz

# Search for specific task ID
grep "<task-id>" /var/log/vmware/vcf/lcm/lcm.log

# View systemd journal for a service
journalctl -u vcf-services -f

# View journal errors from last hour
journalctl -u vcf-services --since "1 hour ago" -p err
```

Appendix D: Glossary and Acronyms

Term	Definition
ABX	Action-Based Extensibility -- custom actions triggered by events in VCF Automation
BOM	Bill of Materials -- component version and build number list for a VCF release
CMMDS	Cluster Monitoring, Membership, and Directory Service (vSAN internal)
CNI	Container Network Interface -- Kubernetes networking plugin (Antrea is default for VKS)

CSI	Container Storage Interface -- allows storage providers to expose persistent volumes to Kubernetes
DFW	Distributed Firewall -- NSX micro-segmentation applied at the VM vNIC level
DRS	Distributed Resource Scheduler -- automatic VM placement and load balancing across hosts
ESA	Express Storage Architecture -- vSAN single-tier NVMe-only storage (VCF 9.0+)
EVC	Enhanced vMotion Compatibility -- CPU feature masking for mixed-generation clusters
FIPS	Federal Information Processing Standards -- cryptographic compliance mode (mandatory in VCF 9.0)
FTT	Failures to Tolerate -- vSAN data protection level (1, 2, or 3 failures)
GENEVE	Generic Network Virtualization Encapsulation -- NSX overlay tunnel protocol (~54 bytes overhead)
HA	High Availability -- automatic VM restart on host failure
HCL	Hardware Compatibility List -- VMware-certified hardware for vSAN and ESXi
LCM	Lifecycle Management -- patching, upgrading, and maintaining VCF components
NSX	VMware's software-defined networking and security platform
NTP	Network Time Protocol -- time synchronization (critical for VCF certificate and cluster operations)
OSA	Original Storage Architecture -- vSAN with cache+capacity disk groups
OVA	Open Virtual Appliance -- packaged VM template for deployment
PEM	Privacy Enhanced Mail -- Base64-encoded certificate format used by all VCF components
PSC	Platform Services Controller -- SSO and certificate authority (embedded in vCenter 9.0)
SAN	Subject Alternative Name -- certificate field listing valid hostnames and IPs
SDDC	Software-Defined Data Center -- the complete VCF infrastructure stack
SOS	Supportability and Serviceability -- SDDC Manager diagnostic and log bundle utility
TEP	Tunnel Endpoint -- overlay network encapsulation point on each ESXi host (uses GENEVE)
TKG	Tanzu Kubernetes Grid -- VMware's Kubernetes distribution for vSphere
VCF	VMware Cloud Foundation -- unified private cloud platform
VDS	vSphere Distributed Switch -- centrally managed virtual switch across multiple hosts
VDT	VCF Diagnostic Tool -- read-only Python health check tool (download from Broadcom KB 344917)
VIB	vSphere Installation Bundle -- ESXi software package format

vLCM	vSphere Lifecycle Manager -- ESXi image-based lifecycle management (replaces baselines in 9.0)
VPC	Virtual Private Cloud -- isolated network environment in VCF Automation
VTEP	Virtual Tunnel Endpoint -- same as TEP; virtual interface for overlay encapsulation
VKS	VMware Kubernetes Service -- managed Kubernetes clusters on VCF

Appendix E: Key Lessons Learned

E.1 Nested Environment Gotchas

NSX Manager sizing for nested environments:

- 16 GB RAM: Kernel OOM, constant crashes, unusable
- 24 GB RAM: Boots and runs basic operations, but MANAGER/SEARCH services crash under load (transport node configuration fails)
- 32 GB RAM / 6 vCPU: Required for stable operation with a 4-host cluster

SDDC Manager deployment timeout loop: Manual ovftool deployment bypasses SDDC Manager's timeout thresholds that are not suited for nested environments. SDDC Manager will delete and retry timed-out deployments in an infinite loop.

vhv.enable ghost setting: The `vhv.enable` setting can persist in a VM's runtime state (vmware.log DICT) even when it is not present in the VMX file. This causes vMotion to fail with "Configuration mismatch: snapshot was taken with VHV enabled." Fix by explicitly adding `vhv.enable = "FALSE"` to the VMX file.

Hot vMotion memory convergence: In nested environments, hot vMotion frequently fails because memory convergence cannot complete within the timeout. Use cold migration (power off, relocate, power on) as a reliable fallback.

NSX nested boot storm: After power-on, NSX Manager runs 12+ Java processes on 6 vCPUs, causing load averages of 30-100+ for 30-60 minutes. The VIP won't come online until load settles below ~20. Do NOT add more vCPUs — co-scheduling overhead makes it worse. Credential operations attempted during this window will fail and can trigger the cascade failure described in [Section 7.2.6](#).

vSAN network latency: Nested vSAN will always show yellow on network latency health checks (typically 5-7ms vs 5ms threshold). This is normal for virtualized NICs in VMware Workstation and does not affect functionality.

VMware Workstation VMX settings required:

```

vhv.enable = "TRUE"           # Nested virtualization
vpmc.enable = "TRUE"          # Virtual Performance Counters
vvttd.enable = "TRUE"         # Virtual Intel VT-d
ethernet0.noPromisc = "FALSE" # Allow nested VM traffic
sata0:0.virtualSSD = "1"      # Mark disks as SSD for vSAN

```

Windows host prerequisite: Must disable Hyper-V (`bcdedit /set hypervisorlaunchtype off`) and reboot before VMware Workstation can pass VT-x to nested ESXi VMs.

E.2 Component-Specific Lessons

SDDC Manager SSH access: Only the `vcf` user can SSH in (root and admin are rejected). Root access is via `su -` from a vcf session. SCP does not work due to the restricted shell; use `ssh vcf@host "cat > file" < localfile` for file transfers.

SDDC Manager vcf account lockout: Failed SSH attempts (including from automated scripts) lock the `vcf` account quickly. SDDC Manager uses `faillock` (not `pam_tally2`). Unlock from console as root: `faillock --user vcf --reset`. If ALL accounts are locked, boot into GRUB single-user mode with `init=/bin/bash`.

SDDC Manager PostgreSQL access: PostgreSQL uses TCP on `127.0.0.1` (not Unix sockets — always use `-h 127.0.0.1` with psql). Data directory is `/data/pgdata`. Password is not easily discoverable — use the temporary `trust` auth workaround in `pg_hba.conf` (always restore immediately after).

Always use `PAGER=cat` to prevent pager traps in remote sessions. Key databases: `platform` (nsxt, lock, task_metadata, task_lock tables), `operationsmanager` (task, execution, processing_task tables).

SDDC Manager credential rotation cascade failure: A failed credential rotation (e.g., NSX unreachable during boot storm) leaves the resource stuck in `ACTIVATING` or `ERROR` state in the `platform.nsxt` table, stale exclusive locks in `platform.lock`, and unresolved tasks piling up in `platform.task_metadata` (resolved=false). All future credential operations are blocked even after the target component recovers. The API cannot cancel stuck tasks (`TA_TASK_CAN_NOT_BE_RETRIED`). Fix: 6-step database repair — (1) enable trust auth, (2) fix nsxt status to ACTIVE, (3) delete stale locks, (4) mark task_metadata resolved=true + clear task_lock, (5) restore pg_hba.conf, (6) restart operationsmanager. See [Section 7.2.6](#).

NSX admin CLI: DNS and NTP are configured via `set name-servers` / `set ntp-servers` commands in the admin CLI, NOT through the NSX UI.

NSX shell limitations: No backslash line continuation is supported. All curl commands and other multi-argument commands must be written on a single line.

NSX certificate SAN requirements: The SAN must include `nsx-manager.lab.local` (the FQDN registered in SDDC Manager for NSX), not just `nsx-node1.lab.local`. Without it, VDT reports a SAN check failure.

NSX certificate trust stores: After replacing NSX self-signed certificates, import the new cert into both:

1. VCF trust store: `/etc/vmware/vcf/commonsvcs/trusted_certificates.store`
(password in `.key` file)
2. Java cacerts: `/etc/alternatives/jre/lib/security/cacerts` (password: `changeit`)

Then restart SDDC Manager services. Reference: Broadcom KB 316056.

vSAN thick-to-thin migration: vCenter's migration wizard cannot thin-provision to vSAN. Use `vmkfstools -i <src> <dst> -d thin` per disk to convert thick-provisioned VMDKs to thin.

VDT is not pre-installed: Must be downloaded from Broadcom KB 344917 and uploaded to SDDC Manager manually via the `ssh cat` method.

Aria Lifecycle OVF properties: Use `ovftool <ova>` to probe the OVA and discover correct property names. The property format is NOT always `vami.ip0.VCF_OPS_Management_Appliance` -- it varies by appliance version.

ovftool single-line commands: On VCF Installer / SDDC Manager, use single-line ovftool commands. Backslash continuation and `--noSSLVerify` can break depending on how commands are pasted.

E.3 Deployment Best Practices

NSX 9.0 TEP on vmk0: Use the "Use VMkernel Adapter" option in the Transport Node Profile IPv4 Assignment to reuse vmk0 for overlay traffic. This eliminates the need for a dedicated TEP VLAN in lab environments.

VCF 9.0.1 vSAN ESA HCL bypass: Add `vsan.esa.sddc.managed.disk.claim=true` to `/etc/vmware/vcf/domainmanager/application-prod.properties` and restart domainmanager before running the VCF Installer wizard.

NFS mount ownership: If VDT reports NFS mount ownership failure, fix with `chown root:vcf /nfs/vmware/vcf/nfs-mount/`. Reference: Broadcom KB 392923.

VCF Upgrade order (always follow this sequence):

1. SDDC Manager (first -- orchestrates all other upgrades)
2. vCenter Server
3. NSX Manager
4. ESXi Hosts (rolling upgrade)
5. vSAN
6. VCF Operations (if deployed)

E.4 VCF 9.0.1 Component Versions (BOM)

Component	Version	Build Number
vCenter Server	9.0.1.0	24957454
ESXi	9.0.1.0	24957456
NSX Manager	9.0.1.0	24952111
SDDC Manager	9.0.1.0	24962180
VCF Operations	9.0.1.0	24960351
Fleet Management	9.0.1.0	24960371
Automation	9.0.1.0	24965341
Operations Collector	9.0.1.0	24960349

E.5 Certificate File Locations

Component	Certificate Path	Key Path
ESXi SSL	/etc/vmware/ssl/rui.crt	/etc/vmware/ssl/rui.key
vCenter	/etc/vmware-vpx/ssl/	/etc/vmware-vpx/ssl/
SDDC Manager	/etc/vmware/vcf/commonsvcs/	/etc/vmware/vcf/commons
NSX Manager	/config/cluster-manager/	/config/cluster-manager
VCF Trust Store	/etc/vmware/vcf/commonsvcs/trusted_certificates.store	Password in trusted_certificates.ke

Java Cacerts	/etc/alternatives/jre/lib/security/cacerts	Password: changeit
--------------	--	--------------------

Appendix F: Diagnostic Scripts Quick Reference

20 Python diagnostic scripts for VCF 9.0.1 nested lab troubleshooting. All use **Paramiko** for SSH and run from a Windows workstation (`pip install paramiko`).

Connection Targets

Target	IP	User	Purpose
SDDC Manager	192.168.1.241	vcf	API gateway, database access (su to root)
NSX Node	192.168.1.71	root	Direct NSX service management
NSX VIP	192.168.1.70	admin	NSX cluster API (via curl from SDDC Mgr)

F.1 Quick Reference Card

Scenario	Script
Is everything healthy?	<code>python quick_status.py</code>
NSX slow after boot?	<code>python nsx_monitor.py</code>
Credential operation failed?	<code>python check_remediate_error.py</code>
Need to update NSX password?	<code>python nsx_cred_update.py</code>
NSX CPU overloaded?	<code>python nsx_slim.py</code>
Put NSX services back?	<code>python nsx_restart_all.py</code>
Clear stale DB locks?	<code>python clear_locks.py</code>
Fix stuck tasks in DB?	<code>python fix_stuck_tasks.py</code>
Full cascade fix?	<code>python full_remediate_fix.py</code>
System clean after fix?	<code>python final_check.py</code>

F.2 Scripts by Category

Status & Health Checks (Read-Only):

Script	Connects To	What It Does
<code>quick_status.py</code>	SDDC Manager	Start here. NSX status, VIP health, resource locks, notifications, credentials
<code>final_check.py</code>	SDDC Manager	Lightweight: notifications and resource locks only
<code>diag.py</code>	localhost	DNS resolution, TCP 443 connectivity, ARP/routing from Windows host
<code>nsx_monitor.py</code>	NSX Node	Polls cluster status + load avg every 60s for 10 iterations

NSX Diagnostics (Read-Only):

Script	Connects To	What It Does
<code>nsx_check.py</code>	SDDC Manager	Tests both NSX VIP and direct node connectivity — diagnoses VIP failover issues
<code>nsx_diag.py</code>	NSX Node	Top CPU consumers, disk space, service health via API, catalina errors
<code>nsx_resource_check.py</code>	SDDC Manager	NSX clusters, credentials, warnings, DB resource state
<code>sddc_nsx_status.py</code>	SDDC Manager	Compares SDDC Manager's NSX status vs actual NSX VIP cluster status

Credential Operations:

Script	Modifies	What It Does
<code>nsx_cred_update.py</code>	Yes	Full workflow: health checks, lists credentials, updates admin API, monitors 200s
<code>nsx_retry_when_ready.py</code>	Yes	Waits up to 15 min for NSX API, then submits update with 450s monitoring
<code>check_disconnected.py</code>	No	Inspects all credential objects for connection status fields
<code>check_remediate_error.py</code>	No	Failed task details with full error messages, NSX connectivity test, log search

NSX Service Management:

Script	Action	What It Does
<code>nsx_slim.py</code>	Stops	Stops 5 non-essential services to free CPU during boot storm

<code>nsx_restart_all.py</code>	Starts	Restarts all services stopped by <code>nsx_slim.py</code>
<code>nsx_fix_svc.py</code>	Restarts	Restarts search, nsx-sha, nsx-appl-proxy, validates health

Database Fixes (Modify SDDC Manager PostgreSQL):

Script	What It Does
<code>clear_locks.py</code>	Fixes NSX status (ACTIVATING/ERROR → ACTIVE), clears <code>lock</code> table, restarts operationsmanager
<code>fix_stuck_tasks.py</code>	Marks stuck <code>task_metadata</code> as resolved, clears <code>task_lock</code> , fixes <code>execution_to_task</code> orphans
<code>full_remediate_fix.py</code>	Complete cascade fix: NSX health check + DB fix (status + locks + tasks) + service restart
<code>find_pg_pass.py</code>	Searches for PostgreSQL password in config files (read-only)
<code>get_task.py</code>	Retrieves task details by ID with subtask errors (edit <code>task_id</code> before running)

WARNING: Do not run credential update scripts if NSX status is not **ACTIVE** in SDDC Manager or **STABLE** at the VIP. A failed update creates stale locks and stuck tasks, requiring database repair.

F.3 Diagnostic Escalation Path

```
python quick_status.py      # 1. Overall health
python nsx_check.py         # 2. VIP + node connectivity
python nsx_diag.py          # 3. Performance & services
python sddc_nsx_status.py   # 4. SDDC Manager vs NSX sync
```

F.4 Service Recovery Sequence

```
python nsx_slim.py          # Free CPU (if load > 30)
# Wait for load to drop below 15
python nsx_restart_all.py   # Bring services back
python nsx_check.py         # Verify cluster health
```

F.5 Troubleshooting Decision Tree

Problem: Credential operation failed

```
|
|-- python quick_status.py
|   |
|   |-- NSX Status = ACTIVATING or ERROR?
|   |   |-- python clear_locks.py (fix DB status + locks)
|   |   |-- python fix_stuck_tasks.py (resolve stuck tasks)
|   |   |-- OR: python full_remediate_fix.py (all-in-one)
|   |
|   |-- NSX VIP returning 503?
|   |   |-- python nsx_diag.py (check load)
|   |   |-- Load > 30? -> python nsx_slim.py (free CPU)
|   |   |-- Wait -> python nsx_monitor.py (track recovery)
|   |
|   |-- All green?
|       |-- python nsx_cred_update.py (retry update)
```

F.6 Database Repair Quick Reference

```
-- Connect: su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

-- Fix NSX status (covers ACTIVATING and ERROR)
UPDATE nsxt SET status = 'ACTIVE' WHERE status != 'ACTIVE';

-- Clear stale locks
DELETE FROM lock;

-- Resolve stuck tasks
UPDATE task_metadata SET resolved = true WHERE resolved = false;
DELETE FROM task_lock;
```

Appendix G: Technical Accomplishments & Highlights

Environment: Dell Precision 7920 (dual Intel Xeon Gold 6140, 192GB RAM, 2x 1TB SSD + 2x 4TB HDD)

Platform: VMware Cloud Foundation 9.0.1 — fully nested in VMware Workstation **Period:** January–February 2026

G.1 Infrastructure Built from Scratch

Component	Details
ESXi Hosts	4 nested ESXi 9.0 hosts (44GB RAM, 8 vCPU each) with nested virtualization
vCenter Server	9.0.1 — deployed via Cloud Builder, embedded PSC
SDDC Manager	9.0.1 — orchestrates full VCF lifecycle
NSX Manager	9.0 — single-node cluster with VIP, 32GB RAM / 6 vCPU
vSAN	OSA (Original Storage Architecture) — 4-node cluster with disk groups
VCF Operations	Aria Operations 9.0.2 — monitoring and alerting
VCF Ops for Logs	9.0.1 — centralized log collection (vCenter, ESXi, NSX, SDDC Manager)
Fleet Management	Cloud proxy for password management and lifecycle
Aria Lifecycle	Component deployment orchestration
DNS / AD	Windows Server (192.168.1.230) — 14+ forward/reverse records
Offline Depot	Python HTTPS server with TLS 1.2+ for air-gapped bundle management

Full VCF Day 0 → Day 2 lifecycle completed — from bare metal ESXi preparation through Cloud Builder bringup, workload domain configuration, certificate management, monitoring deployment, and ongoing operations.

G.2 Major Problems Diagnosed and Solved

1. NSX Certificate Chain Failure

- VDT flagged NSX certs as FAIL — wildcard SAN missing registered FQDN
- Generated new cert with OpenSSL including VIP, node, and SDDC Manager's registered FQDN
- Applied via NSX API (node first, then cluster VIP)
- Imported into SDDC Manager's VCF trust store AND Java cacerts (two separate keystores)

2. SDDC Manager Credential Cascade Failure

- Credential rotation failed during NSX boot storm → cascading database corruption
- NSX resource stuck in ACTIVATING/ERROR state in `platform.nsxt` table
- Stale exclusive locks in `platform.lock` blocking all operations; 47 unresolved tasks in `platform.task_metadata`
- Fixed by direct PostgreSQL database repair — mapped the database schema independently (undocumented by Broadcom)

3. SDDC Manager Storage Migration (914GB → Thin)

- Cloned each disk with `vmkfstools -i -d thin` (vCenter wizard can't thin-provision to vSAN)
- Recovered from mid-clone failure on 512GB disk

4. vMotion Ghost Setting Failure

- Setting invisible in vCenter UI and VMX file — only found in VM runtime logs
- Fix: explicitly set `vhv.enable = "FALSE"` in VMX

5. NSX Boot Storm Resource Management

- NSX Manager load average >100 on 6 cores for 30-60 min after power-on
- Created scripts to stop/restart non-essential services
- Established 32GB RAM / 6 vCPU as minimum viable for nested NSX

6. VCF Operations for Logs Certificate Mismatch

- Fleet Management self-signed cert generator produces wrong SANs
- Generated custom cert with OpenSSL — deployment succeeded

7. SDDC Manager Deployment Loop

- SDDC Manager repeatedly deployed, failed, and deleted VCF Operations VM
- Established sequencing pattern: wait for automated task to fail, then redeploy manually

8. VCF Account Lockout Recovery

- SDDC Manager uses `faillock` (not `pam_tally2`)
- Documented GRUB recovery procedure for when ALL accounts are locked

9. NSX Manager Memory Escalation

- 16GB: kernel OOM → 24GB: crashes under load → 32GB: stable
- Determined that adding more vCPU is counterproductive (co-scheduling overhead)

10. VDT Compliance Remediation

- Fixed NFS mount ownership, NSX certificate SANs and trust stores
- Re-ran VDT — all checks PASS

G.3 Automation & Tooling Built

20 Python Diagnostic Scripts — Remote SSH-based diagnostic toolkit using Paramiko:

Category	Scripts	Purpose
----------	---------	---------

Health monitoring	<code>quick_status.py</code> , <code>final_check.py</code> , <code>nsx_monitor.py</code>	Real-time environment health
NSX diagnostics	<code>nsx_check.py</code> , <code>nsx_diag.py</code> , <code>nsx_resource_check.py</code>	NSX cluster, services, performance
Credential operations	<code>nsx_cred_update.py</code> , <code>nsx_retry_when_ready.py</code>	Automated credential update with health checks
Database repair	<code>clear_locks.py</code> , <code>fix_stuck_tasks.py</code> , <code>full_remediate_fix.py</code>	PostgreSQL cascade failure repair
Failure analysis	<code>check_remediate_error.py</code> , <code>sddc_nsx_status.py</code>	Deep error diagnosis
Service management	<code>nsx_slim.py</code> , <code>nsx_restart_all.py</code> , <code>nsx_fix_svc.py</code>	NSX service load management

Offline Depot Infrastructure — Python HTTPS server with TLS 1.2+ for air-gapped bundle delivery.

G.4 Documentation Produced

Document	Pages	Content
VCF9 Lab Setup Guide	~45	Complete 9-phase deployment guide with troubleshooting
Troubleshooting Handbook	~65	10 sections covering every failure mode encountered
Operations Configuration Handbook	~55	16-phase post-deployment config guide, 19 known issues
Command Reference	~25	28-section quick reference organized by topic
Interview Cheat Sheet	~10	8-section printable interview prep
Offline Depot Handbook	~15	Air-gapped depot setup and management
Master Bible	~100	Consolidated reference across all topics
Diagnostic Scripts Cheatsheet	~5	Quick reference for all 20 scripts
SDDC Manager API Handbook	~25	18-section REST API reference with authentication, endpoints, Python scripts

All 10 documents available in Markdown, PDF, and HTML formats.

G.5 Database-Level Expertise

Mapped SDDC Manager's internal PostgreSQL schema (undocumented by Broadcom):

Database	Key Tables	Purpose
platform	nsxt	NSX cluster resource status (ACTIVE/ACTIVATING/ERROR)
platform	lock	Exclusive operation locks
platform	task_metadata	Task resolution tracking (resolved boolean)
platform	task_lock	Task-to-lock associations
operationsmanager	task (column: state)	Operation tasks
operationsmanager	execution (column: execution_status)	Execution tracking
operationsmanager	processing_task	Active processing queue

G.6 Undocumented by Broadcom — 35 Discoveries

The following issues have no official Broadcom KB articles, documentation, or known workarounds. All were discovered through independent lab investigation. Each entry includes the exact resolution — no guessing required.

Full Reference: See *VCF-Undocumented-Issues-Reference.pdf* for the complete copy-paste-ready resolution steps, OpenSSL configs, SQL queries, and API commands for all 35 issues.

Database & Credential Operations (7)

#	Discovery	Impact	Resolution
1	SDDC Manager PostgreSQL schema — table names, column names, relationships all unmapped	Cannot troubleshoot credential failures without schema knowledge	<pre>ssh vcf@sddc-manager.lab.local → su - → sudo -u postgres psql -h 127.0.0.1 -d platform → SELECT table_name FROM information_schema.tables WHERE table_schema='public'; Key tables: nsxt , lock , task_metadata , task_lock</pre>
2	Credential cascade failure mechanism — failed rotation leaves NSX stuck in ACTIVATING, stale locks, unresolved tasks	All future credential ops blocked; no	Must fix all 3 tables in sequence: nsxt status → lock table → task_metadata resolved flag (see Issue #4)

		Broadcom procedure exists	
3	API cannot cancel stuck tasks — returns <code>TA_TASK_CAN_NOT_BE_RETRIED</code> , DELETE returns HTTP 500	Database repair is the only fix path	Direct PostgreSQL repair required — API has no mechanism to fix stuck tasks. See Issue #4 for full procedure
4	6-step PostgreSQL repair procedure — must fix nsxt status + locks + tasks together in sequence	Partial fix still fails; all three tables participate in prevalidation	Step 1: Edit <code>/opt/vmware/vcf/commonsvcs/conf/pg_hba.conf</code> — add <code>host all all 127.0.0.1/32 trust</code> above existing lines → <code>systemctl restart postgres</code> . Step 2: <code>sudo -u postgres psql -h 127.0.0.1 -d platform → UPDATE nsxt SET state='ACTIVE' WHERE state='ACTIVATING';</code> Step 3: <code>DELETE FROM lock;</code> Step 4: <code>UPDATE task_metadata SET resolved=true WHERE resolved=false;</code> Step 5: <code>DELETE FROM task_lock;</code> Step 6: <code>systemctl restart operationsmanager</code> → Revert <code>pg_hba.conf</code> trust line → <code>systemctl restart postgres</code>
5	PostgreSQL access requires TCP — must use <code>-h 127.0.0.1</code> (Unix sockets don't work)	<code>psql</code> without <code>-h</code> flag silently fails	Always use: <code>sudo -u postgres psql -h 127.0.0.1 -d platform</code>
6	Database column naming inconsistencies — <code>state</code> not <code>status</code> , <code>resolved</code> boolean not <code>status</code> enum	Wrong column names = wrong queries = no fix	Use <code>SELECT column_name, data_type FROM information_schema.columns WHERE table_name='nsxt';</code> to discover correct column names before writing queries
7	Password not discoverable — must use <code>trust</code> auth workaround in <code>pg_hba.conf</code>	No documented method to obtain PostgreSQL password	Edit <code>/opt/vmware/vcf/commonsvcs/conf/pg_hba.conf</code> → add <code>host all all 127.0.0.1/32 trust</code> as first host line → <code>systemctl restart postgres</code> → connect without password → revert after use

NSX in Nested/Resource-Constrained Environments (6)

#	Discovery	Impact	Resolution
---	-----------	--------	------------

8	32GB RAM / 6 vCPU minimum — Broadcom docs say 16GB; actual: 16GB=OOM, 24GB=crashes, 32GB=stable	Under-provisioned NSX cascades into all VCF operations	Power off NSX VM → Edit Settings → set RAM to 30-32GB, vCPU to 6 → Power on. In VMware Workstation: edit .vmx file
9	Boot storm load >100 on 6 cores for 30-60 min is normal; VIP offline until settled	Credential ops during boot storm trigger cascade failure	Wait 30-60 minutes after all VMs power on. Monitor: <code>ssh admin@192.168.1.71 → get cluster status</code> . Do NOT attempt credential operations until cluster status = STABLE
10	Adding more vCPU is counterproductive — co- scheduling overhead increases load	Intuitive fix actually makes it worse	Keep NSX at 6 vCPU. Reduce contention by staggering VM startups and powering off non-essential VMs during boot
11	Services take 10-15 min to stabilize after restart; API returns error 101 during stabilization	Premature API calls fail and can trigger retries	After <code>restart service manager / restart service proton</code> , wait 15 minutes before any API calls. Verify: <code>get cluster status</code> → wait for STABLE
12	NSX admin CLI for DNS/NTP — <code>set name-servers / set ntp-servers</code> , NOT the UI	UI settings don't persist in some nested configs	<code>ssh admin@192.168.1.71 → set name- servers 192.168.1.5 → set ntp- servers 192.168.1.5 → get name- servers / get ntp-servers</code> to verify
13	TEP on vmk0 — NSX 9.0 "Use VMkernel Adapter" reuses vmk0 as TEP (new in 9.0)	Eliminates need for dedicated TEP VLAN in nested environments	During host transport node config in NSX, select "Use VMkernel Adapter" → choose vmk0. No additional VLAN or vmk needed

Certificate Management (5)

#	Discovery	Impact	Resolution
14	NSX cert SAN must include SDDC Manager's registered FQDN (<code>nsx- manager.lab.local</code>)	VDT fails SAN check; SDDC Manager loses trust in NSX	Create OpenSSL config with <code>[alt_names]</code> section: <code>DNS.1=nsx-manager.lab.local , DNS.2=nsx- vip.lab.local , IP.1=192.168.1.71 , IP.2=192.168.1.70 → openssl req -new -nodes - keyout nsx.key -out nsx.csr -config nsx-cert.cnf → openssl x509 -req -days 825 -in nsx.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out nsx.crt -extensions v3_req -extfile nsx-cert.cnf → Import via NSX API using Python for PEM escaping</code>

15	Two separate trust stores — VCF common services + Java cacerts must both be updated	KB 316056 is incomplete; missing either import = VDT failure	Trust store 1: <code>ssh vcf@sddc-manager.lab.local → /opt/vmware/vcf/commonsvcs/utility/bin/certool --importcert --cert=ca.crt</code> Trust store 2: <code>/usr/java/jre-vmware-17/bin/keytool -importcert -alias nsx-ca -file ca.crt -keystore /usr/java/jre-vmware-17/lib/security/cacerts -storepass changeit -noprompt → systemctl restart domainmanager operationsmanager</code>
16	Fleet Management cert generator produces wrong SANs	Precheck fails: "hosts in the certificate doesn't match"	Generate cert manually: create OpenSSL config with <code>DNS.1=fleet.lab.local, IP.1=192.168.1.78 → openssl req -new -nodes -keyout fleet.key -out fleet.csr -config fleet-cert.cnf → openssl x509 -req -days 825 -in fleet.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out fleet.crt -extensions v3_req -extfile fleet-cert.cnf → Upload via Fleet UI: Settings → Certificate → Import</code>
17	VCF Ops for Logs cert generator — same SAN mismatch	Identical pattern to Fleet Management; same OpenSSL workaround	Same procedure as #16 but with Logs hostnames in the OpenSSL config SANs. Upload via Logs appliance UI
18	Shell can't handle PEM escaping — must use Python for JSON cert payload to NSX API	curl with inline PEM breaks on newlines; no documented alternative	Use Python: <code>cert_pem = open('nsx.crt').read() → key_pem = open('nsx.key').read() → payload = json.dumps({"pem_encoded": cert_pem + key_pem}) → requests.post(url, headers=headers, data=payload, verify=False)</code>

VCF Operations 9.x Changes (6)

#	Discovery	Impact	Resolution
19	Adapter log paths changed — <code>/storage/log/vcops/log/adapters/</code> (legacy path doesn't exist)	Cannot find logs for adapter troubleshooting	Use: <code>ls /storage/log/vcops/log/adapters/</code> then <code>tail -f /storage/log/vcops/log/adapters/<adapter name>/adapter.log</code>
20	JRE path changed — <code>/usr/java/jre-vmware-17/</code> (legacy <code>jre-vmware</code> doesn't exist)	Cannot import certs into	Use: <code>/usr/java/jre-vmware-17/bin/keytool importcert -alias <alias> -file cert -keystore /usr/java/jre-vmware-</code>

		correct truststore	17/lib/security/cacerts -storepass changeit -noprompt
21	Two separate NSX adapters — VCF section uses VIP, "Aria Admin" uses node FQDN	Both need credentials; Aria Admin works when VIP is down	Update both adapters in VCF Operations UI: Administration → Solutions → NSX → Edit cre for both instances. Aria Admin adapter uses <code>manager.lab.local</code> , VCF adapter uses <code>nsx.vip.lab.local</code>
22	System Managed Credential ROTATE doesn't work for NSX	Must uncheck and set manually	In Fleet UI: Settings → Password Management find NSX entries → uncheck "System Managed manually set the password → Save
23	SSH enable via Admin UI only — console/systemctl won't work	Cannot SSH for troubleshooting without Admin UI access	Navigate to <code>https://vcf-ops.lab.local/admin</code> → login as admin → Administration → SSH → Enable. Cannot be done from console or systemctl
24	Health adapter silently fails on stale SDDC Manager credential; reboot required	UI stop/start insufficient; must full reboot appliance	Update the credential in VCF Operations UI first then: <code>ssh root@192.168.1.77</code> → <code>reboot</code> 10-15 minutes for full restart. UI adapter stop is NOT sufficient

Infrastructure & Platform (4)

#	Discovery	Impact	Resolution
25	vCenter can't thin-provision to vSAN — migration wizard keeps thick provisioning	Must use <code>vmkfstools -i -d thin</code> per disk (914GB → 108GB)	SSH to ESXi host → <code>vmkfstools -i "/vmfs/volumes/source/vm/disk.vmdk" -d thin "/vmfs/volumes/vsanDatastore/vm/disk.vmdk"</code> per disk. Update .vmx to point to new paths. Register new VM in vCenter
26	vhv.enable ghost setting persists in VM runtime even when absent from VMX file	vMotion fails; must explicitly set <code>FALSE</code> (removing line is not enough)	Power off VM → Edit Settings → VM Options → Advanced → Configuration Parameters → Add <code>vhv.enable = FALSE</code> . Or edit .vmx: add <code>vhv.enable = "FALSE"</code> explicitly
27	Hot vMotion fails in nested environments — memory convergence timeout	Must use cold migration as fallback	Power off VM → right-click → Migrate → select destination host → complete wizard. Hot migration will time out in nested environments due to memory convergence issues

28	VDT not pre-installed on SDDC Manager — must download from KB 344917	Cannot run health checks without manual download	<code>ssh vcf@sddc-manager.lab.local</code> → download VDT from Broadcom KB 344917 → <code>chmod +x vdt-*</code> → <code>./vdt --domain MANAGEMENT</code>
----	---	--	---

Crash Recovery & VCF Operations Suite-API (7) — Discovered March 2026

#	Discovery	Impact	Resolution
29	Suite-API uses vRealizeOpsToken auth header — not Bearer or VMware like every other VMware API	All API calls fail 401 if using standard Bearer format	Always use: <code>Authorization: vRealizeOpsToken</code> POST <code>https://192.168.1.77/suite-api/auth/roles</code> <code>Type: application/json</code> -d '{"username":"admin","password":"Success01!0909!"}'
30	Permissions API requires single JSON object — not wrapped in array or permissions key	Returns "Role with name: null" with no useful error	Use: <code>curl -sk -X PUT "https://192.168.1.77/suite-api/permissions" -H "Authorization: vRealizeOpsToken" -d '{"roleName":"Administrator","allowAllInstances":[]}'</code>
31	Super admin admin user always shows roleNames: [] — this is by design, not a bug	Wastes time trying to "fix" role assignment	No fix needed — this is by design. The admin user assigns roles to this account
32	SDDC Manager domainmanager port 7200 is HTTP (not HTTPS)	<code>curl https://localhost:7200/h</code> fails with confusing "wrong version number"	Use HTTP: <code>curl http://localhost:7200/h</code> Manager API on port 443 is HTTPS
33	NSX adapter credential fields must be uppercase — USERNAME not USER	Fails with "USERNAME is mandatory"; no docs specify field names	Use exact field names: <code>{"name": "USERNAME", "value": "Success01!0909!"}</code>
34	Gemfire cache takes 5-10 min after cluster init — roles/users don't appear immediately	Admins conclude data is missing and take unnecessary action	Wait 5-10 minutes after cluster initialization complete "Authorization: vRealizeOpsToken \$TOKEN" <code>api/api/auth/roles</code> — roles will appear on

35	HSQLDB reset required after unclean shutdown — no automatic recovery for INITIALIZATION_FAILED	VCF Operations completely non-functional; manual fix only	<pre>ssh root@192.168.1.77 → systemctl st watchdog → cp /storage/db/casa/webapp casa.db.script: change "initialization_sta "initialization_state":"NONE" → > /storage/db/casa/webapp/hsqlldb/casa.d hashed_password → systemctl start vmwa curl -sk -X POST https://localhost/c:</pre>
----	---	---	---

G.7 Technical Skills Demonstrated

VMware Stack: VCF 9.0.1, SDDC Manager, NSX 9.0, vSAN OSA, vCenter 9, ESXi 9, VCF Operations, Aria Lifecycle

Infrastructure: Nested virtualization architecture, vSAN disk groups, NSX overlay networking (GENEVE, TEP, transport zones), certificate lifecycle, offline depot management

Troubleshooting: Root cause analysis through cascading failures, SDDC Manager API diagnostics, PostgreSQL database-level repair, log analysis across 6+ component log paths, VDT compliance remediation

Automation: Python/Paramiko remote diagnostics, ovftool CLI deployments, OpenSSL certificate generation, REST API scripting (NSX, SDDC Manager, vCenter)

Linux/DB: PostgreSQL administration (pg_hba.conf, trust auth, SQL repair), systemctl service management, SSH access patterns, keystore management (keytool), faillock account recovery

Documentation: 13 comprehensive technical documents (~430 pages total), all in Markdown/PDF/HTML with professional styling

G.8 Interview Verbal Summary (60–90 seconds)

Use this as your opening when asked "Tell me about your VCF experience":

"Over the past two months, I built a full VMware Cloud Foundation 9.0.1 environment from scratch — four nested ESXi hosts, vCenter, SDDC Manager, NSX, vSAN, and the full VCF Operations stack — all running nested inside VMware Workstation on a single Dell Precision workstation.

What made this valuable wasn't just the deployment — it was the troubleshooting. Nested virtualization amplifies every failure mode you'd see in production, and I hit them all. I diagnosed and resolved over ten major platform issues, including an NSX certificate chain failure where the SAN didn't include SDDC Manager's registered FQDN, a credential cascade failure that required direct PostgreSQL database repair because the API literally cannot cancel stuck tasks, and NSX resource management where I had to figure out that 32GB RAM is the minimum viable config through three rounds of OOM crashes.

In total, I cataloged 35 issues that have no official Broadcom documentation — spanning database administration, NSX sizing, certificate management, VCF Operations 9.x changes, platform constraints, and crash recovery. I mapped the SDDC Manager PostgreSQL database schema independently to understand how the platform, lock, and task tables interact during credential operations. I also performed a full disaster recovery after an unplanned Windows Update crash wiped out the entire environment — recovering vSAN, NSX, SDDC Manager, and VCF Operations from scratch. I built 20 Python diagnostic scripts for remote SSH-based troubleshooting and wrote over 430 pages of technical documentation across 13 documents covering deployment, troubleshooting, operations, API reference, disaster recovery, and health checks — all version-controlled and available in multiple formats."

Then let them ask follow-up questions — each of the 10 problems above is a ready-made STAR story, and the 35 undocumented discoveries are grouped by category if they want to drill into specifics.

Appendix H: Interview Cheat Sheet

Target Role: VMware Cloud Foundation Professional Services Consultant

H.1 VCF Experience Narrative

Q: "Tell me about your experience with VMware Cloud Foundation."

"I've built and managed a complete VCF 9.0.1 environment from the ground up — not just clicking through wizards, but handling the full stack end-to-end. That includes the Cloud Builder deployment, SDDC Manager commissioning, ESXi host preparation, vCenter, vSAN OSA configuration, NSX 9.0 overlay networking, and VCF Operations. The entire environment runs nested in VMware Workstation on a Dell Precision 7920 — dual Xeon Gold 6140, 192GB RAM. I've worked through the entire Day 0 through Day 2 lifecycle — initial bring-up, workload domain creation, certificate management, and ongoing operations. In fact, I cataloged 35 separate issues that have no official Broadcom documentation — spanning database internals, NSX sizing, certificate management, and VCF Operations 9.x changes."

Q: "Walk me through a VCF deployment."

"Starting from Day 0: prepare ESXi hosts with proper networking, DNS, NTP, and AD. Deploy the Cloud Builder appliance, fill out the deployment parameter workbook — the Excel sheet that defines every IP, FQDN, password, VLAN. Cloud Builder orchestrates the bring-up: deploys vCenter, configures vSAN, deploys SDDC Manager, and stands up NSX Manager. Post bring-up: certificate replacement, VCF Operations deployment, compliance checks with VDT, and coordinated upgrade sequences."

H.2 Problem Solving Stories

NSX Certificate Story:

"After deploying NSX 9.0, I needed to replace self-signed certs. The SANs had to include not only the NSX node FQDN but also the VIP FQDN that SDDC Manager uses. After generating the cert and applying via NSX API — node first, then VIP — SDDC Manager still couldn't communicate. The issue: two separate trust stores (VCF common services and Java cacerts) both needed the CA cert imported. I documented the entire process as a repeatable procedure."

Credential Cascade Story:

"SDDC Manager's credential rotation for NSX failed and left the entire password management system broken. Every subsequent attempt failed with 'not in ACTIVE state' and 'Unable to acquire resource level lock.' VCF Operations showed two accounts disconnected.

The root cause was a cascading failure: the rotation failed because NSX was unreachable during a boot storm — load average over 100 on 6 cores. That left the NSX cluster stuck in ACTIVATING state in PostgreSQL, plus 47 unresolved tasks and stale locks piling up with each UI retry. I tried the API first — PATCH returned 'TA_TASK_CAN_NOT_BE_RETRIED', DELETE returned HTTP 500. The API has no mechanism to fix this.

So I went to PostgreSQL directly. Mapped the database schema myself — none of this is documented by Broadcom. Discovered the key tables: `nsxt` for resource status, `Lock` for exclusive locks, `task_metadata` with a `resolved` boolean for task tracking. The column names aren't what you'd expect — I found them through `information_schema` queries after early scripts failed.

The fix was 6 steps: trust auth workaround, fix nsxt status to ACTIVE, clear lock table, mark task_metadata resolved, clear task_lock, restart operationsmanager. All three tables must be fixed together — they all participate in prevalidation. I built three Python scripts to automate it and documented the full procedure as a repeatable runbook."

vMotion Ghost Setting:

"vMotion was failing with a 'snapshot taken with VHV enabled' error. The setting was invisible in vCenter UI and VMX file — only found in VM runtime logs. Fix: explicitly set `vhv.enable = FALSE` rather than just removing the line."

H.3 Troubleshooting Methodology

Q: "How do you approach troubleshooting?"

"Structured approach: first check relevant logs (SDDC Manager domainmanager/operationsmanager logs, NSX syslog, vSAN health). If logs don't point to the issue, isolate the problem — can SDDC Manager reach NSX? Are certs trusted? Is DNS correct? 80% of VCF issues come down to: certificate trust, DNS resolution, service timing, or stale internal state

in SDDC Manager's database. I use the SDDC Manager API for detailed task status and error payloads the UI hides. When the API isn't enough, I go to PostgreSQL. I've also built 20 Python diagnostic scripts for remote troubleshooting."

H.4 Key Technical Details

VCF Day 0 bring-up sequence:

1. Prepare ESXi hosts: DNS (forward + reverse), NTP, AD, same password on all hosts
2. Deploy Cloud Builder OVA
3. Fill out deployment parameter workbook (Excel)
4. Upload to Cloud Builder UI → Validate → Deploy
5. Cloud Builder deploys: vCenter → vSAN → SDDC Manager → NSX Manager (3-6 hours)
6. Access SDDC Manager at <https://sddc-manager.lab.local/>

VCF upgrade order: SDDC Manager → vCenter → NSX Manager → ESXi (rolling) → vSAN → VCF Operations

SDDC Manager API:

```
# Get auth token
curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}'

# Key endpoints
/v1/credentials      /v1/nsxt-clusters      /v1/tasks/{id}      /v1/resource-locks
```

H.5 Quick Answer Reference

Question	Key Points
What is VCF?	Software-defined DC platform. Integrates vSphere, vSAN, NSX, SDDC Manager.
Mgmt vs workload domain?	Management = infrastructure services. Workload = customer apps.
What does SDDC Manager do?	Orchestrates Day 0/1/2. Single pane for full stack. API for automation.
vSAN ESA vs OSA?	ESA = single pool, NVMe native, no disk groups. OSA = disk groups, SAS/SATA.
NSX transport zones?	Overlay = GENEVE tunneling. VLAN = traditional. VCF creates both during bring-up.

Cert management?	SDDC Mgr generates CSRs. Replacement requires updating trust stores (VCF + Java cacerts).
Password mgmt in VCF 9?	Centralized in VCF Ops Fleet Mgmt. Failed ops can leave stale locks → DB repair.

H.6 Undocumented by Broadcom — 35 Discoveries

Q: "What issues did you find that weren't in the documentation?"

"Across the full deployment lifecycle, I cataloged 35 separate issues with no official Broadcom documentation. These fall into six categories. I've documented the exact resolution for every single one — complete with copy-paste-ready commands, SQL queries, and OpenSSL configs."

Full reference with exact commands: [VCF-Undocumented-Issues-Reference.pdf](#)

Database & Credential Operations (7)

#	What Broadcom Doesn't Tell You	How I Fixed It
1	SDDC Manager's PostgreSQL schema — table names, column names, relationships all unmapped	Mapped schema using <code>information_schema.tables</code> and <code>information_schema.columns</code> queries via <code>psql -h 127.0.0.1 -d platform</code>
2	Credential cascade failure — failed rotation leaves NSX stuck in ACTIVATING, stale locks, unresolved tasks	Direct PostgreSQL repair across 3 tables — must fix all together (Issue #4)
3	API cannot cancel stuck tasks — returns <code>TA_TASK_CAN_NOT_BE_RETRIED</code> ; database repair is the only fix	PostgreSQL: <code>UPDATE task_metadata SET resolved=true + DELETE FROM lock + DELETE FROM task_lock</code>
4	6-step repair procedure — must fix nsxt status + locks + tasks together; partial fix still fails	<code>pg_hba.conf</code> trust auth → fix nsxt state → clear locks → mark tasks resolved → clear <code>task_lock</code> → restart <code>operationsmanager</code>
5	PostgreSQL requires <code>-h 127.0.0.1</code> — Unix sockets don't work	Always: <code>sudo -u postgres psql -h 127.0.0.1 -d platform</code>
6	Column naming inconsistencies — <code>state</code> not <code>status</code> , <code>resolved</code> boolean not status enum	Query <code>information_schema.columns</code> first to discover correct column names
7	Password not discoverable — must use trust auth workaround in <code>pg_hba.conf</code>	Add <code>host all all 127.0.0.1/32 trust</code> to <code>pg_hba.conf</code> → restart <code>postgres</code> → revert after use

NSX in Nested/Resource-Constrained Environments (6)

#	What Broadcom Doesn't Tell You	How I Fixed It
8	32GB RAM / 6 vCPU minimum — Broadcom docs say 16GB; actual: 16GB=OOM, 24GB=crashes, 32GB=stable	Set NSX VM to 30-32GB RAM, 6 vCPU in VMware Workstation .vmx
9	Boot storm load >100 for 30-60 min is normal; VIP offline until settled	Wait 30-60 min after power-on; verify with <code>get cluster status</code> → STABLE
10	Adding more vCPU is counterproductive — co-scheduling overhead	Keep at 6 vCPU; stagger VM startups instead
11	Services take 10-15 min to stabilize; API returns error 101 during stabilization	Wait 15 min after service restart before any API calls
12	DNS/NTP via admin CLI (<code>set name-servers</code>), NOT the UI	<code>ssh admin@nsx</code> → <code>set name-servers 192.168.1.5</code> → <code>set ntp-servers 192.168.1.5</code>
13	TEP on vmk0 — NSX 9.0 "Use VMkernel Adapter" eliminates dedicated TEP VLAN	Select "Use VMkernel Adapter" → vmk0 during transport node config

Certificate Management (5)

#	What Broadcom Doesn't Tell You	How I Fixed It
14	NSX cert SAN must include SDDC Manager's registered FQDN (<code>nsx-manager.lab.local</code>)	OpenSSL config with DNS.1=nsx-manager, DNS.2=nsx-vip, IP.1/IP.2 → generate CSR → sign → import via NSX API with Python PEM escaping
15	Two separate trust stores — VCF common services + Java cacerts; KB 316056 is incomplete	Import CA into both: <code>certool --importcert + keytool -importcert</code> into <code>/usr/java/jre-vmware-17/lib/security/cacerts</code>
16	Fleet Management cert generator produces wrong SANs	Generate manually with OpenSSL using correct SANs → upload via Fleet UI
17	VCF Ops for Logs cert generator — same SAN mismatch pattern	Same OpenSSL manual generation with Logs hostnames → upload via Logs UI
18	Shell can't handle PEM escaping — must use Python for JSON cert payload	Python script: read PEM files → <code>json.dumps({"pem_encoded": cert+key})</code> → POST to NSX API

VCF Operations 9.x Changes (6)

#	What Broadcom Doesn't Tell You	How I Fixed It
19	Adapter log paths changed to <code>/storage/log/vcops/log/adapters/</code> — legacy path gone	Use new path: <code>tail -f /storage/log/vcops/log/adapters/<name>/adapter.log</code>
20	JRE path changed to <code>/usr/java/jre-vmware-17/</code> — legacy <code>jre-vmware</code> gone	Use new path for keytool: <code>/usr/java/jre-vmware-17/bin/keytool</code>
21	Two separate NSX adapters — VCF section uses VIP, "Aria Admin" uses node FQDN	Update credentials on both adapters — VIP adapter and node FQDN adapter
22	System Managed Credential ROTATE doesn't work for NSX — must set manually	Fleet UI → uncheck System Managed → set password manually
23	SSH enable via Admin UI only — console/systemctl won't work	<code>https://vcf-ops.lab.local/admin</code> → Administration → SSH → Enable
24	Health adapter silently fails on stale credential; full reboot required	Update credential in UI → <code>ssh root@vcf-ops</code> → <code>reboot</code> (stop/start insufficient)

Infrastructure & Platform (4)

#	What Broadcom Doesn't Tell You	How I Fixed It
25	vCenter can't thin-provision to vSAN — must use <code>vmkfstools -i -d thin</code> per disk	SSH to ESXi → <code>vmkfstools -i source.vmdk -d thin dest.vmdk</code> per disk → update .vmx
26	<code>vhv.enable</code> ghost setting persists in VM runtime — must explicitly set <code>FALSE</code>	Add <code>vhv.enable = "FALSE"</code> to .vmx explicitly — removing the line is NOT enough
27	Hot vMotion fails in nested environments — use cold migration	Power off VM → Migrate → select destination host (hot migration times out)
28	VDT not pre-installed — must download from KB 344917	Download from Broadcom KB 344917 → <code>chmod +x vdt-*</code> → <code>./vdt --domain MANAGEMENT</code>

Crash Recovery & Suite-API (7) — Discovered March 2026

#	What Broadcom Doesn't Tell You	How I Fixed It
29	Suite-API uses <code>vRealizeOpsToken</code> auth	Authorization: <code>vRealizeOpsToken <token></code> for all Suite-API calls

	header — not <code>Bearer</code>	
30	Permissions API requires single JSON object — not array	<code>{"roleName": "Administrator", "allowAllObjects": true}</code> — no wrapper
31	Super admin <code>admin</code> shows <code>roleNames: []</code> — by design	No fix needed — implicit full access by design
32	SDDC Manager domainmanager port 7200 is HTTP not HTTPS	<code>curl http://localhost:7200/health</code> — NOT https
33	NSX adapter credential fields must be uppercase	<code>{"name": "USERNAME", "value": "admin"}</code> and <code>{"name": "PASSWORD", "value": " ... "}</code>
34	Gemfire cache takes 5-10 min after cluster init	Wait 5-10 min; roles/users populate after Gemfire loads
35	HSQLDB reset required after unclean shutdown	Stop services → edit <code>casa.db.script</code> (<code>FAILED</code> → <code>NONE</code>) → clear log → restart → <code>curl -X POST ... /casa/cluster/init</code>

H.7 Closing Questions to Ask

1. "What does a typical engagement look like — greenfield, upgrades, or a mix?"
2. "How does the team handle knowledge sharing? Is there a documentation culture?"
3. "What's the most common challenge your consultants face in customer environments?"
4. "What does the ramp-up period look like for new consultants?"

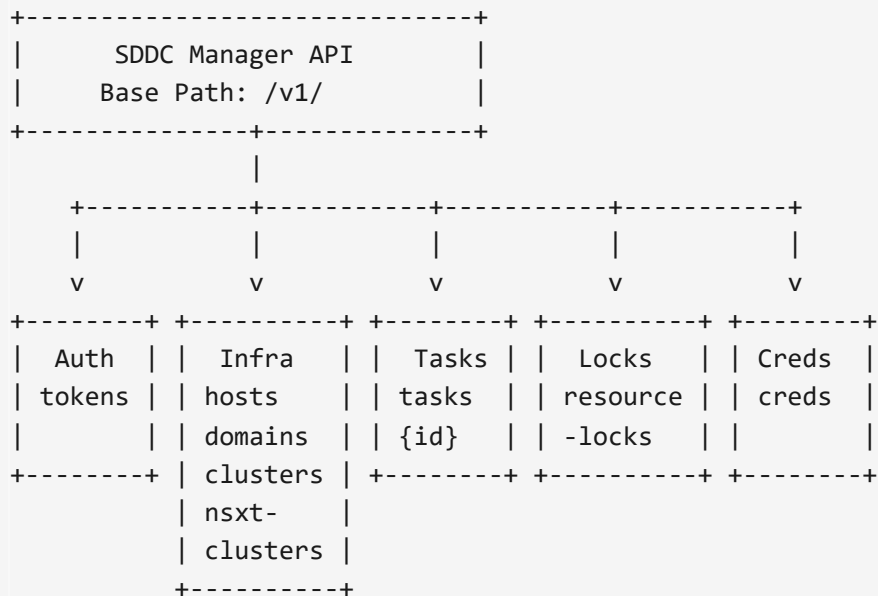
Appendix I: SDDC Manager REST API Reference

Appendix I: SDDC Manager REST API Reference

Condensed reference from the full SDDC Manager REST API Handbook. For complete endpoint details, Python scripts, Postman collection, and lab-tested workflows, see `VCF-SDDC-Manager-API-Handbook.md`.

I.1 API Architecture & Authentication

Base Path: `https://sddc-manager.lab.local/v1/` **Authentication:** Bearer Token (JWT) via `POST /v1/tokens` **Pattern:** Async — mutating operations return task IDs, poll `/v1/tasks/{id}` for status



Token Extraction:

```
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
-H "Content-Type: application/json" \
-d '{"username":"administrator@vsphere.local","password":"Success01!0909!!"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")
```

Access token lifetime	60 minutes
Refresh token lifetime	24 hours
Token type	JWT (JSON Web Token)
Required header	Authorization: Bearer <accessToken>
Token refresh	PATCH /v1/tokens with refreshToken.id

I.2 Core Endpoints Reference

Authentication

POST	/v1/tokens	Authenticate and get Bearer token
PATCH	/v1/tokens	Refresh an expired access token

System

GET	/v1/system	System information and version
GET	/v1/system/health	Overall platform health (GREEN / YELLOW / RED)
GET	/v1/system/notifications	Active notifications

Infrastructure Inventory

GET	/v1/hosts	List all ESXi hosts
POST	/v1/hosts	Commission new host(s)
DELETE	/v1/hosts/{id}	Decommission a host
GET	/v1/domains	List all workload domains
POST	/v1/domains	Create a new workload domain
GET	/v1/clusters	List all clusters

POST	/v1/clusters	Create a new cluster
PATCH	/v1/clusters/{id}	Expand/shrink a cluster
GET	/v1/vcenters	List all vCenter instances
GET	/v1/nsxt-clusters	List all NSX-T clusters

Lifecycle & Bundles

GET	/v1/bundles	List available update bundles
POST	/v1/bundles	Download a bundle
GET	/v1/upgradables	List upgradable components
POST	/v1/upgrades	Start an upgrade operation

I.3 Task Lifecycle & Async Operations

Most mutating operations (credential rotations, upgrades, deployments) return a task ID. Poll until **SUCCESSFUL** , **FAILED** , or **CANCELLED** .

GET	/v1/tasks	List all tasks (filter: ?status=IN_PROGRESS)
GET	/v1/tasks/{id}	Get task details with sub-tasks and errors
PATCH	/v1/tasks/{id}	Attempt to cancel a task

Task polling pattern:

```
TASK_ID="<task-id>"
while true; do
  STATUS=$(curl -sk -H "Authorization: Bearer $TOKEN" \
    https://sddc-manager.lab.local/v1/tasks/$TASK_ID \
    | python3 -c "import sys,json; print(json.load(sys.stdin)['status'])")
  echo "$(date +%H:%M:%S) - $STATUS"
  [ "$STATUS" = "SUCCESSFUL" ] || [ "$STATUS" = "FAILED" ] && break
```

```
sleep 30
done
```

Key Discovery: The API returns `TA_TASK_CAN_NOT_BE_RETRIED` for stuck tasks. `DELETE /v1/tasks/{id}` returns HTTP 500. When the API cannot cancel stuck tasks, direct PostgreSQL database repair is the only option — see Section 7.2.6.

I.4 Credentials & Resource Locks

Credentials API

GET	<code>/v1/credentials</code>	List all stored credentials
PUT	<code>/v1/credentials</code>	Update, rotate, or remediate credentials

Credential operation types:

UPDATE	Sync SDDC Manager's stored password with current password on target
ROTATE	Generate new password and push to both SDDC Manager and target
REMEDiate	Re-attempt a failed credential operation

WARNING: If a credential operation fails mid-flight (e.g., NSX unreachable during boot storm), it triggers a cascade failure. See Section 7.2.6 for the 6-step PostgreSQL repair procedure.

Resource Locks

GET	<code>/v1/resource-locks</code>	List active resource locks

Stale locks from failed operations block all subsequent operations. The API provides **no way to force-release locks**. Fix requires direct database cleanup: `DELETE FROM lock` in the `platform` database.

I.5 Python Automation Examples

Three ready-to-use Python scripts are provided in the full API Handbook:

Full API Client	Queries all key endpoints in one pass (system, health, credentials, NSX, tasks, locks, hosts, domains)
Credential Status Checker	Tabular display of all credentials with type, resource, and status
Task Monitor	Polls a specific task ID every 30 seconds until completion, displays errors on failure

All scripts use `requests` , `urllib3.disable_warnings()` , and `verify=False` for self-signed certs.

I.6 Troubleshooting API Issues

Common Errors and Fixes

<code>TA_TASK_CAN_NOT_BE_RETRIED</code>	Stuck task	DB: <code>UPDATE task_metadata SET resolved = true</code>
Unable to acquire resource level lock(s)	Stale locks	DB: <code>DELETE FROM lock</code> in platform DB
Resources [X] are not in ACTIVE state	NSX stuck	DB: <code>UPDATE nsxt SET status = 'ACTIVE'</code>
HTTP 401	Token expired	Re-authenticate via <code>POST /v1/tokens</code>
HTTP 409	Resource locked	Check <code>/v1/resource-locks</code> , wait or clear DB locks
Connection refused	Services down	SSH: <code>systemctl restart vcf-services</code>

API Log Locations (SSH to SDDC Manager)

```

/var/log/vmware/vcf/operationsmanager/operationsmanager.log # Credential ops
/var/log/vmware/vcf/domainmanager/domainmanager.log         # Domain/cluster ops
/var/log/vmware/vcf/lcm/lcm.log                               # Lifecycle/upgrade ops
/var/log/vmware/vcf/commonsvcs/commonsvcs.log               # Auth/token issues

```

Upgrade Order (Always Follow This Sequence)

1	SDDC Manager	Orchestrates all other upgrades
2	vCenter Server	Required before ESXi upgrades
3	NSX Manager	Required before host network changes
4	ESXi Hosts	Rolling upgrade, one host at a time
5	vSAN	After all hosts are upgraded
6	VCF Operations	Last — depends on all infrastructure

Full reference: See [VCF-SDDC-Manager-API-Handbook.md](#) for complete endpoint documentation, Postman collection (12 pre-built requests), and 3 lab-tested workflows (Full Health Check, Credential Update for NSX, Diagnose Credential Cascade Failure).

(c) 2026 Virtual Control LLC. All rights reserved.